

Redesigning the Home Button

How one fixed mark at the bottom of the screen replaced a sidebar, a mobile dock and a push-drawer — and became a two-deck radial dial that carves itself out of the navigation catalog.

About this record

This is a design record: the reasoning, geometry, motion, material and defect history behind a single control. It documents the RMH hub — the orb docked at the bottom centre of every standard page on rmhstudios.com, and the radial dial that blooms out of it.

WHAT IT IS DRAWN FROM

Everything here is reconstructed from the repository, which is the platform's own stated source of truth for conflicting information. The primary sources are `components/radial/RadialHub.tsx` (`{{stat:hubLines}}` lines), `components/radial/radial.css` (`{{stat:cssLines}}` lines), `components/radial/README.md`, `lib/sidebar-nav.ts`, `docs/design-language.md`, `docs/ui-audit-2026-07-28.md`, and the fifteen commits between `e5003b1` and `feec2c` that built, broke and repaired the control.

HOW THE FIGURES WERE MADE

Every dial diagram in this book is drawn by the production geometry, not by hand. The figure generator carries a faithful port of the component's `RINGS`, `SINGLE_RING`, `INNER_MAX`, `SINGLE_RING_MAX`, `ART_BLEED_DEG` and `ringSector()`, and renders the same annulus sectors the browser clips. A figure therefore cannot quietly disagree with the code it illustrates: change the geometry and the figures change with it.

Code listings are extracted from the working tree at build time, with their real file paths and line numbers. Nothing in a listing is retyped.

A NOTE ON THE LIVE SITE

The live production site was not reachable from the environment this record was compiled in — outbound requests to rmhstudios.com were refused by the network policy at the proxy, not by the site. Every observation about rendered behaviour therefore comes from the code, from the shipped CSS, and from the July 2026 visual audit, which captured the interface across thirteen viewports and three themes and is quoted throughout with its finding identifiers intact.

READING ORDER

Parts I and II are the argument: what a home button is for, what the site had before, and why a radial dial was the answer rather than a hamburger. Parts III to V are the build: geometry, motion, material. Part VI is input and accessibility. Part VII is the defect log — the most useful part for anyone maintaining the thing. Part VIII is verification and open questions. The appendices are reference: annotated source, a full geometry table, the token contract, the commit index and the audit index.

CONVENTIONS

Ring radii are always given as a percentage of the *dial's width*, matching the component, because that is the unit `clip-path` resolves against. The dial is square, so its radius is 50% and the rim sits at `r = 50`. Angles run clockwise from twelve o'clock, which in CSS coordinates is `-90°`.

Contents

PART ONE — THE BUTTON THAT WASN'T THERE	1
1 The job of a home button	2
1.1 Three jobs, usually split across three controls	2
1.2 Why the corner is the wrong place for it	2
1.3 The radial premise	3
1.4 What "always available" has to mean	4
1.5 The claim this record makes	4
2 What the site had instead	5
2.1 The desktop left rail	5
2.2 The mobile dock	5
2.3 The mobile push-drawer	5
2.4 And no home button at all	6
2.5 What was kept	6
3 Evidence: the July 2026 audit	7
3.1 The matrix	7
3.2 AUD-003: the orb occludes content and collides with chrome	7
3.3 AUD-504: the dial itself	8
3.4 The findings that were not the hub's fault but were its problem	9
4 Goals, non-goals and constraints	10
4.1 Goals	10
4.2 Non-goals	10
4.3 Constraints inherited from the platform	10
4.4 The constraint that mattered most	11
PART TWO — THE CONCEPT	12
5 The orb: one control, two jobs	13
5.1 The two states, as declared	13
5.2 Why the trigger is not also a link	14
5.3 The wrapper that generates no box	14
5.4 What sits around the dial	15
5.5 The desktop rail is not a second implementation	15
6 The material	17
6.1 What the hub takes from the material system	17
6.2 The blur is the menu	17
6.3 Where translucency had to be walked back	18
6.4 The plate under the sectors	19
6.5 Degradation, and where it is handled	19
7 The mark that became the button	20
7.1 Hairline weight is not a style choice	20
7.2 The bloom	21
7.3 The ink problem, and the first version of a mistake	21
7.4 The mark as a loading state	22
PART THREE — GEOMETRY	23
8 From pie to annulus	24
8.1 Four problems with the pie	24
8.2 The band table	24
8.3 The dial's size	25
8.4 Why percentages, and not pixels	26
9 The double deck	27
9.1 The arithmetic of crowding	27

9.2	The split rule, as code	28
9.3	The count is not fixed	28
9.4	Priority, encoded as radius	29
10	Carving one destination	31
10.1	The walk	31
10.2	Why ten steps	32
10.3	The angular gap	32
10.4	Starting at twelve o'clock	33
10.5	The return value is three things at once	33
11	The hole, the mask and the hairlines	34
11.1	The bounds array	34
11.2	The hole	34
11.3	The hairlines	35
11.4	Why a drawn edge rather than a wider gap	36
12	Labels, centroids and budgets	38
12.1	The centroid	38
12.2	The direction the box grows	38
12.3	The budgets, and why they differ	39
12.4	Icon-only below 480px	40
12.5	Where the glyph sits inside the box	40
13	The art bleed	42
13.1	What the filter does to an edge	42
13.2	The compensation	42
13.3	Why not radially	43
13.4	What this chapter is really about	43
PART FOUR — MOTION		45
14	One synchronous motion	46
14.1	Why 500ms	46
14.2	The duration family	46
14.3	The stagger, and what it is applied to	47
14.4	What is not animated	48
14.5	Why there is no animation library here	48
15	The phase machine	50
15.1	The problem closing solves	50
15.2	The CSS side of the same idea	50
15.3	The timer's reduced-motion case	51
15.4	Escape, wheel, and the effects that own them	51
15.5	Focus, on the next frame	52
15.6	The whole machine, in one place	52
16	Counter-rotation, and the bed that flashed black	54
16.1	The gesture	54
16.2	The bug	54
16.3	The fix, and why it is not just a colour change	55
16.4	The uncovered-area arithmetic	56
16.5	What the episode is worth	56
17	The blur, and the degradation ladder	57
17.1	Why 150vmax	57
17.2	What the blur costs	57
17.3	The degradation ladder	57
17.4	What reduced motion actually looks like	58
PART FIVE — MATERIAL AND COLOUR		59
18	Two layers, art and hit	60
18.1	The constraint	60

18.2	The two layers	60
18.3	The property this buys: colour correctness by construction	61
18.4	The active state, on the art layer	62
18.5	Why not one layer with the filter on a pseudo-element	62
19	The metaball filter bank	63
19.1	How it works	63
19.2	The three radii	64
19.3	Where the goo is used, and where it is banned	64
19.4	The trap that is not in the code	64
19.5	Cost, and the gate	65
20	White on white	66
20.1	The idea that caused it	66
20.2	The four instances	66
20.3	What replaced the clever idea	66
20.4	Why this class of bug is so easy to ship	67
21	Tokens, themes and the accent	68
21.1	What the hub consumes	68
21.2	The color-mix discipline	68
21.3	The accent, and where it lands	69
21.4	User themes	69
PART SIX — INPUT AND ACCESS		70
22	Hit testing sixteen wedges	71
22.1	The mechanism	71
22.2	Target sizes, honestly	71
22.3	The pointer metaball, and AUD-503	72
22.4	What the hub does not do with the pointer	72
23	Keyboard and screen reader	73
23.1	The semantics, as shipped	73
23.2	What is missing	73
23.3	Focus visibility, and the one place it is suppressed	73
23.4	Translation	74
24	The caption in the hole	75
24.1	The finding	75
24.2	The hole is the room	75
24.3	Exact complementarity	76
24.4	What it does not fix	76
25	Blocking scroll without a scroll lock	77
25.1	The two techniques, and what they do	77
25.2	What the hub does instead	77
25.3	The provenance, and why the comment is so long	78
25.4	What is deliberately not blocked	78
26	Safe areas and the floating stack	79
26.1	The safe area	79
26.2	The floating-bottom stack	79
26.3	Clearance for content	80
26.4	The flexbox trap in the foot pill	81
26.5	The last-resort guarantee	81
PART SEVEN — THE DEFECT LEDGER		83
27	A chronology	84
27.1	The shape of the five days	84
27.2	The commit that removed a state	85
28	Six patterns	86
28.1	Derived contrast that depends on the layer tree	86

28.2 A tint of a tint	86
28.3 A value whose correctness depends on being covered	86
28.4 Chrome that does not publish its footprint	86
28.5 Text boxes that spill in the constrained direction	86
28.6 Decoration in the same pixels as information	87
28.7 What the ledger says about process	87
 PART EIGHT — VERIFICATION, AND WHAT IS OPEN	 88
29 How it is checked	89
29.1 The automated layer	89
29.2 The manual floor	89
29.3 The audit as an instrument	89
29.4 What a real regression suite would need to assert	90
30 What is still open	91
30.1 Discovery — no first-run hint	91
30.2 Keyboard semantics	91
30.3 Touch targets at 320 pixels	91
30.4 Landscape phones	91
30.5 Right-to-left locales	91
30.6 The parked lens optic	92
30.7 A continuous visual check	92
30.8 What is not on this list	92
 APPENDICES	 93
A Geometry reference	94
B Constants and durations	96
C The navigation catalog	98
D Finding index	99
E Glossary	100

PART ONE

The button that wasn't there

Before the orb, rmhstudios.com had three navigation systems and no home button worth the name. This part is the case for the change: what the control is supposed to do, what the site actually had, what the July audit measured, and the goals the replacement was held to.

-
- 1 The job of a home button
 - 2 What the site had instead

- 3 Evidence: the July 2026 audit
- 4 Goals, non-goals and constraints

CHAPTER ONE

The job of a home button

A home button is the cheapest promise an interface makes: *wherever you are, this returns you to the start*. It is also, on almost every website ever built, the least examined control on the page — a logo in the top-left corner, inherited from print mastheads, kept because nobody could think of a reason to move it.

RMH Studios is not almost every website. The platform is a social feed, eighteen browser games, twelve full-screen applications, a blog and news and library system, a coin economy with paid memberships, and a scoped developer API — all served from one React SSR tier with Node realtime hubs and a Go worker fleet behind it. The navigation catalog in `lib/sidebar-nav.ts` lists sixteen destinations at the top level alone, several of which are themselves hubs over a dozen sub-surfaces. A logo in the corner is not a navigation system for that. It is a bookmark.

So the question that produced the redesign was not "where should the logo go". It was: if a platform this wide has exactly *one* control that is present on every page, at a fixed position, that the user can always reach without scrolling and without remembering where they are — what should that control actually do?

1.1 Three jobs, usually split across three controls

Unpack "home button" on a large site and it is doing at least three separate jobs, and conventional layouts hand each to a different piece of chrome.

Return.

Go to the root. The classic job, and the easiest: one link, one destination. Also the least valuable of the three, because on a signed-in social platform the root is the feed, and the feed is where the user usually already is.

Orient.

Answer "where am I in this thing?" Conventionally the job of a breadcrumb, a highlighted rail item, or an active tab underline. It requires the navigation to be *visible*, which is why it is the first casualty of small screens.

Traverse.

Get to any other top-level destination. Conventionally a sidebar on desktop and a hamburger-plus-drawer on mobile — two different implementations of one idea, which is where drift and duplicated logic come from.

The interesting observation is that these three jobs share a natural anchor. Return is a point. Orientation is a position relative to that point. Traversal is movement away from it. Given a single fixed mark on screen, all three can be expressed as relationships to *that mark* — which is the entire idea the redesign rests on, and the reason it ended up radial rather than rectangular.

One control, at one place, that is simultaneously the destination, the origin and the map.

1.2 Why the corner is the wrong place for it

The top-left logo is a print inheritance with three properties that were virtues on paper and are liabilities on a phone.

It is the furthest point from the thumb. On a 390×844 device held one-handed, the top-left corner is the single

hardest region of the screen to reach; the bottom-centre is the easiest. A control the user is expected to hit repeatedly belongs where the hand already is, and no amount of visual hierarchy compensates for a stretch.

It shares the worst real estate on the page. The top strip is contested by the browser's own chrome — the URL bar, the tab strip, the notch, the dynamic island — and on iOS Safari the top bar collapses and re-expands as you scroll, which means anything anchored there moves relative to the content underneath it.

It is a scroll casualty or a sticky tax. Either the masthead scrolls away, in which case the control is not actually always available, or it is pinned, in which case it permanently spends vertical space that a phone does not have. Pinned headers on a 390×844 screen cost 7–9% of the viewport before any content is drawn.

None of that is news. What is unusual is that this platform was willing to act on it, because it had already accepted a much larger premise: that its *information architecture*, not just its palette, could be radial.

1.3 The radial premise

The design language the site adopted is named **Radial Avant-Garde Glass**, and its own reference document states the premise in one sentence:

The site orbits a central RMH mark; content radiates from the centre.

DOCS/DESIGN-LANGUAGE.MD — THE DESIGN LANGUAGE

Taken seriously, that sentence has consequences well beyond decoration. If content radiates from a centre, the centre must exist as a real, persistent, interactive thing — not a watermark. And if the centre is real, then navigation is not a list of links somewhere else on the page; it is the immediate neighbourhood of the centre. A menu that opens *around* the mark is not a stylistic flourish under that premise. It is the only shape consistent with it.

This is worth stating plainly because radial menus have a bad reputation in web design, and mostly deserve it. They are usually a novelty pinned onto a rectangular application: a circular fan of four icons that exists because it looked good in a dribbble shot, wrapped around a layout that is otherwise a perfectly ordinary grid. Those fail for a predictable reason — the geometry carries no meaning, so the user pays the cost of an unfamiliar shape and receives nothing in exchange.

The dial documented in this book is trying to earn the shape. Whether it fully succeeds is a live question, and Chapter 30 is honest about where it does not. But the shape is load-bearing: the orb's docked position is the mark, the dial's rings encode priority, the sector under your finger is the destination, the hole in the middle is where the label goes when the sectors are too small to hold one, and the whole thing is the same square of screen the mark already occupied.

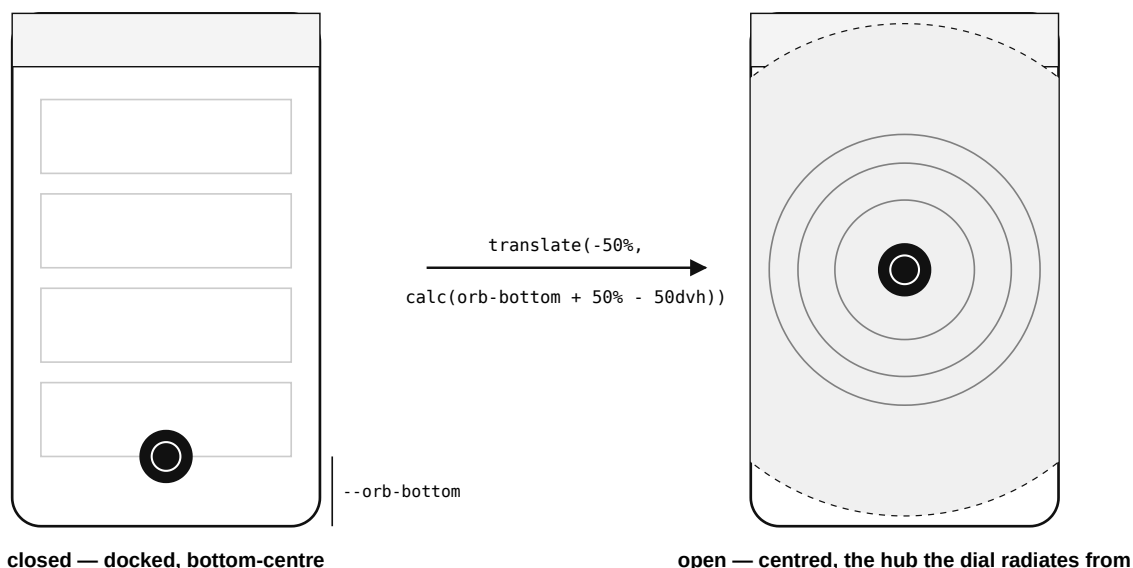


Figure 1.1 — The orb's two states. Docked, it is a home button at the easiest point on the screen to reach. Tapped, it glides to the exact centre of the viewport and becomes the hub the dial radiates from. The same element, the same transform property, one 500ms transition.

1.4 What "always available" has to mean

A persistent navigator makes a stronger promise than a menu bar, and the promise is only worth making if it holds in the cases where navigation usually breaks. The hub is held to five:

1. **Present on every standard page.** Not on the feed only. Every route under the `_site` layout inherits the shell that mounts it, so there is no page where the navigation is somewhere else. Full-screen surfaces — games, the login page, legal pages — are deliberately outside it, and Chapter 4 explains why that is a feature.
2. **Reachable without scrolling.** Fixed position, bottom-centre, above the fold by construction.
3. **Reachable with the safe area respected.** On a device with a home indicator, "bottom of the screen" and "bottom of the usable screen" differ by about 34 logical pixels, and a control that ignores the difference sits inside the system's own gesture strip. Chapter 26 covers what that cost the first implementation.
4. **Never colliding with other floating chrome.** A cookie banner, a mini player, a back-to-top button and a compose button all want the same corner. If the navigator can be buried by a consent bar, it is not always available. This was the single most expensive lesson of the whole project — finding AUD-003, with eight dependent findings.
5. **Legible over whatever is behind it.** A fixed control floats over scrolling content, so text passes underneath it. Either the control is opaque enough to win, or the ground fades in behind it. The hub does the second.

Each of those five is a sentence in a design document and a week of defects in practice. Part VII is the ledger.

1.5 The claim this record makes

Stated as plainly as possible, so it can be argued with: **on a platform with sixteen top-level destinations and a mobile-first mandate, a single fixed mark that expands into a two-deck radial dial is a better navigator than a hamburger drawer, because it collapses three controls into one, puts that one where the thumb already is, and encodes priority in geometry rather than in scroll position.**

The rest of Part I establishes the baseline that claim is measured against.

CHAPTER TWO

What the site had instead

Three navigation systems, each correct in isolation, collectively costing more than they delivered: a desktop left rail, a mobile bottom dock, and a mobile push-drawer that by the end was unreachable code.

It is worth being precise about the "before", because the temptation in a document like this is to caricature it. The pre-radial navigation was not bad work. It was conventional work, built incrementally, and every piece of it was individually defensible. The problem was the sum.

2.1 The desktop left rail

A persistent vertical list of destinations, rendered from `SIDEBAR_NAV`, with live badges for the inbox and for pending admin review. It is the right answer for a wide viewport and it survives to this day: `RadialNavRail` is substantially the same component, still driven by the same catalog, still shown from 1120px up. Nothing about the redesign argued against a rail on a desktop.

What it could not do was scale down. Sixteen items with icons and labels needs roughly 240 logical pixels of width and 600 of height. Below about 1100px that is a third of the screen spent on chrome, so the rail had to disappear — and something else had to take over, which is where the trouble started.

2.2 The mobile dock

`MobileDock` was the phone answer: a fixed bottom bar of four or five primary destinations. This is the industry-standard pattern and it works, for the problem it solves. Its ceiling is arithmetic. A 320px-wide screen divided by a 44px minimum touch target is seven slots, and seven is generous — five is the comfortable maximum with labels. Sixteen destinations do not fit in five slots.

So the dock became a partial view of the navigation: the four the team guessed were most used, plus a fifth slot that opened the rest. That fifth slot is a hamburger by another name, and it produced the third system.

2.3 The mobile push-drawer

A drawer that slid in from the edge, pushing the page sideways, carrying the full sixteen-item list as a scrolling column. It had to block background scroll while open, and the way it did so is the single most valuable piece of engineering to survive the rewrite — so it gets its own chapter later (Chapter 25). The short version: it blocked scroll from its scrim with `touch-action` rather than locking the document, because locking the document clips the page to the visual viewport on iOS and leaves a band of bare background under Safari's floating bottom bar. That finding was established on a real device, and it is the reason the hub's overlay is built the way it is.

The drawer's problem was not its scroll handling. It was that by the time the radial work started, *the drawer was unreachable*. Successive layout changes had removed every path that opened it. It was live code, in the bundle, shipped to every phone, and it could not be opened. Commit `0969119` is titled, with some weariness, *"delete the unreachable mobile push-drawer."*

THE REAL COST OF THREE SYSTEMS

Dead navigation code that nobody notices is a symptom, not the disease. The disease is that the same catalog was rendered three times, by three components, with three sets of breakpoints, three notions of "active", and three chances to drift. A destination added to `SIDEBAR_NAV` appeared in the rail, appeared in the drawer, and silently did not appear in the dock, because the dock's list was a hand-picked subset. Nobody could tell you, from reading one file, what the site's navigation was.

2.4 And no home button at all

Amid three navigation systems, the actual "return to start" control was: a logo in the top-left of the shell header on desktop, and a Home entry inside the dock on mobile. Neither was distinguished from its neighbours; neither was the visual anchor of anything. The site's own mark — a six-fold radial spirograph, one of the more distinctive logos on the platform — appeared at roughly 20 logical pixels tall in a corner, where its filigree resolved into a grey smudge.

That is the honest description of the "before": the brand mark was decoration, the navigation was three overlapping implementations, and the home button was a list item.

2.5 What was kept

The redesign is often described internally as a rewrite, and the shell genuinely was one — `e5003b1`, "*radial front-end rewrite*", is a large commit. But three things were deliberately carried across intact, and it matters that they were, because they are the reason a rewrite of the chrome did not become a rewrite of the site.

1. **The catalog.** `lib/sidebar-nav.ts` did not change shape. The dial consumes exactly the same `SIDEBAR_NAV` array the old rail did, including its stable per-item `ids`, which are the keys users' saved reorder-and-hide preferences validate against. Renaming one is a data migration; the file says so in a comment, and the rewrite honoured it.
2. **The token contract.** Every colour, radius, shadow and duration in the new chrome comes from the same `--site-*` custom properties every theme redefines. The hub was built monochrome, but it was not built *black*; it was built out of tokens that happen to resolve to black in the default theme.
3. **The scroll-blocking technique.** Inherited wholesale from the drawer, complete with the on-device reasoning, which now lives as a comment block in `RadialHub.tsx` and in `radial.css` because the component that learned it was deleted.

The shell nav — `SiteNavigation`, `MobileDock`, `FeedLayout` — was removed. `SiteShell` kept its public `children/overlays` API and became a delegate to `RadialShell`, so every route under `_site` inherited the new chrome without touching a single route file. That is the whole reason a change this visually total was shippable at all.

CHAPTER THREE

Evidence: the July 2026 audit

Two-thirds of the way through the redesign, the whole site was audited across forty routes, thirteen viewports and three themes. The hub came out of it with a P1 systemic finding and its own multi-part defect. This chapter is what the audit measured, because it is the closest thing to an objective account of the control.

The audit — docs/ui-audit-2026-07-28.md — is unusual in that it was written to be actionable by a machine as well as a person. Every finding names the routes, the viewport it was observed at, the evidence capture, the suspected source file with line numbers, the design principle violated, and a concrete fix direction. It is quoted here with its identifiers intact so anything in this chapter can be traced back.

3.1 The matrix

AUDIT VIEWPORT MATRIX — TIER-A ROUTES GOT ALL THIRTEEN

NAME	SIZE	CLASS
m320	320×568	phone, smallest supported
m375	375×667	phone
m390	390×844	phone, iPhone 14 class
m412	412×915	phone, Pixel class
m1and844	844×390	phone, landscape
t768	768×1024	tablet portrait, touch
t1024	1024×768	tablet landscape, touch
l1280	1280×800	small laptop
l1366	1366×768	common laptop
d1536	1536×864	desktop
d1920	1920×1080	desktop
d2560	2560×1440	large desktop
uw3440	3440×1440	ultrawide

Three themes were captured at each: the default (a strict monochrome light palette, referred to in the audit's copy as Daylight), graphite, and high-contrast. Reduced motion was captured separately, as were first-run overlays and a set of scripted interactions — among them, specifically, the hub bloom.

3.2 AUD-003: the orb occludes content and collides with chrome

The headline finding against the hub, rated P1, with eight dependent findings that the audit's own remediation order notes are all healed by fixing it once. Its instances are worth listing in full, because together they are a complete taxonomy of what goes wrong when a fixed control is introduced to a page that was not designed to reserve space for one.

AUD-003 INSTANCES

FINDING WHAT WAS OBSERVED

AUD-102	The orb sat on top of the cookie-consent banner's text at every viewport $\geq 768\text{px}$, on every <code>_site</code> route. The floating-stack coordination that solves exactly this was wrapped in a <code>max-width: 767.98px</code> media query and never ran on wider screens. Consent copy and its "Learn more" link were behind the orb.
AUD-104	The orb covered feed-card text and engagement counters on portrait phones. The feed column and the orb share the same centre axis, and no bottom clearance was reserved. The end of the feed left under 40px of clearance after the last card.
AUD-111	On <code>/pricing</code> , the orb overlapped tier-card feature lists mid-scroll and hid the pinned hero's "Compare tiers" scroll cue entirely — the cue's <code>bottom-8</code> placed it inside the orb's reserved band, so it was never visible at any scroll position.
AUD-203	On <code>/help</code> , the orb sat permanently on the bottom-pinned concierge suggestion chips. With the cookie banner up, the chips were buried entirely.
AUD-105	At 844×390 landscape, top bar + cookie bar + orb left roughly 150px of usable content height — about 38% of the viewport.
AUD-313	Further occlusion instances: the custom-accent hex input at 320px on <code>/settings/appearance</code> ; the "Notification channels & quiet hours" row at 1920px on <code>/settings</code> ; most of the remaining list area on <code>/messages</code> in landscape.
AUD-322	On <code>/shop</code> at 390px, the post-hero landing put the category tab strip directly under the orb, hiding "Avatar Frames".

The pattern is uniform: nothing was wrong with the orb's own rendering. Everything was wrong with the assumption that a page could ignore it. A fixed control is a claim on screen real estate, and a claim that is not published in a way layouts can consume is just a collision waiting for a viewport.

The audit's prescribed fix is the one that shipped, and it is a good template for this class of problem:

AUD-003 FIX DIRECTION, AS WRITTEN

(a) Extend the floating-stack coordination to all widths so banner, FAB and orb never share pixels. (b) Publish the orb's occupied band as a CSS variable — the inset and size tokens already exist — and consume it as bottom clearance in the feed wheel, full-height pages, and pinned-hero cues. (c) Consider a subtle glass scrim behind the orb so text passing beneath it stays legible.

All three landed. The tokens are `--site-hub-orb-size` and `--site-hub-orb-inset`, declared in `globals.css` rather than in the radial stylesheet precisely so the floating stack's reserve and the orb's geometry cannot drift apart; the derived clearance is `--site-floating-reserve`; and the scrim is `.radial-hub__scrim`, a radial gradient of the page background that fades the ground in behind the docked orb and retracts the moment the dial opens. Chapter 26 covers the arithmetic.

3.3 AUD-504: the dial itself

The audit re-captured the hub bloom specifically, and reported on the flagship navigation in four parts — with, unusually for this document, an explicit list of what worked.

AUD-504 — HUB DIAL FINDINGS FROM THE RE-CAPTURED BLOOM

FINDING

a	The mobile dial was entirely icon-only — sixteen unlabelled glyphs as the site's sole navigation . Desktop wedges showed labels; phones did not. The audit's framing: this is recall, not recognition.
b	The account pill's avatar disc painted over the settings gear beside it, at both viewports.
c	Long wedge labels broke on desktop: "Adaptive Intelligence" clipped at its wedge edge, "Predictions" overflowed the disc.
+	Positives, recorded: the bloom is a fast, clean fade-and-scale, fully formed by about 120ms with no tearing; Escape returns cleanly; and glyph auto-contrast works.

Each of (a), (b) and (c) has a chapter of its own later in this record — the centre caption in Chapter 24, the flexbox `min-width: auto` trap in Chapter 26, and the centroid label-box geometry in Chapter 12 — because each turned out

to be a genuinely instructive bug rather than a typo.

3.4 The findings that were not the hub's fault but were its problem

Two more audit entries shaped the hub even though neither is about it.

AUD-503, the metaball cursor. The pointer drop swelled $\times 1.85$ over interactive elements — largest exactly when it was covering a control's label, so "Play now" read as "▷ ●now". It shares the radial module and the same goo filter bank as the dial, and its repair (cap the swollen diameter around 22px, thin the fill as it swells) is documented here in Chapter 22 because it is the same lesson as the dial's glyph layer: a decorative liquid effect must never be in the same layer as something you have to read.

AUD-118, the first-run problem. Filed among the miscellaneous nits, and arguably the most important sentence in the document about the hub: *"sub-1120px navigation is a single unlabelled orb with no first-run hint (recognition-over-recall; consider a transient 'Menu' caption)." A control that collapses three systems into one is only a win if the user knows it is there. Chapter 30 is about the fact that this one is still open.*

A HARNESS INCIDENT WORTH RECORDING

The audit's own tooling briefly failed to capture the hub at all: the interact phase's hub captures silently skipped because the orb's live accessible name is "Open menu" — the string in the shipped English catalog — while the code's `defaultValue` is "Open navigation". The harness was selecting by accessible name. It was re-captured by class selector. The lesson generalises: a translated `defaultValue` and its catalog entry drifting apart is invisible in the UI and load-bearing for anything that automates against it.

CHAPTER FOUR

Goals, non-goals and constraints

The brief, reconstructed. Some of it was written down before the work; much of it was discovered during, and is now written down as rules in the module's README so the next person does not rediscover it.

4.1 Goals

1. **One navigation, all widths.** One component, one catalog, one notion of active. A desktop rail may render the same catalog persistently, but it must not be a second implementation of the navigation's logic.
2. **Sixteen destinations, reachable in one gesture.** Not four primary plus a drawer for the rest. All of them, at the same cost.
3. **Thumb-first.** The trigger sits at the bottom centre, and every target in the opened dial is within the arc a thumb sweeps.
4. **The brand mark does real work.** The spirograph becomes the control, at a size where its filigree resolves.
5. **Theme-safe by construction.** Three shipped themes, fourteen accent presets, and a marketplace where users author their own token sets. Correctness cannot depend on the author having looked at a particular palette.
6. **Cheap on a phone.** Decorative cost gated behind width and motion-preference queries; the interaction itself must run on a low-memory device with no filters at all.
7. **Reversible.** Opening and closing are the same motion in two directions, not two animations. This turned out to constrain the implementation more than anything else on this list.

4.2 Non-goals

1. **Not a gesture menu.** No press-and-drag-to-select, no flick direction. Tap to open, tap to choose. Radial menus in games rely on muscle memory built over hours; a website does not get that budget, and a gesture-only dial would be unusable to anyone arriving for the first time.
2. **Not the navigation for full-screen surfaces.** Games and the full-screen apps live at top-level routes outside `_site` and keep their own chrome, deliberately. A game needs the whole viewport and its own input model; a site navigator floating over it is a liability, not a service.
3. **Not a command palette.** One already exists, mounted globally, and it is the right tool for "I know the name of the thing". The dial is for browsing a fixed catalog, and the two do not need to converge.
4. **Not sixteen equal destinations.** The two decks exist precisely to encode that the first six matter more than the last ten.

4.3 Constraints inherited from the platform

These were not negotiable, and several of them explain implementation choices that would otherwise look eccentric.

Tokens only.

No hex, no `rounded-lg`, no bespoke transition durations. Every value through a `--site-*` custom property, because a theme the author never opened must still look deliberate. This is the platform's stated definition of done for a UI commit, and it is the reason the dial's fills are `color-mix()` expressions over tokens rather than colours.

Ink tracks its surface.

A filled surface takes its paired foreground token — `bg-site-accent` implies `text-site-accent-fg`. Never `text-white` on a themed surface. Chapter 20 is the story of what happens when a clever alternative is used instead.

Every string translated.

Sixteen locales, two of them right-to-left, all user-facing strings through `t("key", { defaultValue })`. Every wedge label, the orb's accessible name, the dial's accessible name, the foot pill's controls.

Degradation is central, not per component.

High contrast, reduced transparency, reduced motion, forced colours and a `perf-lite` mode are handled by the token layer and the shared glass classes. A component that branches on them by hand is considered a bug. The hub's reduced-motion path is the one sanctioned exception, and even that is expressed as media queries in CSS rather than logic in the component.

No framer-motion in the hub.

The rest of the site uses it freely. The hub is CSS transitions and `clip-path` only, to stay light on mobile — which is why its state machine is three strings and a `setTimeout` rather than an animation orchestrator.

4.4 The constraint that mattered most

Of all of them, "reversible" did the most to shape the code. It sounds like a polish requirement and is actually an architectural one.

The natural way to animate a menu open is a keyframe animation: declare the frames, trigger it on mount, done. But a one-shot keyframe only plays forwards. Closing then needs a second animation, whose frames must mirror the first, and the two drift the moment anybody tunes one of them. Worse, if the user taps the orb again halfway through opening, a keyframe has no notion of "where I currently am" to reverse from — it restarts.

So every layer of the hub animates on a *transition* between two declarative states, driven by a single `data-phase` attribute on the wrapper. The dial's opacity and scale, the wedges' rotation and fade, the level hairlines, the foot pill, the blur's `clip-path` radius, the orb's transform: all of them are "property A when closed, property B when open", and the browser interpolates in whichever direction the attribute just changed. Interrupt it mid-flight and it reverses from exactly where it was, because that is what transitions do.

Reversibility is not a polish requirement. It is the reason the hub is a state machine with three states instead of a pair of keyframe animations.

The cost is that a couple of effects that would be trivial as keyframes are awkward as transitions — the counter-rotating decks in particular need their rest state to be *rotated* and their open state to be zero, which is backwards from how one naturally writes it, and which is exactly the trap that produced the black-flash bug in Chapter 16. It was still the right trade.

PART TWO

The concept

One control doing two jobs, a material that makes the second job legible, and a brand mark that stopped being decoration. This part is the design before the arithmetic.

5 The orb: one control, two jobs

6 The material

7 The mark that became the button

CHAPTER FIVE

The orb: one control, two jobs

Docked, it is a home button. Tapped, it becomes the centre the navigation radiates from. Those are not two components that swap places; they are one element with two declared transforms and a 500ms transition between them, and that identity is the whole trick.

Most menus are separate from the thing that opens them. A hamburger is a button; the drawer is a panel; the button stays put while the panel arrives from somewhere else. The relationship between them is learned, not seen — you press the icon, something appears elsewhere, and you infer the connection.

The hub refuses that separation. When you tap the orb, the orb itself travels to the middle of the screen, and the menu grows out of where it lands. There is never a moment when the thing you touched and the thing that opened are two objects. The dial is literally an annulus around the orb; the hole in its middle is the orb's parking space, sized from the orb's own tokens.

5.1 The two states, as declared

The orb's docked position is three declarations: fixed, horizontally centred, and lifted off the bottom edge by its own inset plus the device's safe area.

THE DOCKED ORB — components/radial/radial.css:722–742

```

722 .radial-hub__orb {
723   position: fixed;
724   left: 50%;
725   /* The orb is the site's only navigation control on mobile, so it MUST clear
726      the home-indicator / browser bottom bar — with viewport-fit=cover a fixed
727      bottom-anchored element is not covered by the body's safe-area padding, so
728      the inset has to be added here. Without it the orb sits inside the home
729      indicator's gesture strip (unreliable to tap) and hangs visibly lower than
730      the compose FAB beside it, which has always been safe-area-aware.
731      Size/inset come from globals.css so the floating-bottom stack ($5.5x A.1)
732      can reserve exactly this footprint. */
733   --orb-bottom: calc(env(safe-area-inset-bottom, 0px) + var(--site-hub-orb-inset));
734   bottom: var(--orb-bottom);
735   z-index: var(--z-floating);
736   transform: translateX(-50%);
737   display: grid;
738   place-items: center;
739   width: var(--site-hub-orb-size);
740   height: var(--site-hub-orb-size);
741   border-radius: 999px;
742   border: none;

```

Its open position is one more rule, and the arithmetic in it is the entire "glide to centre" effect:

CENTRED ONLY WHILE OPEN — components/radial/radial.css:801–807

```

801 /* Centre only while OPEN — it becomes the hub the wedges radiate from. On
802    'closing' the base (docked) transform takes over, so the orb glides home in
803    sync with the blur contracting. translateY = (bottom + half orb) - half
804    viewport → dead-centre. */
805 .radial-hub[data-phase='open'] .radial-hub__orb {

```

```
806   transform: translate(-50%, calc(var(--orb-bottom) + 50% - 50dvh));
807 }
```

Read that `translateY` carefully, because it is doing something slightly clever. The orb is anchored to the bottom of the viewport, so to place its centre at the viewport's centre it needs to move up by (distance from bottom edge to its own centre) minus (half the viewport). `--orb-bottom` is the gap below it; `50%` in a transform resolves against the element's own size, so it is half the orb; and `50dvh` is half the dynamic viewport height. The result is dead centre, with no measurement, no JavaScript, and — critically — `dvh` rather than `vh`, so it stays centred as a mobile browser's toolbars collapse and re-expand.

WHY THE CENTRING RULE IS SCOPED TO OPEN AND NOT CLOSING

There is no explicit "return home" transform. The centring rule applies only at `[data-phase='open']`; the moment the phase flips to `closing`, that rule stops matching and the base docked transform takes over. The browser transitions from wherever the orb currently is back to the dock. One declaration, both directions — the reversibility constraint from Chapter 4, paying off in three lines of CSS.

5.2 Why the trigger is not also a link

An obvious objection: if the orb is the home button, tapping it should go home. It does not — it opens the dial, and *Home* is the first wedge inside.

This was a deliberate trade and it is worth defending, because it is the one place the design knowingly costs a user something. Going home became two taps instead of one.

What it bought: a single, unambiguous interaction contract. A control that navigates on tap and opens a menu on long-press is a control with a hidden mode, and hidden modes on touch surfaces are how people end up somewhere they did not ask to be. The orb has exactly one behaviour, its `aria-haspopup="menu"` and `aria-expanded` describe that behaviour accurately, and nothing about it is discovered by holding it down longer than usual.

It also bought the geometry. If tapping the orb navigated, the dial would need a different trigger, and the "menu grows out of the thing you touched" identity — the reason the shape means anything — would be gone.

The mitigation is that *Home* is the first wedge, at twelve o'clock on the inner deck: the largest, closest, most predictable target on the dial. Two taps, both of them in the same place, neither requiring the user to read anything.

5.3 The wrapper that generates no box

The component's root element is one of the more unusual lines in the module:

A WRAPPER THAT IS ONLY A STATE CARRIER — `components/radial/radial.css:715–719`

```
715 /* Grouping wrapper – generates no box (children are fixed), it just carries the
716    open/centering/closing phase for the CSS state machine below. */
717 .radial-hub {
718   display: contents;
719 }
```

`display: contents` removes the wrapper from the box tree entirely — its children are laid out as though they were children of its parent. That is precisely what is wanted here, because every one of those children is `position: fixed` and positions itself against the viewport. A real wrapper box would add nothing except a risk: any wrapper

that later acquired a `filter`, a `transform` or a `backdrop-filter` would become a containing block for its fixed descendants and quietly break all of them.

What the wrapper is for is the `data-phase` attribute. Every animated layer in the hub is selected as a descendant of `.radial-hub[data-phase='...']`, so one attribute change on one element re-declares the resting state of the orb, the aura, the scrim, the overlay, the blur, the dial, all sixteen wedge wrappers, the hairlines and the foot pill simultaneously. That is a CSS state machine with a single write, and it is why the whole thing animates in lockstep without an orchestrator.

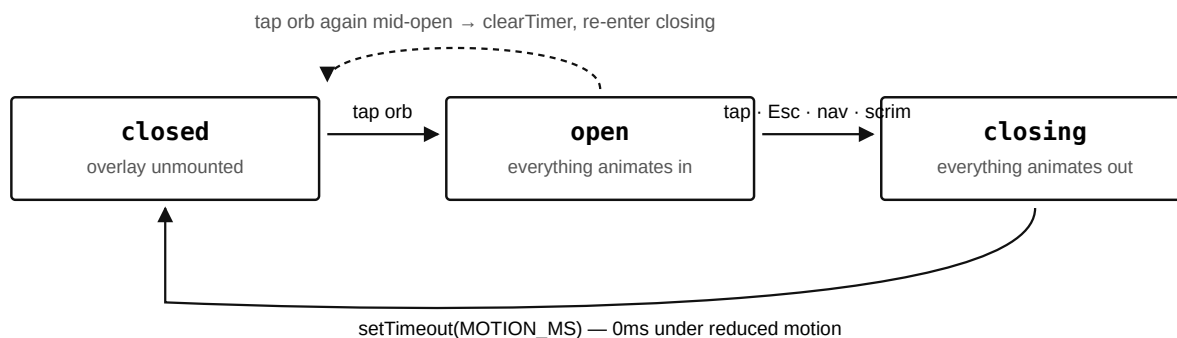


Figure 5.1 — The phase machine. Three states, one attribute, one timer. The `closing` state exists solely so the overlay stays mounted long enough for every layer to animate out; Chapter 15 covers why a component that unmounts its overlay on close cannot have a close animation at all.

5.4 What sits around the dial

The opened hub is not only the dial. Three other pieces share the overlay, and each exists for a specific reason.

The scrim.

A transparent, full-screen `<button>` beneath the blur that closes the menu when pressed. It is a button rather than a div with a click handler so it is focusable, nameable and keyboard-operable by default — it carries a translated "Close" label. Because it sits below the blur and the dial in the stack, only genuinely empty regions dismiss.

The foot pill.

A floating capsule at the bottom of the overlay holding identity, settings, sign-out and an explicit close button — signed out, it holds a single "Sign in" call to action instead. These are the destinations that are *about you* rather than about the site, and putting them on the dial as wedges would have implied they are peers of Library and Arcade. It is the one L4 glass surface in the hub: it frosts what passes under it rather than hiding it.

The aura.

Purely decorative: a disc the size of the orb plus three orbiting blobs, all in the accent colour, all under the goo filter, so they fuse into one wobbling liquid mass that bulges and necks as the blobs swing. It is a separate fixed layer rather than a pseudo-element on the orb — Chapter 18 explains why putting it on the orb would have destroyed the mark.

The explicit close button in the foot deserves a note. There are already three ways to close the dial: tap the orb again, tap the scrim, press Escape. A fourth, visible, labelled control is redundant for anyone who knows the first three — and it is the only one that is *discoverable* to someone who does not. It is also the only close affordance that survives on a touch device with no keyboard and no expectation that background taps dismiss. It is filled in the accent colour, which makes it the single most prominent control in the overlay after the dial itself.

5.5 The desktop rail is not a second implementation

From 1120px the shell also shows a persistent left rail, and it would be easy to read that as an admission that the dial

does not work on desktop. It is not. The rail renders the same `SIDEBAR_NAV`, applies the same auth and admin gating, and computes active state the same way; it adds only live badge counts, which a dial sector has no room for. The hub remains mounted and functional at every width.

The reasoning is that a wide screen has spare horizontal space and a mouse, and under those conditions a persistent list is strictly faster than any menu that has to open — no state transition, no motion, no dismissal. The dial's advantage is that it costs no layout space, and layout space is only scarce on small screens. Serving both is not inconsistency; it is the same catalog rendered at the density each viewport can afford.

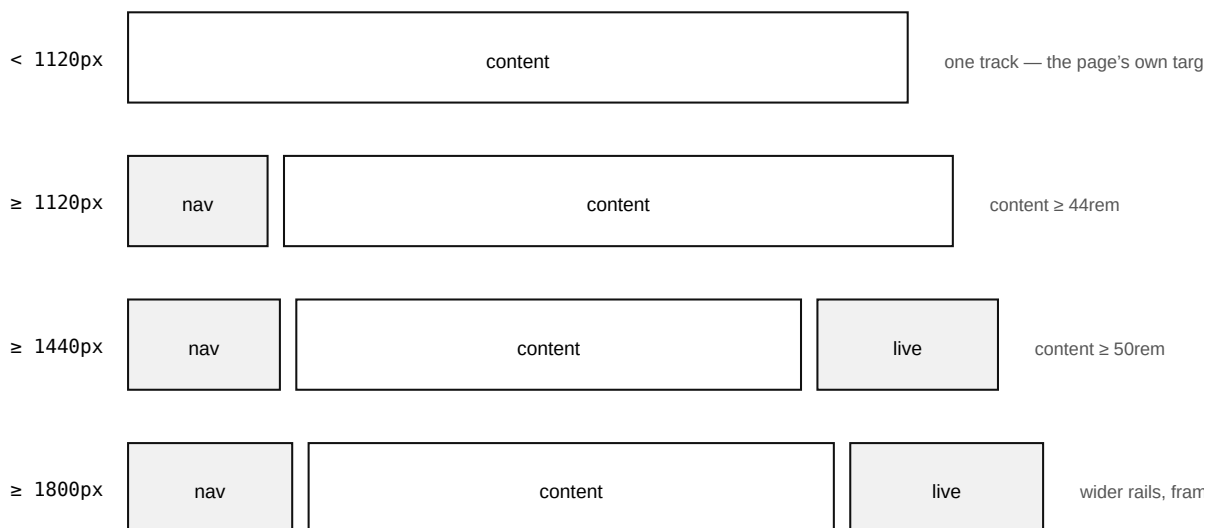


Figure 5.2 — The shell's frame by track count. Rails appear at the width that affords them, and a rail below its breakpoint is `display: none`, so it leaves the grid entirely rather than becoming a zero-width track. Track count and visible rails can never disagree.

CHAPTER SIX

The material

The dial is not drawn on a coloured disc. It is a hole cut in a frosted plate — an expanding circular blur with translucent sectors over it. That choice is what makes a sixteen-sector menu covering most of the screen feel like something that opened rather than something that landed.

The platform's design language is **Radial Avant-Garde Glass**: Apple's Liquid Glass material, used theatrically rather than literally, draped over a radial information architecture. The material half of that name is a large system in its own right — an elevation ladder of five explicit CSS classes, a specular rim glint that tracks a global scene light, a pointer-tracked diffuse highlight, lens-model edge refraction, an optional chromatic prism, travelling liquid sheens, and a drifting aurora canvas painted on the document body that every `backdrop-filter` on the page samples.

The hub uses almost none of it. That restraint is deliberate and worth explaining.

6.1 What the hub takes from the material system

GLASS FEATURES, AND WHETHER THE HUB USES THEM

FEATURE	IN HUB	WHY
Backdrop blur (L4 overlay tier)	Yes	The dial's entire surface is an expanding blurred disc; the foot pill is a floating L4 capsule.
Translucent tints over the scene	Yes	Sector fills are a token mix, not a colour, so the blurred page shows through.
Saturation boost	Yes	<code>saturate(118%)</code> on the blur, so the frosted scene keeps some life.
Specular rim glint	No	An always-on border-box highlight would draw a lit edge around sixteen sectors — sixteen competing highlights on one disc.
Pointer diffuse light	No	The wedge's own hover fill already answers "where is my pointer", unambiguously.
Edge refraction / prism	No	Budgeted at two and one per page respectively, and reserved for hero surfaces. Also currently parked platform-wide: Chromium composites the displacement map into the bevel instead of bending the backdrop through it.
Travelling liquid sheen	No	Budgeted at three per page; a sheen crossing an open menu is motion the user did not cause.
Metaball goo filters	Yes	The hub's signature. Chapter 19 is entirely about them.

The pattern: the hub takes the material's *depth* and its *liquidity*, and declines every optic whose job is to make a surface look lit. A menu is a temporary thing the user summoned; it should read as an atmospheric change, not as a new physical object with its own lighting.

6.2 The blur is the menu

The single most consequential material decision: there is **no drawn disc** behind the dial. The layer that fills the screen when the menu opens is a `backdrop-filter` with a `clip-path` circle animating from radius zero to `150vmax`, and only a whisper of background tint — 20% of the page background mixed with transparency — so it reads as blur rather than as a coloured circle.

THE EXPANDING CIRCULAR BLUR — `components/radial/radial.css:972–988`

```

972 /* The expanding circular blur – the menu's "surface". A frosted disc grows from
973    the centre (where the orb now sits) out to the viewport corners; only a
974    whisper of tint, so it reads as BLUR, not a coloured circle. */

```

```

975 .radial-hub__blur {
976   position: absolute;
977   inset: 0;
978   z-index: 1;
979   pointer-events: none;
980   background: color-mix(in srgb, var(--site-bg) 20%, transparent);
981   -webkit-backdrop-filter: blur(20px) saturate(118%);
982   backdrop-filter: blur(20px) saturate(118%);
983   clip-path: circle(0 at 50% 50%);
984   transition: clip-path 0.5s var(--ease-glass, cubic-bezier(0.16, 1, 0.3, 1));
985 }
986 .radial-hub[data-phase='open'] .radial-hub__blur {
987   clip-path: circle(150vmax at 50% 50%);
988 }

```

Three things follow from this that would not follow from a drawn disc.

The page stays visible. You can see, blurred, where you were. The menu is an overlay in the literal sense — a change in the optical properties of the page, not a replacement for it. Dismiss it and the frost contracts back to a point; nothing was ever hidden.

The expansion is free. Animating a `clip-path` circle's radius is a compositor-friendly operation on one element. A drawn disc growing to cover the viewport would be a large paint region changing size every frame.

It cannot be the wrong colour. A drawn disc needs a fill, a fill needs a token, and a token that works on a white page and a near-black page and a yellow-accented high-contrast page is a genuinely hard thing to choose. Blur has no colour of its own. It samples whatever is behind it, in whatever theme, and is correct by construction.

The menu's surface is not a colour that had to be chosen. It is the page you were already looking at, out of focus.

6.3 Where translucency had to be walked back

Translucency is correct for the blur and wrong for the sectors, and discovering that cost a shipped defect — commit `c631cb4`, *"the radial menu was invisible on a light page."*

The sectors were originally filled with `color-mix(in srgb, var(--site-surface) 60%, transparent): 60%` of the surface token, 40% transparent. Reasonable-looking code. The trap is that `--site-surface` is *itself* translucent in this material — the design language is explicit that surfaces are tints over the aurora, not paper. So the expression was a tint of a tint, and over a light page it resolved to very nearly nothing. The sectors disappeared; the dial was a rim and some floating glyphs.

The repair uses the opaque companion token the material ships for exactly this case, and then does one more thing that is the actually interesting part:

THE SECTOR FILL — `components/radial/radial.css:1164–1169`

```

1164 .radial-hub__wedge-art {
1165   position: absolute;
1166   inset: 0;
1167   display: block;
1168   background: color-mix(in srgb, var(--site-surface-opaque) 94%, var(--site-text) 6%);

```


1169 }

That 6% of `--site-text` is what separates the dial from the page. On the default theme both the page background and the opaque surface token are white; without the mix, the sectors would be white on white and only the rim would show. Mixing toward the ink adapts automatically in a way a fixed colour cannot: a touch darker than a light page, a touch lighter than a dark one, in any theme anyone ever authors, including ones that do not exist yet.

THE GENERALISABLE RULE

When a surface must be distinguishable from an unknown background, do not pick a colour — pick a *direction*. `color-mix(surface, text, 6%)` says "slightly toward the ink", and every theme's ink is by definition contrasting with every theme's background. A hex value says "this specific grey", which is a bet on a palette.

6.4 The plate under the sectors

Beneath the sectors is the dial's own background — the "divider bed". Its job is mundane: the sectors tile the disc with thin angular gaps between them, and the bed is what shows through those gaps as the hairline dividers. It also stops the disc dissolving into a light page.

Its colour is a shade darker than the sectors that sit on it, built the same way: `color-mix(in srgb, var(--site-surface-opaque) 86%, var(--site-text) 14%)`. The value 14% versus the sectors' 6% is the entire visual language of the dividers — they are not drawn lines, they are the bed showing through.

This bed is also the villain of Chapter 16, where a perfectly sensible earlier choice for its colour made the whole dial flash black on open the moment the decks started spinning. It is a good example of a value that is correct until a motion change makes it visible.

6.5 Degradation, and where it is handled

The platform's rule is that accessibility and performance degradation happen in the token layer and the shared glass classes, not in components — a component that branches on `prefers-reduced-transparency` by hand is considered a bug. The hub obeys this, with the consequence that the interesting behaviour is in media queries rather than in TypeScript.

WHAT THE HUB LOOKS LIKE UNDER EACH DEGRADATION

CONDITION	EFFECT ON THE HUB
<code>prefers-reduced-motion</code>	All goo filters off. The aura is not rendered. The logo bloom and stroke-draw do not run. Every transition is dropped, and the close timer becomes 0ms so the overlay unmounts immediately.
<code>< 768px</code>	All decorative goo off regardless of motion preference — the filter cost is a phone budget question, not a preference question. Sectors render crisply.
<code>high-contrast theme</code>	No glass at all: opaque black and white, a yellow accent, 2px borders. The dial becomes flat sectors with firm edges, which is exactly what that theme is for.
<code>reduced-transparency</code>	Tints resolve to their opaque companions through the token layer. The blur stops being the surface and the plate becomes the surface.
<code>forced-colors</code>	System colours win. The pointer metaball turns off entirely.
<code>< 4 GB device memory</code>	The pointer metaball turns off entirely. The dial keeps working; only decoration is dropped.

Note what is *not* in that table: the interaction. In every degraded mode the dial still opens, still has sixteen labelled targets, still traps focus sensibly, still closes on Escape. The material is the thing that degrades. The navigation is not optional.

CHAPTER SEVEN

The mark that became the button

The RMH mark is a six-fold radial spirograph: several hundred overlapping curves in one continuous path. Putting it inside a 44-pixel disc — and keeping it legible there — required deciding what a logo is *for* when it is also a control.

The mark predates the redesign. It is an Inkscape-generated spirograph with its path baked, six-fold rotationally symmetric, drawn as line art. At poster size it is genuinely beautiful; at 20 pixels in a header it is a grey smudge, which is roughly what it had been reduced to.

The radial premise offered a much better job for it. A mark whose own construction is radial, placed at the centre a radial interface orbits, is not decoration — it is a statement of what the geometry means. So the mark went into the orb, at 74% of the orb's size, and the work became: make a few hundred overlapping curves survive being rendered at 44 pixels.

THE MARK, AS SHIPPED — components/radial/RmhLogo.tsx:1–9

```
1 'use client';
2
3 // The RMH Studios mark – a 6-fold radial spirograph (Inkscape-generated, path
4 // baked). Rendered as line-art in `currentColor` so it inherits the orb's
5 // accent-contrast ink, with `pathLength={1}` so the stroke-draw "bloom" is
6 // resolution-independent. Geometry is the source SVG's exact transforms.
7 export function RmhLogo({ className }: { className?: string }) {
8   return (
9     <svg className={className} viewBox="0 0 150 150" role="img" aria-hidden="true" focusable="false">
```

7.1 Hairline weight is not a style choice

The controlling problem is stroke weight. The path is dense: at any reasonable weight, overlapping curves bleed into each other and the whole thing fills in as a solid muddy blob. The component's own comment is direct about it:

WHY THE STROKE IS 0.8 — components/radial/radial.css:895–903

```
895 .radial-hub__logo path {
896   /* Hairline line-art: the mark is a very dense spirograph rendered tiny (~44px
897    in the orb), so any weight bleeds the overlapping curves into a solid muddy
898    blob. Kept very fine (this scales DOWN with the geometry, so on-screen it's
899    a sub-pixel filigree) → a delicate white mark, not a heavy black disc. */
900   stroke-width: 0.8;
901   stroke-dasharray: 1;
902   stroke-dashoffset: 0;
903 }
```

The mark is authored in a 150-unit viewBox and rendered into roughly 44 CSS pixels, so it is scaled down by about 3.4×. A stroke width of `0.8` in that space is a genuinely sub-pixel filigree on screen — which is the point. The result reads as a delicate white tracery rather than a heavy disc, and the six-fold symmetry survives even when individual curves do not fully resolve. What the eye recovers at that size is the *rotational structure*, and rotational structure is exactly what the interface is about.

AN IMPLEMENTATION DETAIL WORTH STEALING

The path carries `pathLength={1}`. That normalises its length to one unit, so `stroke-dasharray: 1; stroke-dashoffset: 1` is a complete, unfilled outline regardless of the path's real length — which for this path is enormous and would otherwise have to be measured. The bloom animation then just runs `stroke-dashoffset` from 1 to 0 and the mark draws itself, resolution- and geometry-independent. Change the artwork and the animation still works.

7.2 The bloom

On mount, the mark scales and rotates out of the orb's core while its stroke draws itself:

THE MOUNT BLOOM — `components/radial/radial.css:904–924`

```

904 /* On mount the mark blooms open – scaling + rotating out of the core while its
905    stroke draws itself (pathLength-normalised), leaning into the SVG. */
906 @media (prefers-reduced-motion: no-preference) {
907   .radial-hub__logo {
908     animation: rmh-logo-bloom 1.05s var(--ease-glass, cubic-bezier(0.16, 1, 0.3, 1)) both;
909   }
910   .radial-hub__logo path {
911     animation: rmh-logo-draw 1.3s ease both;
912   }
913 }
914 @keyframes rmh-logo-bloom {
915   from {
916     transform: scale(0.15) rotate(-50deg);
917     opacity: 0;
918   }
919 }
920 @keyframes rmh-logo-draw {
921   from {
922     stroke-dashoffset: 1;
923   }
924 }
```

Two animations, deliberately different lengths — the geometry settles at 1.05s while the stroke finishes drawing at 1.3s, so the mark arrives and then completes rather than doing both at once. Both are wrapped in `prefers-reduced-motion: no-preference`, so a user who has asked for less motion simply gets the finished mark.

Worth noting what this animation is *not*: it is not tied to opening the menu. It runs once, when the shell mounts. It is the site introducing itself, not a response to an interaction — which is why it is a keyframe animation rather than a transition, the one place in the hub where a one-shot is correct, because there is no reverse state to return to.

7.3 The ink problem, and the first version of a mistake

The mark sits on the orb's solid accent fill, so the correct ink for it is that surface's paired foreground: `--site-accent-fg`. Every theme authors that pairing, every accent preset overrides it, and a runtime contrast check — `ensureReadableAccent()` — refuses to ship a pair that fails. Naming the token makes the mark correct in every theme, including user-authored ones.

That is the current code. An earlier revision instead used `mix-blend-mode: difference`, reasoning that white blended in difference mode against any backdrop yields that backdrop's exact inverse, and therefore maximum contrast automatically, for free, in every theme.

It is a genuinely elegant idea and it is wrong, for a reason that has nothing to do with colour theory: **a blend mode only composites against the backdrop inside the element's own stacking context**. Put the blended element in a stacking context that does not contain the thing you meant to blend against, and it inverts the wrong backdrop — or nothing at all. That is exactly how the dial's glyphs ended up white on white, which is Chapter 20, and it is why the comment explaining the current token-based approach in `radial.css` ends with a warning about the revision it replaced.

"Automatic" contrast that depends on stacking context is not automatic. It is a bet on a layer tree that the compositor is entitled to change.

7.4 The mark as a loading state

One more consequence of promoting the mark: it gave the platform a house loading indicator. `ui/radial-loader.tsx` is the same visual idea — blobs orbiting a pulsing core, fused by the same goo filter — and it is what the shared `Spinner` primitive renders. So a wait anywhere on the site speaks the same language as the hub.

This is a small thing that indicates the redesign was working. When a navigation control's visual vocabulary is coherent enough that unrelated parts of the product can borrow it and look native, the vocabulary is real. It is also, per the audit, not free: finding AUD-302 records that on phones the feed swapped a full-viewport goo loader for the whole page and produced a measured layout shift of 0.13–0.21, well past the 0.1 threshold. A house style that is applied without a size reservation is still a regression.

PART THREE

Geometry

Sixteen destinations, one circle, and a 320-pixel-wide phone. This part is the arithmetic: annulus sectors, two decks, a polygon walk, a masked hole, drawn hairlines, an angular bleed, and the label boxes that spill in the one direction there is no room.

-
- 8 From pie to annulus
 - 9 The double deck
 - 10 Carving one destination

- 11 The hole, the mask and the hairlines
- 12 Labels, centroids and budgets
- 13 The art bleed

CHAPTER EIGHT

From pie to annulus

A pie slice is the obvious shape for a radial menu and the wrong one. Its area is concentrated at the rim, its point converges on the exact spot the orb needs to occupy, and its centroid drifts outward as the slice narrows. Every one of the dial's geometric decisions follows from replacing the pie with a ring segment.

8.1 Four problems with the pie

The point is in the way. Sixteen pie slices converge on a single point at the centre of the disc, and that point is exactly where the orb parks. Either the orb covers the convergence — hiding the tips of sixteen targets — or the slices are truncated, at which point they are annulus sectors and one may as well say so.

Area is in the wrong place. A sector's area grows with the square of the radius, so most of a pie slice's area is in its outer third. The glyph and label, to sit in the visual middle of the slice, have to sit far out toward the rim; the inner two-thirds is a large region that belongs to the target but reads as empty.

The centroid moves as the count changes. A pie slice's centroid sits at $2r \sin(\theta/2) / (3 \cdot \theta/2)$ from the centre — a function of the slice's angular width. Add a destination, all the slices narrow, and every glyph shifts radially outward for no reason the user can see. A ring segment's radial midpoint is $(r_0 + r_1) / 2$, which does not depend on the angle at all: glyphs stay on a fixed circle regardless of how many destinations exist.

You cannot stack pies. Two concentric decks of pie slices share their inner region, so the inner deck's slices *are* the inner parts of the outer deck's slices. There is nowhere to put a second level. Two rings of annulus sectors nest naturally, which is the whole reason the double-deck solution in Chapter 9 is available at all.

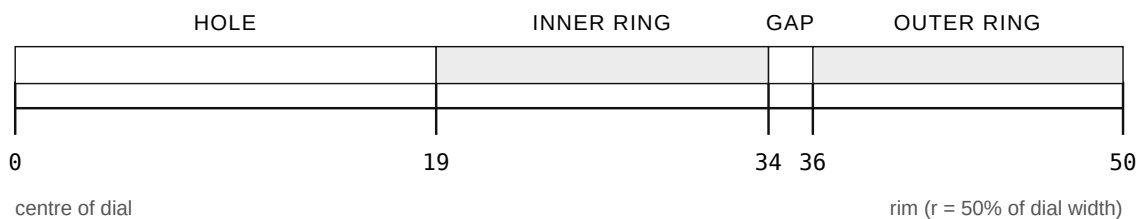


Figure 8.1 — The dial's radial bands, as percentages of the dial's width. The dial is square, so the rim is at $r = 50$. These five numbers are the only radial geometry in the component; the hairlines and the centre-hole mask are both derived from them rather than restated.

8.2 The band table

The bands are declared once, as a constant, with a comment that draws the ruler in ASCII so the numbers are legible without running anything:

RING GEOMETRY — components/radial/RadialHub.tsx:35–52

```
35 /**
36  * Ring geometry, as a percentage of the dial's width (the dial is a square, so
37  * its radius is 50%). The hole in the middle clears the orb, which glides to the
```

```

38 * exact centre when the menu opens.
39 *
40 * 0 — 19 ————— 34 — 36 ————— 50
41 *   hole       inner ring   gap       outer ring   rim
42 *
43 * The gap between the decks is deliberately wide enough to read as a track, not
44 * a seam — `RadialHub` also renders a hairline at every boundary below, so the
45 * two levels are separated by a drawn border rather than by absence of fill.
46 */
47 const RINGS = [
48   { r0: 19, r1: 34 },
49   { r0: 36, r1: 50 },
50 ] as const;
51 /** With few enough destinations one ring is clearer than two. */
52 const SINGLE_RING = { r0: 19, r1: 50 } as const;

```

BAND DIMENSIONS, AT THE LARGEST DIAL (460PX)

BAND	R ₀	R ₁	DEPTH	DEPTH @460PX	MID-RADIUS
hole	0	19	19	87.4px	—
inner ring	19	34	15	69.0px	26.5
gap	34	36	2	9.2px	—
outer ring	36	50	14	64.4px	43.0

Single-ring mode: $r_0 = 19$, $r_1 = 50$, depth 31 (142.6px), mid-radius 34.5

Two observations about these numbers. First, the two rings are nearly the same depth — 15 and 14 — which is deliberate: a deeper primary ring would read as a hierarchy of *importance by size*, and the hierarchy is already carried by proximity to the centre. Second, the gap between them is 2, or about 9 pixels at the largest dial. That is wide enough to read as a track and narrow enough not to waste a band.

8.3 The dial's size

The dial's outer dimension is a single clamp, and it is doing more work than it looks:

```
--dial: min(86vw, 70vh, 460px);
```

`86vw` keeps a 7% margin on each side at every width, so the dial never touches the screen edge on a phone. `70vh` is the constraint that actually binds in landscape — on an 844×390 device, `70vh` is 273px, which is what stops the dial colliding with the foot pill. And `460px` caps it on desktop, because a menu the size of a dinner plate on a 27-inch display is not more usable, just louder.

RESOLVED DIAL SIZE, AND THE RESULTING BAND DEPTHS

VIEWPORT	DIAL BINDING TERM	INNER DEPTH	OUTER DEPTH	HOLE Ø
320×568	275px 86vw	41.3px	38.5px	104px
375×667	323px 86vw	48.4px	45.2px	123px
390×844	335px 86vw	50.3px	46.9px	127px
844×390	273px 70vh	41.0px	38.2px	104px
768×1024	460px 460px cap	69.0px	64.4px	175px
1920×1080	460px 460px cap	69.0px	64.4px	175px

The important row is the first. At 320px — the smallest supported width — the inner band is 41 pixels deep. The platform's touch-target floor is 44. A single sector's *radial* extent is therefore just under the floor at the smallest size, which is acceptable only because a sector's hit region is two-dimensional: the arc length at the inner deck's mid-radius

on a 275px dial is about 76 pixels per sector at six sectors. The target is 41×76, not 41×41. Chapter 22 returns to this, because it is the tightest number in the whole design.

8.4 Why percentages, and not pixels

All of this geometry is expressed as percentages of the dial's width, and that is not a stylistic preference — it is a requirement of the mechanism. The sectors are carved with `clip-path: polygon(...)`, and percentage coordinates in a `clip-path` resolve against the element's own box. So a sector defined in percentages is correct at any dial size, and the single `--dial` clamp resizes the entire construction — sixteen polygons, five hairlines, one mask — with no recomputation and no JavaScript listening to a resize event.

It also means the figures in this book, which are drawn in the same percentage space and then scaled, are dimensionally identical to what the browser clips. There is no translation step in which an error could hide.

ONE SOURCE OF TRUTH FOR THE RADII

The module's README states this as a rule for anyone touching the geometry: `RINGS` defines the bands; the drawn boundary hairlines and the mask that carves the centre hole are both derived from the radii the component actually used and handed to CSS inline. Do not re-type those numbers in `radial.css` — a hairline sitting where the bands are not is invisible until someone opens the menu.

CHAPTER NINE

The double deck

Sixteen destinations on one ring gives each 22.5° of arc. At the dial's inner radius on a phone that is a sliver about 20 pixels wide — too narrow to label, too narrow to hit reliably, and too narrow to tell apart from its neighbours. The dial is double-decked because of that number.

9.1 The arithmetic of crowding

A ring divided into n sectors gives each $360/n$ degrees. The usable arc length of one sector, at radius r , is $2\pi r/n$ minus the divider gap. What matters for usability is that arc length at the sector's *narrowest* point — its inner edge — because that is where the target is thinnest and where two neighbours are closest together.

ONE RING: SECTOR WIDTH AT THE INNER EDGE, ON A 335PX DIAL (390PX PHONE)

N	ANGLE	ARC AT R=19	ARC AT R=34	VERDICT
4	90.0°	100px	179px	generous
6	60.0°	67px	119px	comfortable — the single-ring ceiling
8	45.0°	50px	89px	labels start to clamp
10	36.0°	40px	72px	below the 44px floor at the inner edge
12	30.0°	33px	60px	unlabellable
16	22.5°	25px	45px	slivers

The double deck attacks this by dividing each ring independently. With sixteen destinations the split is six inner and ten outer, so the inner ring's sectors are 60° wide and the outer ring's are 36° — and crucially, the outer ring's sectors are 36° at a *larger radius*, where 36° of arc is 90 pixels at the band's inner edge rather than 40. The longer circumference is doing the work.

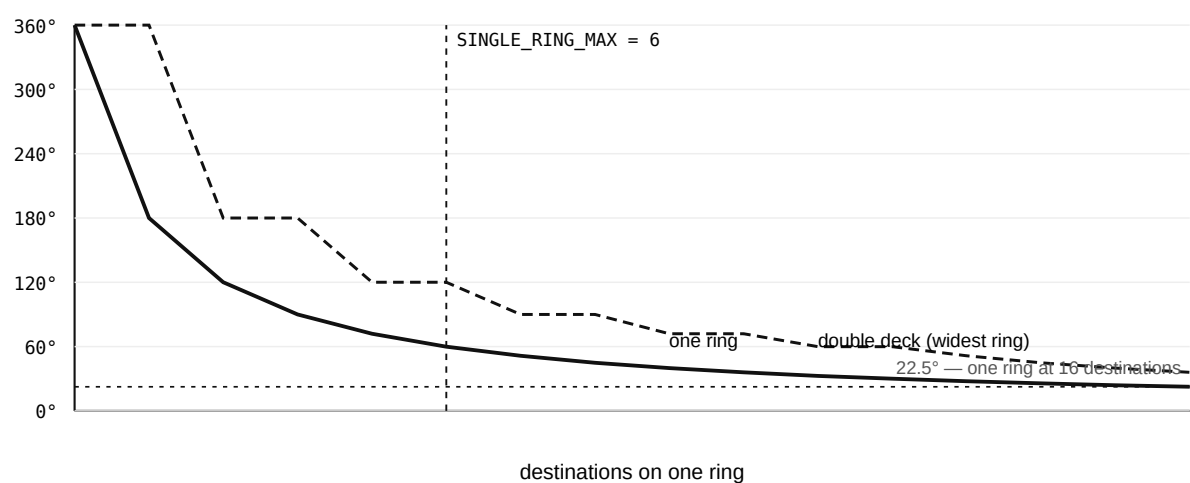


Figure 9.1 — Sector angle against destination count. The solid line is a single ring; the dashed line is the widest ring of the double deck. The deck split changes the shape of the curve: past six destinations, adding one no longer narrows every target.

9.2 The split rule, as code

CHOOSING ONE RING OR TWO — components/radial/RadialHub.tsx:158–171

```

158   const { wedges, bounds } = useMemo<{ wedges: Wedge[]; bounds: number[] }>(() => {
159     if (leaves.length === 0) return { wedges: [], bounds: [] };
160
161     const groups: Array<{ items: NavLeaf[]; band: { r0: number; r1: number }; ring: 0 | 1 }> =
162       leaves.length <= SINGLE_RING_MAX
163       ? [{ items: leaves, band: SINGLE_RING, ring: 0 }]
164       : (() => {
165         const innerCount = Math.min(INNER_MAX, Math.ceil(leaves.length / 2));
166         return [
167           { items: leaves.slice(0, innerCount), band: RINGS[0], ring: 0 as const },
168           { items: leaves.slice(innerCount), band: RINGS[1], ring: 1 as const },
169         ];
170       })();
171

```

Three constants govern this and each is a judgement rather than a derivation.

SINGLE_RING_MAX = 6

Below seven destinations, splitting is pointless: one wide band of 31 depth reads better than two thin ones, and the second deck would carry two or three items on a circle, which looks like an accident. The comment in the source is exactly one line: *"With few enough destinations one ring is clearer than two."*

INNER_MAX = 6

A cap on the inner deck, because it has the shorter circumference. Without it, a naive half-and-half split of, say, twenty-four destinations would put twelve on the inner ring, and twelve at radius 19 is the unlabellable case from the table above. With it, the inner deck never exceeds six and the outer takes the entire tail however long it gets.

Math.ceil(leaves.length / 2)

The nominal split, before the cap. For fourteen or more destinations the cap always wins, so in practice the shipped dial is always six-and-the-rest. The `ceil` matters only in the seven-to-eleven range, where it biases the *inner* deck to be the fuller one — correct, because the inner deck holds the higher-priority items and a fuller inner ring means a less crowded outer one.

HOW THE SPLIT RESOLVES, BY DESTINATION COUNT

N MODE	INNER	OUTER	INNER ANGLE	OUTER ANGLE
1–6 single ring	n	—	360/n	—
7 double	4	3	90.0°	120.0°
8 double	4	4	90.0°	90.0°
10 double	5	5	72.0°	72.0°
12 double	6	6	60.0°	60.0°
14 double	6	8	60.0°	45.0°
15 double	6	9	60.0°	40.0°
16 double	6	10	60.0°	36.0°

9.3 The count is not fixed

It is easy to read "sixteen destinations" as a constant. It is not — it is the count that survives filtering, and the filter runs on every render:

```

130 const leaves = useMemo<NavLeaf[]>(() => {
131   const out: NavLeaf[] = [];
132   for (const item of SIDEBAR_NAV) {
133     if (isNavGroup(item)) out.push(...item.children);
134     else out.push(item);
135   }
136   return out.filter((leaf) => {
137     if (leaf.requiresAuth && !session) return false;
138     if (leaf.requiresAdmin && !user?.isAdmin) return false;
139     return true;
140   });
141 }, [session, user?.isAdmin]);

```

Signed out, *Inbox* and *Admin* both drop, leaving fourteen. Signed in as a non-admin, fifteen. Signed in as an admin, sixteen. And because the geometry is derived from the filtered array rather than from a constant, the dial re-carves itself: fourteen destinations is still six inner and eight outer, at 60° and 45°.

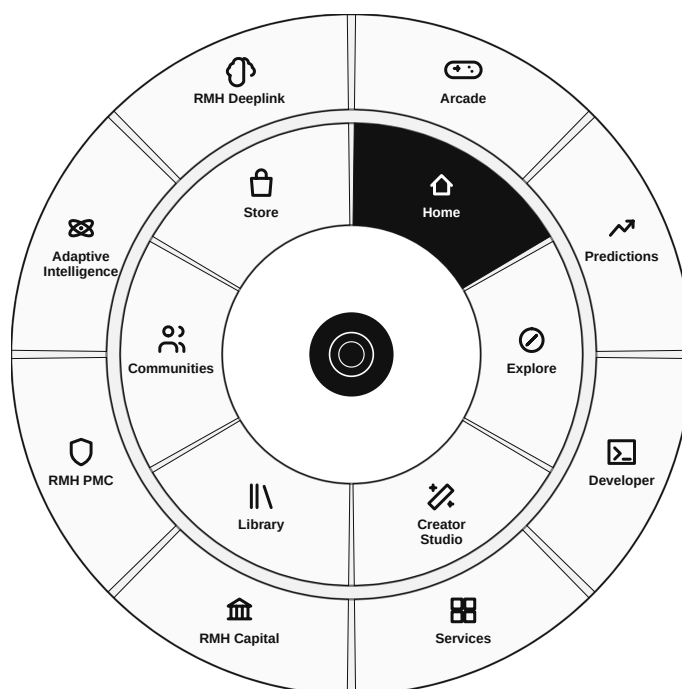


Figure — Signed out: fourteen destinations. Auth and admin gating changes the count, so the geometry re-derives; nothing about the layout is hand-placed.

This is the property that makes the geometry defensible rather than merely clever. A hand-laid dial would need three layouts maintained in parallel, and would break the day someone added a seventeenth destination. This one has no layout to maintain: it has a catalog and a function.

9.4 Priority, encoded as radius

The split takes the first six items in catalog order for the inner deck and the rest for the outer. So the catalog's order is not merely a rendering order — it is the priority hierarchy, and it is visible in the geometry: closer to the centre means more important.

The default order puts Home, Explore, Inbox, Creator Studio, Library and Communities on the inner deck, and Store, Arcade, Predictions, Developer, Services and the four RMH Ventures microsites plus Admin on the outer. That is a defensible reading of the platform: the social core inside, the destinations and ventures outside.

It also means users can change it. The platform ships a reorder-and-hide editor for the navigation, validated against the same stable ids, and its saved order is applied through `orderNavItems()`. A user who lives in the Arcade can promote it to the inner deck, and the dial re-carves. One exception is pinned by construction: Admin's id is deliberately absent from the orderable set, so it always falls to the end of the outer ring and can never be hidden.

THE FORWARD-SAFETY PROPERTY

`orderNavItems()` puts items present in the saved order first, in that order, then appends anything not mentioned in its default relative position. So a destination shipped after a user saved their preferences still appears, rather than vanishing because it was not in their list. For a dial whose geometry is derived from the resulting array, that property is load-bearing: a silently dropped item would not just be missing, it would silently change the angle of every remaining sector.

CHAPTER TEN

Carving one destination

One function, thirteen lines, called once per destination per render. It walks out along the outer arc and back along the inner one, sampling ten steps each way, and returns a `polygon()` string. Everything the dial is — its shape, its hit regions, its dividers — comes out of this.

RINGSECTOR() — `components/radial/RadialHub.tsx:74–94`

```

74 /**
75  * An annulus sector – the ring-segment polygon that carves one destination out
76  * of a ring: out along the outer arc over [a0,a1], then back along the inner one.
77  * Coordinates are percentages of the dial's box, which is what `clip-path`
78  * resolves against (and, because a clip bounds hit-testing too, is also exactly
79  * the region that catches this destination's clicks).
80  */
81 function ringSector(a0: number, a1: number, r0: number, r1: number): string {
82   const pts: string[] = [];
83   const steps = 10;
84   const at = (k: number) => (a0 + ((a1 - a0) * k) / steps) * DEG;
85   for (let k = 0; k <= steps; k++) {
86     const t = at(k);
87     pts.push(`${(50 + r1 * Math.cos(t)).toFixed(2)}% ${(50 + r1 * Math.sin(t)).toFixed(2)}%`);
88   }
89   for (let k = steps; k >= 0; k--) {
90     const t = at(k);
91     pts.push(`${(50 + r0 * Math.cos(t)).toFixed(2)}% ${(50 + r0 * Math.sin(t)).toFixed(2)}%`);
92   }
93   return `polygon(${pts.join(', ')} )`;
94 }

```

10.1 The walk

The polygon is built in two passes. The first walks the outer radius from `a0` to `a1` in ten equal steps, emitting eleven points. The second walks the inner radius back from `a1` to `a0`, emitting eleven more. Twenty-two points, closed implicitly by `polygon()`.

Each point is $50 + r \cdot \cos(\theta)$, $50 + r \cdot \sin(\theta)$ — the centre of the dial is (50%, 50%), and r is a percentage of the width, so the whole thing lands in the coordinate space `clip-path` expects with no conversion.

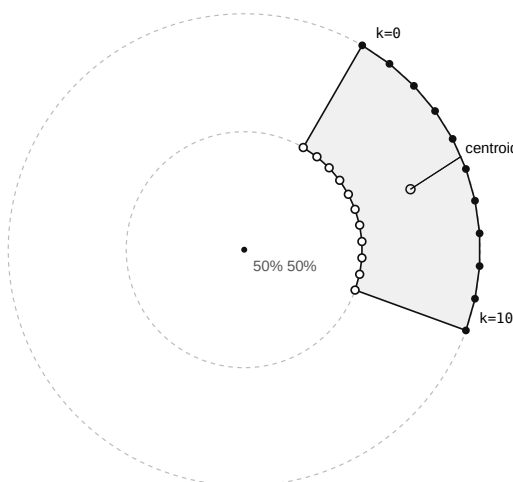


Figure 10.1 — The walk, on an exaggerated 80° sector. Filled dots are the eleven outer samples; hollow dots are the eleven inner samples on the way back. The ring at the centre of the band is the centroid the glyph is positioned at.

10.2 Why ten steps

`steps = 10` is the only tuning number in the function, and it is a straightforward accuracy-versus-size trade. A polygon approximates an arc with chords, and the maximum deviation of a chord from its arc — the sagitta — is $r(1 - \cos(\phi/2))$ for a chord subtending ϕ .

CHORD ERROR AT THE RIM (R = 50% OF A 460PX DIAL = 230PX)

SECTOR ANGLE	Φ PER STEP	SAGITTA VISIBLE?
120° (n=3)	12.0°	1.26px marginal on a hairline edge
90° (n=4)	9.0°	0.71px no
60° (n=6, inner deck)	6.0°	0.32px no
36° (n=10, outer deck)	3.6°	0.11px no
22.5° (n=16)	2.25°	0.04px no

At the shipped counts the error is a fraction of a device pixel, well inside antialiasing. Note the pleasant property that the error *falls* as the dial gets busier: more sectors means narrower sectors means shorter steps. The worst case is the single-ring mode with very few destinations — and even a three-item dial is off by about a pixel at the rim, on an edge that is itself antialiased against a blurred backdrop.

The cost side: each point is about 14 characters of inline style, so 22 points is roughly 320 characters per sector, times two layers, times sixteen destinations — around 10KB of inline `clip-path` in the DOM. Doubling the steps to 20 would double that for no perceptible gain. Halving it to 5 would put a 60° sector's error at 1.3px, which on the inner deck's crisp mode is visible as faceting.

10.3 The angular gap

Sectors do not touch. Each is inset from its nominal span by half a gap on each side:

```
const seg = 360 / n;
const gap = Math.min(1.4, seg * 0.06);
const a0 = -90 + i * seg + gap / 2;
const a1 = -90 + (i + 1) * seg - gap / 2;
```

The gap is 6% of the segment, capped at 1.4°. The proportional term keeps the dividers visually consistent as the count changes — a fixed 1.4° gap between 22.5° sectors is a much heavier line than between 90° ones. The cap stops the gap becoming absurd when there are very few, very wide sectors: at four destinations, 6% of 90° would be a 5.4° gap, a channel wide enough to read as a missing sector rather than a divider.

RESOLVED GAP, AND THE DIVIDER WIDTH IT PRODUCES AT THE BAND'S MID-RADIUS

N	SEG	6% OF SEG	GAP USED	DIVIDER @ INNER MID (460PX)
4	90.0°	5.40°	1.40° (capped)	3.0px
6	60.0°	3.60°	1.40° (capped)	3.0px
10	36.0°	2.16°	1.40° (capped)	3.0px
16	22.5°	1.35°	1.35°	2.9px
24	15.0°	0.90°	0.90°	1.9px

In practice the cap is what is active at every shipped count, so the dividers are a consistent ~3 pixels. The proportional term is insurance for a future dial with many more destinations, where it degrades gracefully rather than closing the

gaps entirely.

10.4 Starting at twelve o'clock

The `-90` in the angle expression is the whole of the dial's orientation logic. CSS angles put 0° at three o'clock and increase clockwise; subtracting 90° puts the first sector's leading edge at twelve. Sectors then proceed clockwise, which is the reading order for every left-to-right language and — more importantly — the direction a right thumb sweeps naturally from the bottom-centre of a phone.

AN OPEN QUESTION ABOUT RIGHT-TO-LEFT LOCALES

The site ships two RTL locales, and the dial's sector order does not mirror in them. Whether it should is genuinely unclear: a dial is not a line of text, and the correspondence between reading direction and rotational direction is a convention rather than a rule — clocks run clockwise in Arabic-speaking countries too. It is listed in Chapter 30 among the things still open, because the honest answer is that nobody has tested it with RTL readers.

10.5 The return value is three things at once

The polygon string produced here is used as a `clip-path`, and that single fact gives it three jobs simultaneously.

1. **It is the shape.** The sector's fill is clipped to it, so the visible sector is exactly this polygon.
2. **It is the hit region.** A `clip-path` bounds hit-testing as well as painting. The anchor element is a full-size square overlaying the whole dial; clipped, it only catches pointer events inside its own sector. No hit-testing code exists in the component at all.
3. **It is the divider.** The gaps between adjacent polygons are where the bed shows through, so the dividers are defined by the absence of clip rather than by any drawn line.

Getting all three from one declaration is the reason this component is 521 lines rather than fifteen hundred. It is also the reason the art layer needs a *separate, slightly wider* polygon, which is Chapter 13.

CHAPTER ELEVEN

The hole, the mask and the hairlines

Three pieces of derived geometry, all of which exist so that a change to `RINGS` cannot leave the dial internally inconsistent. Each is computed from the radii the component actually used and handed to CSS as an inline custom property.

11.1 The bounds array

After carving the sectors, the component collects the distinct band boundaries it used:

THE BOUNDARY RADII, DEDUPLICATED AND SORTED — `components/radial/RadialHub.tsx:194–198`

```
194     const bounds = [...new Set(groups.flatMap(({ band }) => [band.r0, band.r1]))].sort(
195       (a, b) => a - b,
196     );
197     return { wedges: out, bounds };
198   }, [leaves]);
```

For the double deck that yields `[19, 34, 36, 50]`; for single-ring mode, `[19, 50]`. Two consumers read it: the hairline elements, one per boundary, and the centre-hole mask, which takes `bounds[0]`.

The point of routing both through this array rather than re-typing the numbers is stated as a rule in the module README, and it is the kind of rule that only gets written after somebody has been bitten: a hairline drawn at a radius where no band boundary exists is invisible until someone opens the menu, and then it is a mystery line.

11.2 The hole

The dial's divider bed is a full disc, so without intervention the middle of the dial is a solid grey circle behind the orb — a bullseye. The hole is cut with a mask:

CARVING THE CENTRE OUT OF THE BED — `components/radial/radial.css:1019–1031`

```
1019  /* Carve the centre out of the divider bed, so the double-decked dial reads as
1020     two rings around the orb instead of a bullseye with a solid grey core. The
1021     mask applies to the sectors too, but they are already annuli – no change. */
1022  -webkit-mask: radial-gradient(
1023    circle closest-side at 50% 50%,
1024    transparent 0 var(--dial-hole),
1025    #000 var(--dial-hole)
1026  );
1027  mask: radial-gradient(
1028    circle closest-side at 50% 50%,
1029    transparent 0 var(--dial-hole),
1030    #000 var(--dial-hole)
1031  );
```

And the radius comes from the component:

THE HOLE RADIUS, DERIVED — `components/radial/RadialHub.tsx:334–343`

```
334     // radius (doubled: a radial-gradient stop is a fraction of the RADIUS,
335     // and these radii are fractions of the WIDTH).
```



```

336      // Pulled in by the border width so the innermost hairline, which
337      // straddles this radius, is not half-eaten by the mask.
338      style={
339        {
340          '--dial-hole': `calc(${(bounds[0] ?? 19) * 2}% - var(--dial-level-w))`,
341        } as CSSProperties
342      }
343    >

```

There are two unit conversions in that one expression, and both are easy to get wrong.

Radius versus width. The band radii are fractions of the dial's *width*. A `radial-gradient` stop with `closest-side` on a square element is a fraction of the gradient's *radius*, which is half the width. So the value has to be doubled: `19` becomes `38%`.

The border straddle. The innermost hairline is drawn *on* radius 19, centred on it, so half its 2px width falls inside the hole. If the mask cut exactly at 19, that half would be masked away and the innermost line would render at half weight — subtly lighter than the other three, for no visible reason. So the mask is pulled inward by the border width: `calc(38% - var(--dial-level-w))`.

Both of these are the kind of detail that is invisible when right and inexplicable when wrong, which is precisely why the arithmetic lives in one place with a comment rather than being distributed as magic numbers.

11.3 The hairlines

The boundaries between decks are drawn, not implied. Four elements — one per entry in `bounds` — each a circle with a border:

ONE HAIRLINE PER BAND BOUNDARY — `components/radial/radial.css:1118–1149`

```

1118 /* — Level borders —————
1119   One hairline circle per band boundary (radii supplied by RadialHub from the
1120   same geometry that carves the sectors). They are what separates the decks:
1121   the gap alone reads as a missing fill, a drawn edge reads as structure. */
1122 .radial-hub__levels {
1123   position: absolute;
1124   inset: 0;
1125   pointer-events: none;
1126 }
1127 .radial-hub__level {
1128   position: absolute;
1129   left: 50%;
1130   top: 50%;
1131   /* `--r` is a fraction of the dial's WIDTH, so the diameter is twice it. The
1132      extra border-width straddles the line ON the boundary instead of just
1133      inside it, which matters for the innermost one: the dial's mask clears
1134      everything within the hole, and a border drawn wholly inside that radius
1135      would be masked away. */
1136   width: calc(var(--r) * 2 + var(--dial-level-w));
1137   height: calc(var(--r) * 2 + var(--dial-level-w));
1138   transform: translate(-50%, -50%);
1139   border-radius: 50%;
1140   border: var(--dial-level-w) solid var(--dial-level-c);
1141   /* Spins in with the decks so the borders never look pasted over the motion. */
1142   opacity: 0;
1143   transition:
1144     opacity 0.34s ease,
1145     transform 0.55s var(--ease-glass, cubic-bezier(0.16, 1, 0.3, 1));
1146 }

```

```

1147 .radial-hub[data-phase='open'] .radial-hub__level {
1148   opacity: 1;
1149 }

```

Two decisions in there are worth pulling out.

Real bordered elements, not gradient stops. The obvious alternative is hard colour stops in the bed's own `radial-gradient`. It is cheaper and it antialiases badly: a hard stop in a gradient is resolved at whatever resolution the gradient is rasterised at, and at some dial sizes it produces a visibly stepped ring. A bordered element is antialiased by the same path that draws every other rounded box on the page, at any size.

Width, again, not radius. `--r` is a percentage of the dial's width, and the element needs a diameter, so `width: calc(var(--r) * 2 + var(--dial-level-w))`. The extra border width is the same straddle correction as the mask, from the other side: it centres the line on the boundary rather than placing it just inside.

11.4 Why a drawn edge rather than a wider gap

The gap between the decks is only 2 units — about 9 pixels at the largest dial — and an earlier instinct was to widen it instead of drawing a line. The reason that is wrong is stated compactly in the CSS:

They are what separates the decks: the gap alone reads as a missing fill, a drawn edge reads as structure.

COMPONENTS/RADIAL/RADIAL.CSS — THE LEVEL BORDERS

This is a real perceptual difference and not a matter of taste. An unfilled channel between two rings of sectors is ambiguous — it could be a boundary, or it could be a row of sectors that failed to render. A line is unambiguous: something drew that, on purpose. The dial has two decks, and the line is the sentence that says so.

It matters more once the decks start moving. During the open animation the two rings counter-rotate into alignment, which means mid-animation the sectors are not where the structure says they should be. The hairlines are the only thing holding the dial's shape together at that moment, which is why they fade in on the same curve as the wedges rather than appearing at the end.

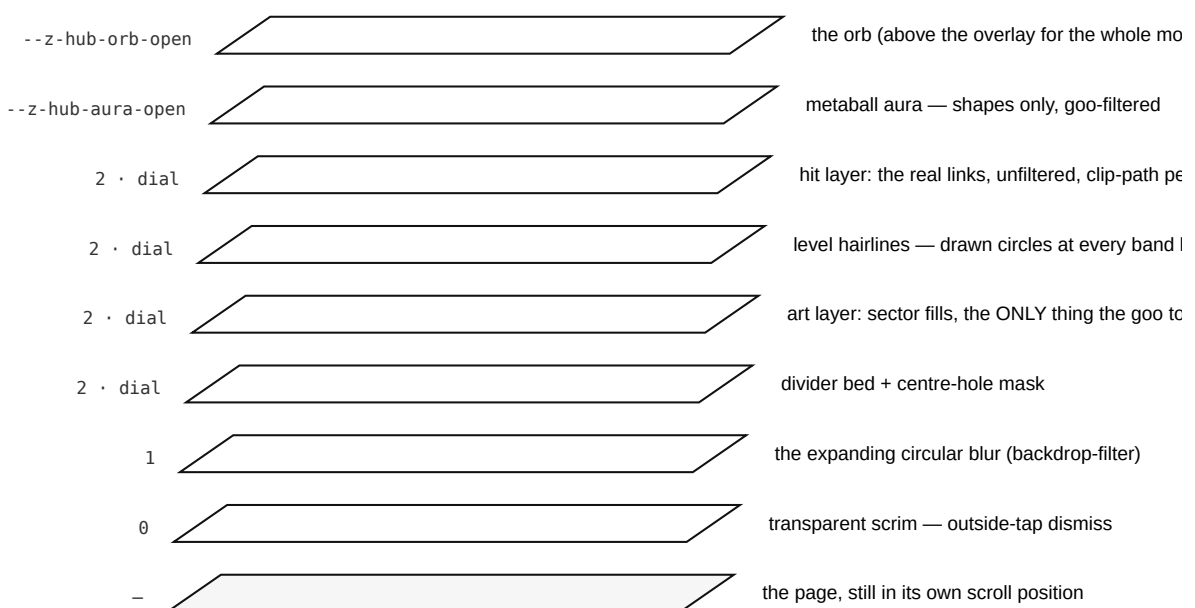


Figure 11.1 — The hub's layer stack, exploded. The hairlines are painted between the two wedge layers: above the goo-filtered art, which would chew them, and below the links, which must keep every pixel of their own sector. They are `pointer-events: none` regardless.

CHAPTER TWELVE

Labels, centroids and budgets

"Adaptive Intelligence" is nineteen characters. It has to fit in a sector that is 36° wide and 64 pixels deep, without escaping a clip-path that will silently cut it in half. This chapter is about the single most fiddly geometry in the dial, and about a bug — AUD-504(c) — that is a small lesson in which direction a text box grows.

12.1 The centroid

Each destination's glyph and label are positioned at the centroid of its visible band — the midpoint of its angular span, at the mid-radius of its ring:

```
const rm = (band.r0 + band.r1) / 2;  
const am = ((a0 + a1) / 2) * DEG;  
cx: 50 + rm * Math.cos(am),  
cy: 50 + rm * Math.sin(am),
```

These are handed to the label wrapper as inline `left` and `top` percentages, and the wrapper is pulled back by half its own size with `transform: translate(-50%, -50%)`. So the label box is centred on the centroid, which is the visual centre of the sector.

As noted in Chapter 8, the annulus form makes this stable: `rm` does not depend on the sector's angle, so all six inner glyphs sit on one circle at radius 26.5 and all ten outer glyphs on another at radius 43, no matter how the counts change.

12.2 The direction the box grows

Here is the bug. A label box centred on a centroid grows in two directions, and they are not equivalent.

Width grows angularly. The box is axis-aligned, but the sector is a wedge. Growing the box horizontally pushes its corners across the sector's radial edges — across the `clip-path` — and the clip cuts them off. Silently. There is no overflow indicator, no ellipsis; the text simply stops at the sector boundary.

Height grows radially. Growing the box vertically pushes it into the band's depth, where the inner deck has 69 pixels and the outer 64. There is room there.

The first implementation set a generous `max-width` and let long names take one line. The audit's finding is the predictable result: *"Adaptive Intelligence" clips at its wedge edge, "Predictions" overflows the disc*. Two different failures from the same cause — one name was cut by the clip, the other was long enough to reach past the rim before the clip caught it.

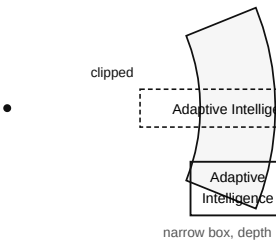


Figure 12.1 — The same label in a 44° outer sector. Above: a wide box, whose corners cross the sector's radial edges and are cut by the clip-path. Below: a narrow box that wraps to two lines, spilling into the band's depth where there is room.

The fix, therefore, is counter-intuitive: make the boxes *narrower* than the band is deep, on purpose, and let long names wrap. The comment in the CSS states it plainly:

THE PER-RING LABEL BUDGET — components/radial/radial.css:1222–1234

```
1222 /* Per-ring label budget: the inner ring has the shorter arc, so its labels get
1223 less room and clamp to a single line; the outer ring can afford two. */
1224 /* Narrower than the band is deep, on purpose. These boxes are centred on the
1225 sector's centroid, so width spills ANGULARLY – past the sector's clip-path,
1226 which cuts it ("Adaptive Intelligence" lost its tail, "Predictions"
1227 overflowed the disc). Height spills radially, into the band, where there is
1228 room. So: keep the boxes narrow and let long names take another line. */
1229 .radial-hub__wedge-inner[data-ring='0'] {
1230   max-width: 4.1rem;
1231 }
1232 .radial-hub__wedge-inner[data-ring='1'] {
1233   max-width: 5.4rem;
1234 }
```

12.3 The budgets, and why they differ

LABEL BUDGET PER DECK

DECK	MAX-WIDTH	≈ CHARS/LINE	BAND DEPTH	LINES THAT FIT
inner (ring 0)	4.1rem / 66px	9–11	69px	2 + icon
outer (ring 1)	5.4rem / 86px	12–14	64px	2 + icon

The inner deck gets *less* width despite having wider sectors, which looks backwards until you remember what constrains it: the inner deck sits at a smaller radius, so the same angular span is a shorter arc, and it is arc length that the box's width has to fit inside. Wide angle, small radius, short chord.

The label itself is clamped to three lines with `overflow-wrap: anywhere` and `hyphens: auto`, so a very long single word — a hypothetical destination named something unhelpful — degrades to a hyphenated wrap rather than escaping its box. That clamp is a hard bound: a destination name can never spill across its neighbours, whatever it is, in whatever locale.

AUD-504(C), AND THE ONE-LINE CLAMP THAT MADE IT WORSE

An intermediate fix clamped labels to a single line, which stopped the clipping and introduced a new failure: "Communities" became "Communi...". The comment recording the correction is precise about why two lines are affordable — *"the inner band is deep enough for two lines (icon + label ≈ 50px in a ~69px band at the largest dial), so the shorter arc costs width, not lines."* Depth was never the scarce resource. Width was.

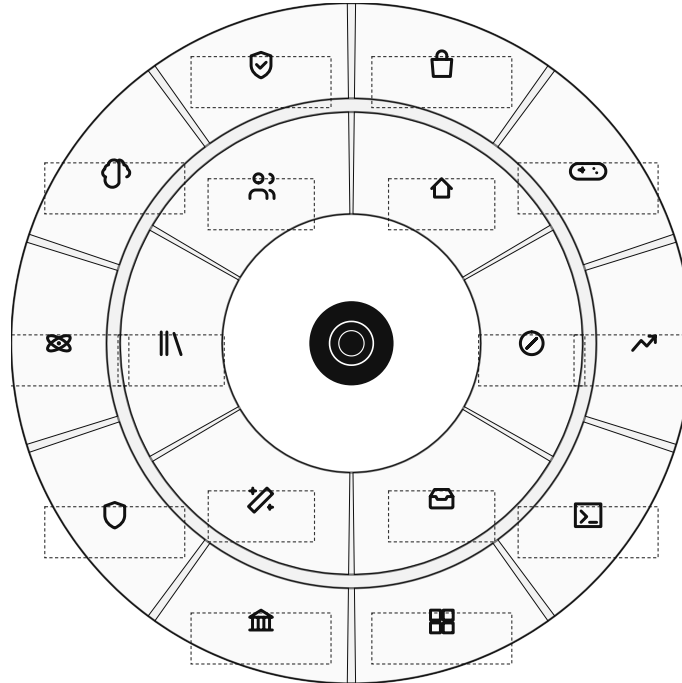


Figure — The per-ring label budget as boxes: 4.1rem on the inner deck, 5.4rem on the outer. Each box is centred on its sector's centroid, so width spills angularly — through the clip — while height spills radially, into the band, where there is room.

12.4 Icon-only below 480px

Under 480 pixels the labels are not rendered at all. The dial at 320px is 275 pixels across; the outer band is 38 pixels deep and its sectors are 36° wide, which at that radius is about 37 pixels of arc. There is no typography that puts "Adaptive Intelligence" in a 37×38 box.

So the phone dial is sixteen glyphs. The audit's judgement on that was blunt and correct — *sixteen unlabelled glyphs as the site's sole navigation* is recall, not recognition — and the fix is the centre caption in Chapter 24, which puts the name of the focused destination in the one place on a 320px dial that has room for a word: the hole in the middle.

The breakpoint itself is 480px, registered as the platform's `xs` breakpoint. Above it, labels appear and the caption is hidden, because each wedge then carries its own name and a caption would be a duplicate. The two mechanisms are exact complements: exactly one of them is showing a label at any width.

12.5 Where the glyph sits inside the box

The glyph and label are a two-row grid with a 0.3rem gap, icon above label, both centred. The icon is 1.3rem square; the label is 0.6875rem at weight 600 with 1.1 line-height. Total for icon plus two lines of label: about 50 pixels, in a 64-to-69-pixel band. That is the arithmetic quoted in the comment above, and it is the reason the label budget can afford two lines rather than one.

One detail that matters for input: the whole inner wrapper is `pointer-events: none`. Clicks fall through the glyph to the clipped anchor underneath. Without that, the label box — an axis-aligned rectangle overlapping several sectors'

territory near the dividers — would steal clicks for its own sector from its neighbours, in a region where the user is aiming at a visible boundary. Chapter 22 covers the rest of the hit-testing chain.

CHAPTER THIRTEEN

The art bleed

One constant, `ART_BLEED_DEG = 0.25`, and a comment four times its length. It exists because the goo filter eats a couple of pixels off every edge it touches, and because the correct compensation is angular but not radial.

13.1 What the filter does to an edge

The goo filter blurs its input and then runs the blurred alpha through a steep contrast ramp, which snaps it back to a hard edge. Chapter 19 covers the mechanism; what matters here is a side effect. The ramp's threshold sits at roughly the halfway point of the blurred alpha gradient, so the recovered edge is not where the original edge was — it is pulled inward, by an amount that depends on the blur radius and on how much of the neighbourhood was filled.

For a large shape, that inward pull is a couple of pixels and nobody notices. For the 3-pixel channel between two sectors, both sides pull inward by a couple of pixels and the channel becomes 7 or 8 — which is no longer a hairline divider but a visible gutter. The dial's careful proportional gap arithmetic from Chapter 10 is undone by the filter that was supposed to make it look liquid.

13.2 The compensation

So the decorative layer draws each sector slightly wider than the hit layer, and only in the angular direction:

WHY THE ART LAYER IS BLED — `components/radial/RadialHub.tsx:53–62`

```
53 /**
54  * The art layer is drawn slightly wider ANGULARLY than the hit layer. The goo
55  * filter blurs-then-thresholds each sector, which erodes a couple of pixels off
56  * every edge; without the bleed that erosion turns the thin angular dividers
57  * inside a ring into wide gaps. It is deliberately NOT applied radially: the
58  * boundaries between levels are meant to be pronounced, so the erosion there is
59  * welcome and the drawn hairlines sit on top of it. Hit testing always uses the
60  * exact geometry, so the bleed can't steal a neighbour's clicks.
61  */
62 const ART_BLEED_DEG = 0.25;
```

The bleed is a fraction of the gap rather than a fixed angle, which keeps it proportional to what it is compensating for. At the shipped counts the gap is capped at 1.4°, so the bleed is 0.35° on each side — a 0.7° total widening of a 22.5° sector, or about 3%.

BLEED MAGNITUDE, AT THE MID-RADIUS OF EACH DECK ON A 460PX DIAL

DECK	GAP	BLEED EACH SIDE	PX EACH SIDE	CHANNEL BEFORE	CHANNEL AFTER
inner, n=6	1.40°	0.35°	0.74px	3.0px	1.5px
outer, n=10	1.40°	0.35°	1.21px	4.8px	2.4px

The bleed roughly halves the drawn channel, and the filter's erosion then reopens it to something close to the original width. The numbers are small enough that this reads as tuning rather than as engineering, and that is a fair description — it was arrived at by looking at the dial, which is the only way to arrive at a number whose correct value is "the amount a threshold on a blur removes".

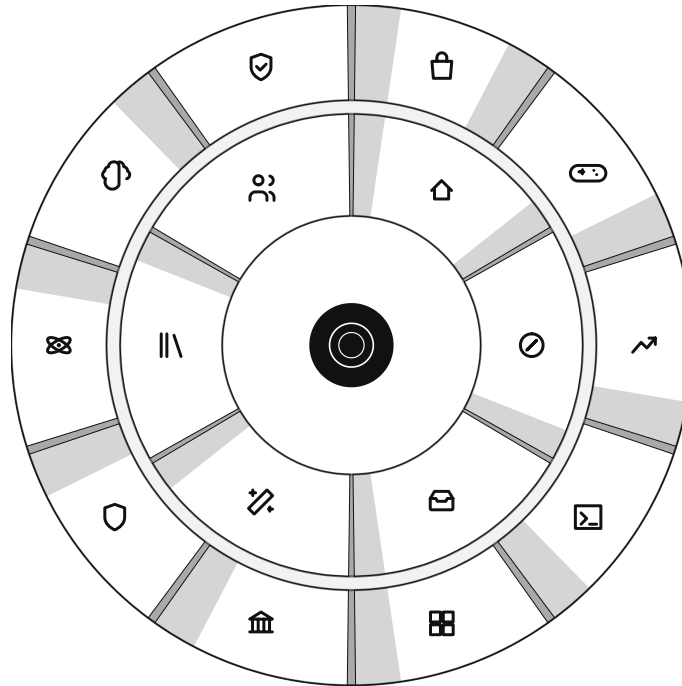


Figure — `ART_BLEED_DEG`, exaggerated $\times 26$ for legibility. Grey is the art sector, outlined white is the hit sector. The bleed is angular only: it pays back the couple of pixels the goo threshold erodes off each divider, and never touches the radial boundaries, where erosion is welcome.

13.3 Why not radially

The interesting half of the decision is what the bleed deliberately does *not* do. It is applied to the angles only; the radii passed to `ringSector()` for the art layer are the band's own, unmodified.

Radially, the erosion is welcome. The boundaries between decks are meant to be pronounced — that is the entire argument of Chapter 11 — and a filter that pulls the sectors slightly back from those boundaries makes the drawn hairlines sit in a clean channel rather than being crowded by fill on both sides. Compensating there would fight the structure the hairlines are drawing.

So the same filter artefact is a defect in one axis and a feature in the other, and the constant is named and commented so that the asymmetry looks deliberate rather than forgotten.

THE INVARIANT THE BLEED MUST NOT BREAK

Hit testing always uses the exact geometry. The bled polygon is only ever applied to the `aria-hidden` art layer, which is `pointer-events: none`; the anchors use `w.clip`, never `w.clipArt`. So the bleed cannot steal a neighbour's clicks, and a user pressing a pixel that *looks* like it belongs to one sector but geometrically belongs to another gets the geometric answer. That is a 1-pixel discrepancy at the dividers, in exchange for never having a target whose visible extent and clickable extent disagree by more than that.

13.4 What this chapter is really about

A quarter of a degree is a preposterous thing to have a constant for. It is in this book because it is a compact example of what building the dial actually consisted of: not the radial idea, which took an afternoon, but a long tail of interactions between a geometric construction and the rendering pipeline it is drawn by.

The dial's shape is defined by clip paths. Clip paths bound hit-testing. Hit-testing has to be exact. Filters move edges. Filters must not touch text. Text inherits colour from the element it is inside, which must therefore be the sector.

Blend modes composite against a stacking context, not against what you see. Masks measure radius where the geometry measures width. Every one of those is a constraint imposed by the platform rather than by the design, and the design's job was to survive all of them without becoming a special case.

The radial idea took an afternoon. The quarter of a degree took a week.

PART FOUR

Motion

Nine layers, one duration, one attribute, and no animation library. This part is how the hub opens: the synchronous 500ms, the three-state machine, the two decks that spin in from opposite directions, and the plate that flashed black when they started doing it.

14 One synchronous motion

15 The phase machine

16 Counter-rotation, and the bed that flashed black

17 The blur, and the degradation ladder

CHAPTER FOURTEEN

One synchronous motion

The orb glides, the blur expands, the dial scales and fades, two rings of sectors counter-rotate, four hairlines appear, and a capsule rises from the bottom edge. All of it in half a second, all of it on one clock, none of it orchestrated by anything but shared duration constants.

14.1 Why 500ms

Half a second is slow for a menu. The conventional range for an overlay is 150–250ms, and anything past 300ms is usually a sign that someone fell in love with an easing curve.

The hub is long deliberately, because the motion is doing work that a fade cannot. The orb has to travel roughly 40% of the viewport's height, and the user has to be able to *follow* it — the entire identity claim of Chapter 5, that the menu grows out of the thing you touched, only lands if the eye can track the thing as it moves. Travel that distance in 200ms and the orb does not glide, it teleports, and the connection between the button and the dial becomes an inference again.

The easing does most of the perceptual work: `cubic-bezier(0.16, 1, 0.3, 1)`, the platform's `--ease-glass`. It is aggressively front-loaded — roughly 80% of the distance is covered in the first 40% of the duration — so the motion *feels* fast while lasting long enough to be followed. The audit's own note confirms the subjective result: the bloom reads as "*fully formed by about 120ms*", despite the declared duration being four times that. The tail is settling, not waiting.

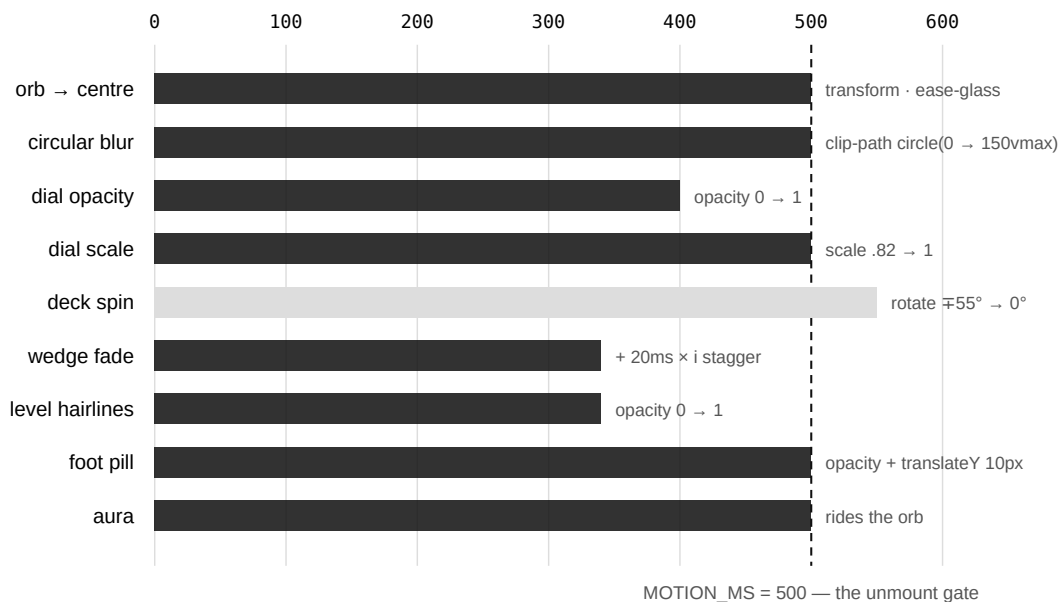


Figure 14.1 — The open motion. Nine animated properties across seven elements, sharing one duration family. `MOTION_MS = 500` is also the unmount gate: the overlay stays mounted for exactly this long on close, so every layer finishes animating out before it disappears.

14.2 The duration family

Not everything is exactly 500ms, and the deviations are deliberate.

DECLARED DURATIONS, AND WHY EACH DIFFERS

LAYER	DURATION	CURVE	REASON
orb transform	500ms	ease-glass	The reference. Everything else is tuned against it.
orb box-shadow	280ms	ease	Shadow is a lighting cue, not a position cue; it should settle before the travel ends.
blur clip-path	500ms	ease-glass	Locked to the orb — the frost must arrive with the orb, not after it.
dial transform	500ms	ease-glass	Locked to the orb.
dial opacity	400ms	ease-glass	Deliberately shorter than its own scale, so the dial is fully opaque slightly before it stops growing. Reaching full opacity last would read as fading in rather than blooming.
wedge transform	550ms	ease-glass	Longest in the set. The spin is the largest visual displacement, and it is the last thing to settle — the dial arrives, then locks.
wedge opacity	340ms	ease	Plus a 20ms-per-index stagger. Short so that the stagger is perceptible without the last wedge lagging.
hairline opacity	340ms	ease	Matches the wedges, so structure and fill arrive together.
foot opacity	400ms	ease	Matches the dial's opacity.
foot transform	500ms	ease-glass	A 10px rise, locked to the orb.
aura transform	500ms	ease-glass	Must match the orb <i>exactly</i> ; any mismatch and the liquid mass detaches from the disc it belongs to.
scrim opacity	280ms	ease	The legibility scrim under the docked orb retracts quickly — the overlay owns the screen now.
wedge hover	180ms	ease	A hover response, on a different clock entirely. It must feel instant.

The rule visible in that table: anything whose job is to communicate *position* is locked to 500ms and the glass curve; anything whose job is to communicate *presence* is shorter and on a plain ease. Position must be synchronous or the illusion breaks. Presence can be staggered, and is, because a stagger is what makes sixteen simultaneous arrivals read as a bloom rather than a flash.

14.3 The stagger, and what it is applied to

Each wedge wrapper carries its bloom order as a custom property, and the open state turns that into a delay:

THE STAGGER, ON OPACITY ONLY — `components/radial/radial.css:1099–1116`

```

1099 .radial-hub__wedge-wrap {
1100   opacity: 0;
1101   transform: scale(0.9) rotate(var(--ring-spin, 0deg));
1102   transition:
1103     opacity 0.34s ease var(--fade-delay, 0ms),
1104     transform 0.55s var(--ease-glass, cubic-bezier(0.16, 1, 0.3, 1)) 0ms;
1105 }
1106 .radial-hub__wedge-wrap[data-ring='0'] {
1107   --ring-spin: -55deg;
1108 }
1109 .radial-hub__wedge-wrap[data-ring='1'] {
1110   --ring-spin: 55deg;
1111 }
1112 .radial-hub[data-phase='open'] .radial-hub__wedge-wrap {
1113   opacity: 1;
1114   transform: scale(1) rotate(0deg);
1115   --fade-delay: calc(var(--i) * 20ms);
1116 }

```

Two details are load-bearing.

The stagger is on opacity, not on transform. Sixteen wedges $\times$ 20ms is a 320ms spread; applied to the rotation, that would mean each sector arriving at its resting angle at a different moment, and a ring would appear to unwind rather than turn. Applied to opacity, the ring turns as one body while its members fade up in sequence. The comment in the source says exactly this: *"the spin is deliberately NOT staggered: every wedge of a ring shares one delay, so the ring turns as a body."*

The stagger only applies on open. The delay lives in the `[data-phase='open']` block, so on close it reverts to 0ms and all sixteen fade together as the blur contracts. Closing a menu should be one gesture, not a reverse choreography — the user has already decided; the interface should get out of the way at once.

The order itself runs inner deck first, then outer, in catalog order, because `order` is assigned as the groups are walked. So the bloom radiates outward, which is the same direction the whole design says things move in.

14.4 What is not animated

Three absences are worth noting, because each was a decision.

No motion on the glyphs. The icons and labels do not scale, rotate or slide; they inherit their sector's opacity and nothing else. Animating text inside a rotating element means animating text through sub-pixel positions, which on most rendering paths means animating text through several different antialiasing decisions — the visible symptom is shimmering type. The glyphs simply fade with their sector.

No motion on the hairlines' geometry. They fade, they do not grow. A hairline circle scaling up would sweep its 2px stroke across a range of radii, and a stroked circle at non-integer scale is where antialiasing artefacts live. They are declared with a transform transition — so the property is there if it is ever needed — but the open state changes only opacity.

No colour transitions during the bloom. Sector fills are static during open and close; the only colour transitions in the hub are the 180ms hover and focus responses. Animating a `color-mix()` of two tokens is not something to do sixteen times per frame for aesthetic reasons.

THE MOTION CONTRACT, STATED ONCE

Transform and opacity only. Every animated property in the hub is one of those two, plus `clip-path` on the blur — all compositor-friendly, none of them triggering layout. There is no animated width, height, top, left, margin, padding, border-radius or filter anywhere in the module. That is a platform-wide rule, and it is what makes a nine-layer synchronous animation viable on a phone.

14.5 Why there is no animation library here

The rest of the site uses `framer-motion` freely, and the hub deliberately does not. Its docblock says so in one line: *"framer-motion-free (CSS transitions/clip-path) to stay light on mobile."*

The reasoning is a bundle question and a correctness question. The bundle question is obvious: this component ships on every page of the site, including to phones, and its animation needs are fully expressible in CSS. The correctness question is subtler and more interesting — a JavaScript animation runs on the main thread's clock, which means a busy main thread produces a stuttering menu, and the main thread on a page that has just started rendering a feed is busy. CSS transitions on transform and opacity run on the compositor. The hub opens smoothly while the page behind it is still hydrating, which is exactly the moment a user is most likely to open it.

The cost is that everything has to be expressible as a declarative state, which is a real constraint — Chapter 16 is a bug that exists precisely because of it. That is a fair price for a menu that does not stutter on a mid-range Android during hydration.

CHAPTER FIFTEEN

The phase machine

Three states, one string, one timer. The interesting one is `closing`, which exists for a reason that catches out a great many overlay components: a component that unmounts its overlay on close cannot have a close animation.

THE PHASE TYPE — `components/radial/RadialHub.tsx:14–18`

```
14 type HubUser = { id: string; handle?: string | null; isAdmin?: boolean };
15
16 /** closed → open (orb glides to centre AS the menu blooms) → closing → closed. */
17 type Phase = 'closed' | 'open' | 'closing';
18
```

15.1 The problem `closing` solves

The naive shape for a menu is a boolean. `open ? <Overlay/> : null`, or a class that is toggled. Both have the same defect: the moment the state flips to closed, the overlay is gone — either unmounted from the tree or set to `visibility: hidden` — and a transition on an element that is no longer rendered does not run. The menu vanishes instead of closing.

The usual workarounds are an animation library that manages exit transitions, or a `transitionend` listener, or an `animation-fill-mode` trick. The hub takes the simplest available option: a third state that means "still visible, animating out", and a timer that leaves it.

CLOSE() AND TOGGLE() — `components/radial/RadialHub.tsx:205–220`

```
205 const close = useCallback(() => {
206   clearTimer();
207   setPhase((p) => (p === 'closed' ? p : 'closing'));
208   // Keep the overlay mounted while every layer animates out, then hide it.
209   timerRef.current = setTimeout(() => setPhase('closed'), reduced ? 0 : MOTION_MS);
210 }, [reduced]);
211
212 const toggle = useCallback(() => {
213   clearTimer();
214   setPhase((p) => {
215     // Opening is a single synchronous transition – no pre-centring step.
216     if (p === 'closed') return 'open';
217     timerRef.current = setTimeout(() => setPhase('closed'), reduced ? 0 : MOTION_MS);
218     return 'closing';
219   });
220 }, [reduced]);
```

Both handlers clear the pending timer first. That is what makes the machine interruptible: tap the orb three times quickly and you do not accumulate three timers racing to set the phase, you get one timer, restarted, and the CSS transitions reverse from wherever they currently are.

15.2 The CSS side of the same idea

The overlay's visibility is keyed to both live phases, not just `open`:

THE OVERLAY IS MOUNTED FOR OPEN AND CLOSING — components/radial/radial.css:931–944

```

931 .radial-hub__overlay {
932   position: fixed;
933   inset: 0;
934   z-index: var(--z-hub-overlay);
935   display: grid;
936   place-items: center;
937   visibility: hidden;
938   pointer-events: none;
939 }
940 .radial-hub[data-phase='open'] .radial-hub__overlay,
941 .radial-hub[data-phase='closing'] .radial-hub__overlay {
942   visibility: visible;
943   pointer-events: auto;
944 }

```

And every animated layer has the same shape: a resting declaration, and an override under `[data-phase='open']`. The `closing` phase deliberately has almost no rules of its own — it inherits the resting state, which is precisely how the reversal happens. Two exceptions exist, both about stacking: the orb and the aura keep their raised z-index during `closing`, so the orb stays above the overlay for the whole return journey rather than popping behind the frost halfway home.

15.3 The timer's reduced-motion case

Both the close paths use `reduced ? 0 : MOTION_MS`. Under reduced motion there are no transitions to wait for, so waiting half a second before unmounting would leave an invisible, non-interactive overlay covering the screen — technically `pointer-events: auto`, technically transparent, and completely indefensible.

This is one of only two places in the component where a preference is read in JavaScript rather than in CSS, and it is the exception that proves the rule: the platform's guidance is that degradation belongs in the token layer, and the reason this one cannot is that a `setTimeout` is not a CSS property. A media query cannot shorten a timer.

15.4 Escape, wheel, and the effects that own them

KEYBOARD AND WHEEL, WHILE THE MENU IS LIVE — components/radial/RadialHub.tsx:234–249

```

234 useEffect(() => {
235   if (phase === 'closed') return;
236   const onKey = (e: KeyboardEvent) => {
237     if (e.key === 'Escape') close();
238   };
239   const onWheel = (e: WheelEvent) => {
240     if (e.cancelable) e.preventDefault();
241   };
242   const overlay = overlayRef.current;
243   window.addEventListener('keydown', onKey);
244   overlay?.addEventListener('wheel', onWheel, { passive: false });
245   return () => {
246     window.removeEventListener('keydown', onKey);
247     overlay?.removeEventListener('wheel', onWheel);
248   };
249 }, [phase, close]);

```

Three things about that effect.

It is keyed on `phase` and returns early when closed, so no global listener exists while the menu is shut — a hub on

every page of the site must not leave a `keydown` handler on the window at all times.

The wheel listener is `{ passive: false }`, because it calls `preventDefault()`, and a passive listener may not. It is attached to the overlay rather than the window so it cannot suppress scrolling anywhere else, and it checks `e.cancelable` first, because a wheel event during an already-started scroll may not be cancellable and calling `preventDefault()` on it is a console warning for nothing.

It exists at all because `touch-action: none` — the technique that blocks touch panning, Chapter 25 — does not cover pointer devices. Two mechanisms, two input classes, one outcome.

15.5 Focus, on the next frame

MOVING FOCUS INTO THE BLOOMED DIAL — `components/radial/RadialHub.tsx:251–257`

```
251   useEffect(() => {
252     if (phase !== 'open') return;
253     const raf = requestAnimationFrame(() =>
254       dialRef.current?.querySelector<HTMLElement>('a[href], [tabindex]')?.focus(),
255     );
256     return () => cancelAnimationFrame(raf);
257   }, [phase]);
```

The `requestAnimationFrame` is not decoration. On the render that sets the phase to `open`, the dial's links have just had their `tabIndex` changed from `-1` to `0` and the overlay's visibility has just changed; focusing synchronously in that same commit can land on an element the browser still considers unfocusable. Deferring by one frame puts the focus call after style resolution.

The query — `a[href], [tabindex]` — takes the first focusable thing in the dial, which is the first wedge, which is the first catalog entry, which is *Home*. So a keyboard user's focus lands on the most likely destination without any explicit initial-focus configuration. The cleanup cancels the frame, so a menu opened and closed within a single frame does not steal focus after the fact.

WHAT THIS FOCUS HANDLING DOES NOT DO

There is no focus trap. Tab from the last wedge and focus leaves the dial for the foot pill, and beyond that for the document. There is also no explicit focus restore to the orb on close. Both are noted in Chapter 30 as gaps: a modal navigation overlay conventionally traps focus and returns it, and this one does neither. What it does have is `tabIndex={-1}` on every control while closed, so the closed hub is genuinely absent from the tab order rather than being a set of invisible tab stops — which is the more common and more serious version of this bug.

15.6 The whole machine, in one place

PHASE TRANSITIONS

FROM	TRIGGER	TO	SIDE EFFECTS
closed	orb click	open	Clear timer. Overlay becomes visible and interactive; all nine layers begin animating in; focus moves to the first wedge on the next frame; the docked scrim retracts.
open	orb click	closing	Clear timer, set a new one for <code>MOTION_MS</code> .
open	Escape	closing	As above, via <code>close()</code> .
open	scrim press	closing	As above.
open	any menu item	closing	As above; the router navigates in the same tick, so the dial animates out over the incoming page.

PHASE TRANSITIONS (CONT.)

FROM	TRIGGER	TO	SIDE EFFECTS
open	foot control	closing	As above. Sign-out additionally awaits the auth client, then navigates and reloads.
closing	timer	closed	Overlay hidden and non-interactive; hovered-label state cleared by an effect keyed on the phase.
closing	orb click	open	Clear timer. Every transition reverses from its current interpolated value.
any	unmount	—	Timer cleared by an effect's cleanup, so no <code>setState</code> after unmount.

Nine rows, three states, and one genuinely valuable property: the row that matters most is the last-but-one. A user who taps the orb twice in rapid succession — which people do, constantly, on any control that takes half a second to respond — gets a smooth reversal rather than a queue of animations.

CHAPTER SIXTEEN

Counter-rotation, and the bed that flashed black

The two decks arrive by spinning into alignment from opposite directions — inner anticlockwise, outer clockwise. It is the dial's signature gesture, it cost one line of CSS, and it broke the dial the day it landed, in a way that is a small classic of front-end debugging.

16.1 The gesture

TWO DECKS, TWO DIRECTIONS — components/radial/radial.css:1106–1116

```
1106 .radial-hub__wedge-wrap[data-ring='0'] {
1107   --ring-spin: -55deg;
1108 }
1109 .radial-hub__wedge-wrap[data-ring='1'] {
1110   --ring-spin: 55deg;
1111 }
1112 .radial-hub[data-phase='open'] .radial-hub__wedge-wrap {
1113   opacity: 1;
1114   transform: scale(1) rotate(0deg);
1115   --fade-delay: calc(var(--i) * 20ms);
1116 }
```

That is the entire effect. Each wedge wrapper is `inset: 0` over the whole dial square, so its transform origin is already the dial's centre and a rotation carries its clipped sector around that centre — no `transform-origin` declaration needed. The resting state is rotated by $\pm 55^\circ$; the open state is zero. Every wedge in a ring shares its ring's angle, so the ring turns as a body.

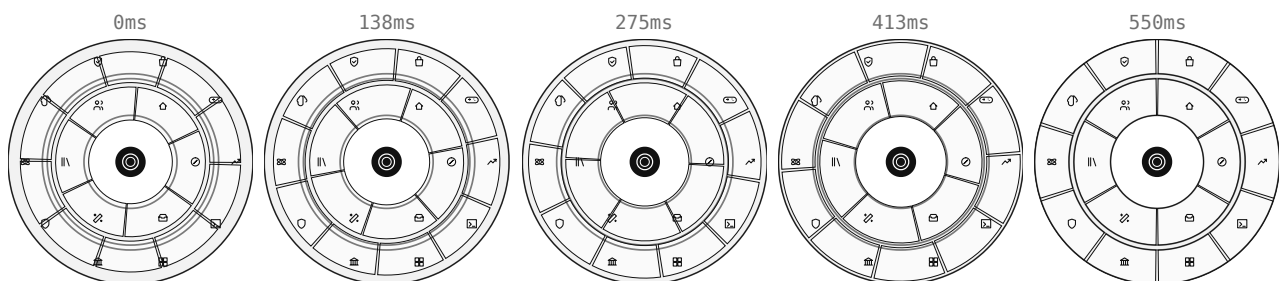


Figure 16.1 — Five frames of the open, from the resting $\pm 55^\circ$ to alignment. The decks pass through each other's angular territory on the way, which is what makes the two levels read as independent rings rather than as one disc with a line through it — and which is also what uncovers the plate underneath them.

The reason for opposite directions is legibility of structure. Two rings rotating the same way read as one object turning. Two rings rotating against each other cannot be anything but two objects, and the viewer learns the dial's most important structural fact — that it has two decks — from the motion, before any of the labels have finished fading in.

Fifty-five degrees is a little more than one inner sector (60°) and a little more than one and a half outer ones (36°). That is deliberate: the rotation is large enough that every sector visibly passes through a neighbour's resting position, so the eye cannot mistake the motion for a settling wobble.

16.2 The bug

Commit `d308f26` added the spin and the hairlines together. It also, without touching it, broke the dial's background.

The dial's divider bed had been `--site-border-bright` — which in the default theme is pure black. That was a perfectly reasonable choice for as long as the sectors only *scaled*: at every frame of a scale animation the sectors cover essentially the whole annulus, so the bed is visible only in the 3-pixel channels between them, where a strong dark line is exactly what is wanted.

The moment the sectors *rotate*, that stops being true. A ring rotated by 55° relative to its resting position does not cover the same annulus — it covers a rotated copy of it, and because the sectors are clipped to a fixed dial box, rotating the wrapper sweeps each sector's polygon around while the dial's own circular boundary stays put. Large regions of the bed are uncovered mid-animation. Uncovered black bed, on a white page, for a few hundred milliseconds, sixteen times over: **the whole dial flashed black on open.**

THE SHAPE OF THE FAILURE

Nothing in the diff touched the background. The background's value had been correct for the entire life of the component. What changed was the *coverage* of the layer above it — and a value whose correctness depends on always being covered is a value with an undeclared dependency on the animation. This is a general hazard with "it never shows anyway" reasoning: it is true until a motion change makes it false, and then it is a full-screen flash rather than a subtle regression.

16.3 The fix, and why it is not just a colour change

The bed became a light plate — `color-mix(in srgb, var(--site-surface-opaque) 86%, var(--site-text) 14%)` — a shade darker than the sectors that sit on it rather than an emphatic dark line. The comment recording it is one of the more useful in the codebase:

THE BED, AND WHY IT IS LIGHT — `components/radial/radial.css:989–999`

```

989 /* — Wedge (ring) navigation dial ————— */
990 /* The dial is also the DIVIDER BED: the sectors tile it with thin angular gaps,
991    so this background is what shows through them as the hairlines between
992    wedges, and it stops the disc dissolving into a light page.
993
994    It is a light plate, NOT `--site-border-bright` (which is pure black in the
995    default theme). That was fine while the sectors only scaled – they covered the
996    bed at every frame – but they now SPIN in from opposite angles, which uncovers
997    large wedges of bed mid-animation and made the whole dial flash black on open.
998    The level borders below draw the structure that the black bed used to imply,
999    so the plate only has to be a shade darker than the sectors that sit on it. */

```

The important clause is the last one: *"the level borders below draw the structure that the black bed used to imply, so the plate only has to be a shade darker than the sectors that sit on it."*

The two halves of commit `d308f26` are not independent. The spin uncovered the bed, which forced the bed to become light, which removed the strong dark structure the black bed had been providing, which is why the same commit had to add the drawn hairlines. Adding the spin without the hairlines would have produced a dial that no longer flashed but also no longer read as two decks. The commit's title — *"spin the double-decked dial open, and draw the borders between levels"* — is two changes because it had to be.

A value that is only correct while something covers it is not a value. It is a bet on the animation.

16.4 The uncovered-area arithmetic

It is worth quantifying, because the number explains why this was a flash rather than a flicker.

Each ring's sectors tile their annulus with 1.4° gaps, so at rest they cover about 97% of it. Rotating the ring by 55° does not reduce that coverage — the sectors still tile the annulus, just rotated. What is uncovered is not part of the annulus at all: it is the region where the rotated sector has swung *outside* the dial's circular clip and been cut off, leaving its former territory empty.

For the outer ring, whose sectors reach the rim, that loss is small. For the inner ring, at radius 19–34, the rotation carries sectors around a circle entirely inside the dial, so nothing is cut and nothing is uncovered — the inner deck simply turns.

What actually produced the flash was the combination with *scale*. The wrappers rest at `scale(0.9)` `rotate(±55deg)`. At 0.9 scale, every sector's outer radius is 45 rather than 50, and its inner radius is 17.1 rather than 19 — so the annulus the sectors occupy is smaller than the annulus the bed fills, in both directions, at every frame of the animation. A 5%-wide black band at the rim and a black ring at the hole boundary, growing thinner as the dial scales up. On a white page, that is a flash.

UNCOVERED BED DURING THE OPEN, PER FRAME (460PX DIAL)

T	SCALE	SECTOR RIM	BED RIM	UNCOVERED BAND
0.00	0.900	207.0px	230px	23.0px
0.25	0.925	212.8px	230px	17.3px
0.50	0.950	218.5px	230px	11.5px
0.75	0.975	224.3px	230px	5.8px
1.00	1.000	230.0px	230px	0.0px

A 23-pixel black ring at the outer edge of a 460-pixel disc, decaying to nothing over 550ms. Which, read as a table, is obvious, and which as a bug report reads as "the menu flashes black" — and is why the first instinct was to look at the animation rather than at a background colour that had not changed in weeks.

16.5 What the episode is worth

Three transferable observations.

1. **Scale-from-below plus a full-size backdrop is a flash waiting to happen.** If a layer rests at `scale(<1)` and something behind it is full size, the difference is visible for the whole animation. Either scale the backdrop with it, or make the backdrop something that is safe to see.
2. **Adding motion to a static layer changes the requirements of every layer beneath it.** The bug was not in the diff; it was in the interaction between the diff and an assumption made elsewhere.
3. **"Only visible in the gaps" should be written down.** Had the bed's original declaration carried a comment saying *this is only ever seen in the 3px dividers*, the person adding the spin would have had a chance to notice the assumption breaking. It now carries a comment several times longer than the declaration.

CHAPTER SEVENTEEN

The blur, and the degradation ladder

A circle clipped from radius zero to `150vmax`, and a set of media queries that decide how much of the hub's character survives on a phone, on a low-memory device, or for a user who has asked the operating system for less motion.

17.1 Why 150vmax

The blur's clip circle has to cover the viewport from a centre point that is the middle of the screen. Half the diagonal of the viewport would suffice — $\sqrt{w^2 + h^2} / 2$ — but CSS has no diagonal unit, and `50vmax` is only enough for a square viewport. `150vmax` is a comfortable over-provision: three times the larger dimension, which covers any aspect ratio anyone will ever hand it, including a 21:9 ultrawide.

Over-provisioning a clip radius costs nothing. The clipped element is `position: absolute; inset: 0`, so its box is the viewport regardless; the clip only decides how much of that box paints. A radius larger than the box is simply a fully-painted box. What the animation is actually interpolating is a number that stops mattering once it exceeds the corner distance — which means the visible expansion finishes somewhat before the transition does, and the last portion of the curve is spent invisibly. Given how front-loaded the easing is, that is a feature: the frost reaches the corners early and the tail of the transition is silent.

17.2 What the blur costs

`backdrop-filter: blur(20px) saturate(118%)` over a full viewport is the single most expensive thing the hub does. It requires the compositor to read back the content behind the element, blur it, and composite the result — every frame, while the clip radius is changing.

It is affordable for exactly one reason: **it is transient and modal**. It exists only while the menu is open, which is a few seconds at most, during which nothing else on the page is animating and the user is not scrolling — the overlay has blocked that. Compare this with the platform's standing budget for the material, which is at most eight `backdrop-filter` elements per viewport and *zero* on repeated list items. The hub spends one, briefly, when the user asks for it.

The tint is a deliberate whisper: `color-mix(in srgb, var(--site-bg) 20%, transparent)`. Twenty percent of the page background, mixed with transparency. Enough to lift contrast for the sectors and glyphs sitting on it; not enough to read as a coloured scrim. The distinction the CSS comment draws is exact — *"only a whisper of tint, so it reads as BLUR, not a coloured circle."*

17.3 The degradation ladder

The hub's decorative layers are gated, and it is worth being precise about which gate each one sits behind, because the gates are not the same and the difference reflects a judgement about what kind of thing each gate is measuring.

WHAT IS GATED, AND BEHIND WHAT

FEATURE	GATE	RATIONALE
Dial goo filter	<code>≥768px AND no-preference</code>	An SVG filter is per-frame GPU work. Both a device-class question and a preference question, so both gates.
Orb aura	<code>≥768px AND no-preference</code>	As above; the aura is <code>display: none</code> below the gate, so it is not merely unanimated but absent.

WHAT IS GATED, AND BEHIND WHAT (CONT.)

FEATURE	GATE	RATIONALE
Aura orbit animation	$\geq 768\text{px}$ AND no-preference	Inside the same query as the filter, so a static aura can never appear without its goo.
Logo bloom + stroke draw	no-preference	Motion only. A phone can afford a one-shot animation on mount; a user who asked for less motion cannot be given one.
Wedge labels	$\geq 480\text{px}$	Space only. Nothing to do with motion or cost.
Centre caption	$< 480\text{px}$	The exact complement of the labels.
Top bar backdrop blur	$\geq 768\text{px}$	Paint cost on phones. Below it the bar is fully opaque, because a partly-transparent bar with no blur just ghosts the feed underneath.
All hub transitions	no-preference	A single block near the end of <code>radial.css</code> lists the orb, blur, dial, wedge wrappers, hairlines, wedges, foot and aura and drops their transitions together.
Pointer metaball	no-preference AND no forced-colors AND no reduced-transparency AND $\geq 4\text{GB}$	The most heavily gated thing in the module, and the only one that consults device memory.

The pattern: **cost is gated by width, comfort is gated by preference, and space is gated by width for entirely different reasons.** Conflating those is how components end up with a phone that has no labels because someone reasoned about GPU budget when the actual constraint was arc length.

17.4 What reduced motion actually looks like

Worth spelling out, because "respects reduced motion" is often a claim rather than a description.

Under `prefers-reduced-motion: reduce`, tapping the orb produces: the overlay becoming visible immediately; the blur at full radius immediately; the dial at full opacity and scale immediately; all sixteen sectors at their resting angles, unrotated, fully opaque; the hairlines drawn; the foot pill in place; the orb at the centre. No aura. No goo — the sectors are crisp-edged, which is arguably a clearer dial than the filtered one. And closing sets the phase to `closing` and then to `closed` on a 0ms timer, so the overlay is gone within a frame.

The important property is that it is the same dial. Not a fallback list, not a different component, not a simplified layout. Every target is in the same place, at the same size, with the same label. A user who has asked for less motion has asked for less motion, not for a different information architecture.

THE ONE THING REDUCED MOTION CHANGES ABOUT THE LAYOUT

Nothing. Deliberately. The wedges' resting transform includes `scale(0.9)`, and it is the transition that is dropped, not the transform — so the open state's `scale(1) rotate(0deg)` still applies, instantly. If the reduced-motion block had dropped the transforms rather than the transitions, the dial would have rendered at 90% with its decks rotated, permanently. That is a real trap, and the distinction between "remove the animation" and "remove the animated property" is the whole of it.

PART FIVE

Material and colour

Why the dial is two stacked copies of the same geometry, how the metaball filter works, and the four-commit story of a menu whose icons were white on white.

18 Two layers, art and hit
19 The metaball filter bank

20 White on white
21 Tokens, themes and the accent

CHAPTER EIGHTEEN

Two layers, art and hit

The dial renders every sector twice: once as a decorative fill that the goo filter melts, and once as a real link that nothing filters. That duplication is the architectural answer to a hard constraint — the filter that makes the dial liquid destroys text.

18.1 The constraint

A goo filter works by blurring its input and then running the blurred alpha through a steep contrast ramp. The ramp is indiscriminate: it operates on alpha, and antialiased glyph edges *are* alpha. Run a 16-pixel-tall label through a blur and a $\times 19$ alpha multiply and what comes out is not slightly softer text; it is chewed. Strokes fatten unevenly, counters fill in, and hinting is destroyed.

The module's README states the rule first among three: **a goo group contains shapes only**. Anything readable rides in a sibling layer above the filter.

18.2 The two layers

THE SPLIT, AND WHAT THE FILTER TOUCHES — components/radial/radial.css:1054–1076

```

1054 /* The dial is TWO stacked layers of the same geometry:
1055     --art decorative sector fills; the only thing the goo filter touches.
1056     --hit the real links, unfiltered, each holding its own icon + label.
1057     Splitting them is what keeps the glyphs crisp (the filter's alpha ramp chews
1058     antialiasing) while still letting each glyph live INSIDE its sector, so it
1059     simply inherits that sector's `color` and is correct by construction in every
1060     theme — no blend modes, no cross-layer state tracking. */
1061 .radial-hub__wedges--art {
1062     pointer-events: none;
1063 }
1064 .radial-hub__wedges--hit {
1065     z-index: 1;
1066 }
1067 /* METABALL DIAL — blur-then-threshold melts the art sectors into ONE liquid
1068     disc: the thin dividers become gooey necks, the sharp centre point and rim
1069     corners round off, and each sector bulges as it grows in. Filter cost is real,
1070     so it is desktop + no-reduced-motion only; elsewhere the sectors render
1071     crisply. */
1072 @media (min-width: 768px) and (prefers-reduced-motion: no-preference) {
1073     .radial-hub__wedges--art {
1074         filter: url(#rmh-liquid);
1075     }
1076 }
```

The art layer is `aria-hidden`, `pointer-events: none`, and carries the bled polygon from Chapter 13. It contributes nothing but colour. The hit layer is the real navigation: sixteen anchors, each clipped to its exact sector, each containing its own icon and label.

Both are absolutely positioned over the same square with the same geometry, generated from the same `wedges` array in the same render. There is no synchronisation problem because there is nothing to synchronise: one array, two `map` calls.

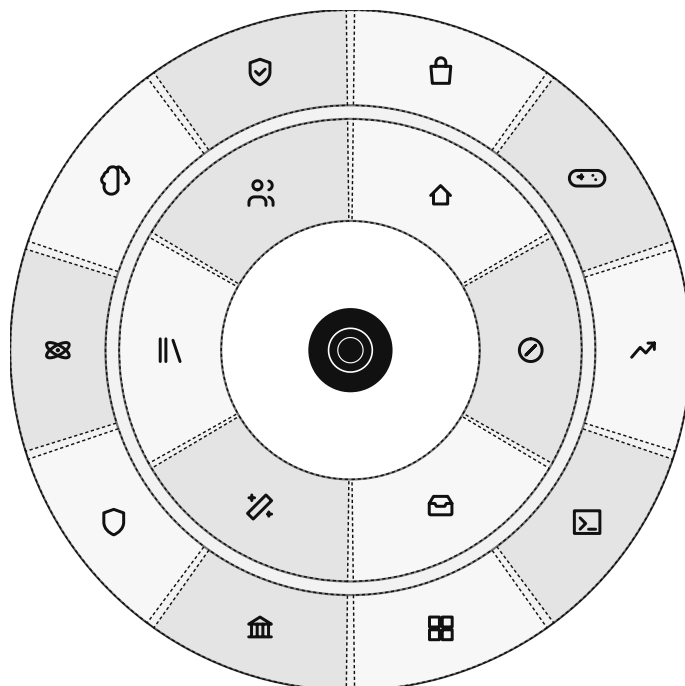


Figure — The hit map. Each dashed region is one anchor's `clip-path`, and a clip bounds hit-testing as well as paint, so the sectors tile the disc with no gap for a click to fall through.

18.3 The property this buys: colour correctness by construction

The split's obvious benefit is crisp glyphs. Its more valuable benefit is subtler, and it is the thing that finally solved the white-on-white problem in Chapter 20.

Because the glyph lives *inside* its sector's anchor, it inherits that anchor's `color`. And the anchor's colour is declared alongside its background:

COLOUR THAT TRACKS ITS OWN STATE — `components/radial/radial.css:1173–1199`

```
1173 .radial-hub__wedge {
1174   position: absolute;
1175   inset: 0;
1176   display: block;
1177   /* Re-enable hit-testing that the wrapper switched off. The inline clip-path
1178      carves this element down to one pie slice – and the clip bounds both paint
1179      AND hit-testing, so each wedge only catches clicks inside its own sector and
1180      the sectors tile the disc with no gaps for a click to slip through. */
1181   pointer-events: auto;
1182   /* Transparent at rest: the art layer beneath supplies the fill. What matters
1183      here is `color` – the glyph inside inherits it, so the icon + label are the
1184      theme's ink on the resting sector and flip to the accent's contrast ink the
1185      moment the sector fills below. Both tokens are contrast-checked against the
1186      surface they sit on, so this can never land white-on-white. */
1187   background: transparent;
1188   color: var(--site-text);
1189   text-decoration: none;
1190   transition:
1191     background-color 0.18s ease,
1192     color 0.18s ease;
1193 }
1194 .radial-hub__wedge:hover,
1195 .radial-hub__wedge:focus-visible,
1196 .radial-hub__wedge.is-active {
1197   background: var(--site-accent);
1198   color: var(--site-accent-fg);
1199 }
```

1199 }

Read the chain. At rest the anchor is transparent with `color: var(--site-text)` — the page's ink, on a sector whose fill comes from the art layer beneath, which is a near-surface tint. On hover, focus or active the anchor fills with `var(--site-accent)` and its colour becomes `var(--site-accent-fg)` — that fill's paired foreground. The glyph follows automatically, because it inherits.

Both of those pairings are contrast-guaranteed by the platform: `--site-text` against `--site-bg` is the theme's fundamental contrast pair, and `--site-accent-fg` against `--site-accent` is checked at runtime by `ensureReadableAccent()`, which refuses to install a custom accent whose pair fails. So the glyph cannot be illegible in any theme, on any accent, in any state — not because someone tested it, but because there is no code path that produces an unpaired combination.

The glyph is correct because of where it lives, not because of what it was told.

18.4 The active state, on the art layer

One asymmetry: the current page's sector is filled on the *art* layer, not only on the hit layer.

```
.radial-hub__wedge-art.is-active { background: var(--site-accent); }
```

Both layers carry `is-active`, and the reason the art layer needs it is the filter. If only the unfiltered hit layer painted the active fill, the active sector's colour would sit crisply on top of a goo-melted neighbourhood — a hard-edged wedge in a liquid disc. Filling the art layer puts the active colour *inside* the filter, so the current destination melts into the dial like every other sector and simply happens to be the accent colour.

The active glyph's own ink still comes from the hit layer's `color`, unfiltered. Accent fill, liquid, filtered; contrast ink, crisp, on top.

18.5 Why not one layer with the filter on a pseudo-element

The obvious economy is a single set of anchors, each with a `::before` carrying the filtered fill. It does not work, for a reason that is worth knowing because it recurs throughout this module: **filter creates a new stacking context and a containing block for position: fixed descendants.**

A filter on a pseudo-element of the anchor would be fine in isolation. But the goo effect requires the sectors to be filtered *as a group* — that is the entire mechanism; neighbouring shapes must be in the same filter region to fuse. Sixteen individually filtered pseudo-elements produce sixteen independently rounded sectors and no fusion at all. The filter has to be on a common ancestor, and a common ancestor of the anchors is also an ancestor of their glyphs.

Hence: two sibling groups, one filtered wholesale, one untouched. It is more DOM — thirty-two list items instead of sixteen — and it is the only arrangement that satisfies both constraints at once.

The same stacking-context hazard is why the orb's aura is a separate fixed layer rather than a pseudo-element on the orb: filtering the orb would filter the mark, and the mark is hairline line art that the alpha ramp would obliterate. And it is why the README's second rule reads *never filter an ancestor of fixed chrome*, with the note that nothing filters

```
.radial-shell.
```

The metaball filter bank

Three SVG filters in one hidden `<defs>`, mounted once in the shell. Blur, then a steep ramp on alpha, then the sharp original composited back. Fifty-eight lines of component for the effect that gives the whole design language its name.

THE WHOLE FILTER BANK — components/radial/LiquidGoo.tsx:29–58

```

29 export function LiquidGoo() {
30   return (
31     <svg className="radial-goo-defs" width="0" height="0" aria-hidden focusable="false">
32       <defs>
33         {G00.map(({ id, blur, ramp }) => (
34           <filter key={id} id={id} colorInterpolationFilters="sRGB">
35             <feGaussianBlur in="SourceGraphic" stdDeviation={blur} result="blur" />
36             /* Alpha contrast ramp: multiply alpha hard, then pull it back so the
37              blurred halo collapses into a crisp fused silhouette. */
38             <feColorMatrix
39               in="blur"
40               mode="matrix"
41               values={`1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 ${ramp} -${(ramp - 1) / 2}`}
42               result="goo"
43             />
44             /* Composite the sharp original back over the fused shape so interior
45              detail (borders, inner highlights) stays legible. */
46             <feBlend in="SourceGraphic" in2="goo" />
47           </filter>
48         ))}
49       </defs>
50     </svg>
51   );
52 }
53
54 const G00 = [
55   { id: 'rmh-liquid-sm', blur: 4, ramp: 15 },
56   { id: 'rmh-liquid', blur: 8, ramp: 19 },
57   { id: 'rmh-liquid-lg', blur: 16, ramp: 24 },
58 ] as const;

```

19.1 How it works

Three primitives, in order.

feGaussianBlur spreads each shape's alpha outward. Two shapes that were near each other now have overlapping alpha halos, and in the region between them the summed alpha is higher than either halo alone.

feColorMatrix multiplies alpha by a large factor and subtracts a constant, which is a steep linear ramp clamped at both ends. Alpha above the threshold snaps to 1; below it, to 0. The blurred halo collapses back to a hard edge — but the *position* of that edge is now determined by the summed alpha, so the region between two nearby shapes, where alpha was boosted by both, is inside the new silhouette. A smooth neck. Metaballs.

feBlend composites the sharp original back over the fused result, so interior detail — borders, inner highlights — survives rather than being replaced by a flat silhouette.

The matrix row that does the work is the last one, and it is worth reading as arithmetic: $0 \ 0 \ 0 \ \{\text{ramp}\}$

$-\{(\text{ramp}-1)/2\}$. Output alpha is $\text{ramp} \times \alpha - (\text{ramp}-1)/2$. Set $\alpha = 0.5$ and the output is $\text{ramp}/2 - \text{ramp}/2 + 0.5 = 0.5$. So the ramp is *pivoted on* $\alpha = 0.5$, for any ramp value: half-alpha stays half-alpha, and everything either side is pushed to the extremes. That is what makes the three filters differ in aggressiveness but not in where they cut.

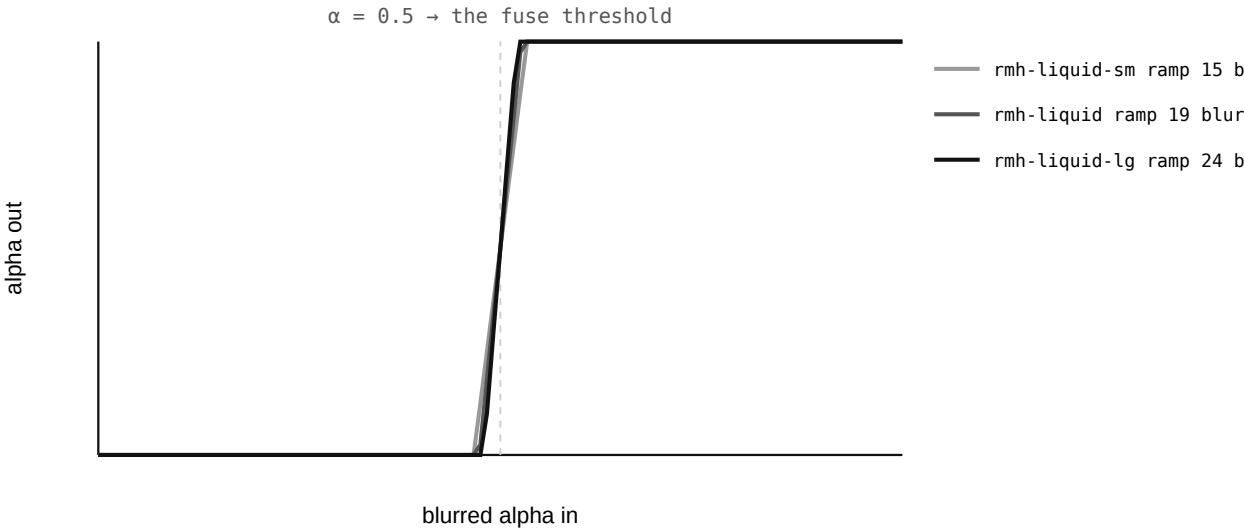


Figure 19.1 — The three alpha ramps. All three pivot on $\alpha = 0.5$; the difference is steepness, and therefore how far apart two shapes can be and still fuse. The blur radius sets the distance at which alpha is still non-negligible; the ramp sets how much of that halo survives.

19.2 The three radii

THE FILTER BANK

FILTER	BLUR	RAMP	A THRESHOLD USED BY
rmh-liquid-sm	4	15	0.467 Tight: rounds corners, fuses only touching shapes.
rmh-liquid	8	19	0.474 The default. The hub dial and the orb aura.
rmh-liquid-lg	16	24	0.479 Wide: distant blobs pull together. The backdrop blob field.

The dial uses the middle one. With `stdDeviation = 8`, alpha is still meaningful about 16–20 pixels from an edge, which comfortably spans the dial's ~3-pixel dividers — so adjacent sectors fuse across them and the dividers become gooey necks rather than channels. It also, as Chapter 13 documents, erodes a couple of pixels off every edge, which is what the art bleed pays back.

19.3 Where the goo is used, and where it is banned

THE LIQUID LAYER, ACROSS THE MODULE

SURFACE	WHAT FUSES
Hub dial	The clip-path sectors melt into one liquid disc; dividers become necks; the rim corners round off.
Orb aura	Three orbiting blobs stretch and neck in and out of the orb-sized core disc.
Backdrop blob field	Huge faint blobs drift, swell together and pull apart behind everything.
Pointer drop	The pointer's blobs fuse into one trailing drop — mouse and finger alike.
Loading mark	Orbiting blobs melt into a pulsing core, which is what the shared <code>Spinner</code> renders.

Never: any group containing text or an icon; any ancestor of fixed chrome; `.radial-shell` itself.

19.4 The trap that is not in the code

The README's fourth rule is the longest in the module and describes a technique the code deliberately does not use.

It is included here in full because it is the most transferable piece of knowledge the whole project produced.

There is a cheap, popular way to get a goo effect without an SVG filter: put an opaque plate behind some white blobs, apply `blur()` `contrast()` in CSS to threshold them, and add `mix-blend-mode: difference` to cancel the plate — black being difference's identity element. It works. It looks identical. And it is a landmine.

It stays invisible only while the compositor blends against the true page backdrop. The moment the layer gets its own render surface — which is a GPU-compositing decision, made by the browser, on criteria the author does not control — the plate stops cancelling and the entire box appears as a bright rectangle trailing the pointer. It does not reproduce under software rasterisation, so it is not a bug you can test your way out of.

The pointer metaball therefore uses an SVG alpha ramp — blur, then a steep curve on alpha alone — which emits the fused silhouette and transparency everywhere else. There is no box to reveal, on any path, and no blend mode at all, which also spares the per-frame backdrop readback that blending forces.

THE GENERAL FORM OF THE LESSON

An effect that depends on *what a layer is composited against* is an effect that depends on a decision the browser makes for performance reasons and can change between versions, between GPUs, and between one scroll position and the next. Alpha is a property of the layer. Blending is a property of the layer's relationship to its surroundings. Prefer alpha.

19.5 Cost, and the gate

An always-on SVG filter is continuous GPU work, so every decorative goo layer sits behind `@media (min-width: 768px)` and `(prefers-reduced-motion: no-preference)` — the same budget the ring backdrop already respects on phones. Below the gate the sectors render crisply, which is not a degraded dial so much as a different one: sharper, more legible, and arguably better on a small screen where the liquid character is mostly lost anyway.

The filter bank itself is always mounted — one hidden `<svg width="0" height="0">` holding three `<filter>` definitions. Definitions cost nothing until referenced. Mounting it unconditionally means CSS can reference the filters from inside a media query without the component needing to know which queries matched, which keeps the gating entirely in the stylesheet where the rest of the degradation lives.

CHAPTER TWENTY

White on white

Four commits, one recurring cause. The dial's icons were invisible, then the dial's sectors were invisible, then the logo mark was invisible. Every instance traces to the same idea: deriving contrast from the backdrop instead of from a token.

20.1 The idea that caused it

Sixteen sectors, any of which may be resting, hovered, focused or active, each with a different fill, on a dial that floats over an arbitrary blurred page in an arbitrary theme. What colour should a glyph be?

`mix-blend-mode: difference` is an extremely attractive answer. White, blended in difference mode, yields the exact inverse of whatever is beneath it. Maximum contrast, automatically, against any backdrop, in any theme, in any state, with no state tracking and no token plumbing. The glyph layer was made a single sibling above the filtered sectors and given difference blending, and it worked in the browser it was built in.

The failure is not about colour. It is about *what "beneath" means*. A blend mode composites against the backdrop **within the element's own stacking context** — not against whatever is visually behind it on screen. The glyph layer sat in a stacking context that did not contain the sectors it was meant to invert. So it inverted the context's own backdrop, which was nothing, which is to say it rendered white. On a white page, over a white-tinted dial. Commit `28e03b2`: *"radial menu icons were white-on-white."*

20.2 The four instances

THE WHITE-ON-WHITE FAMILY

COMMIT	SYMPTOM	CAUSE	FIX
28e03b2	Dial icons invisible	Glyph layer used difference blending against a stacking context that did not contain the sectors.	Move the glyph inside its sector's anchor; inherit <code>color</code> .
c631cb4	Whole dial invisible on a light page	Sector fill was <code>color-mix(--site-surface 60%, transparent)</code> — a tint of an already-translucent token.	Build on <code>--site-surface-opaque</code> , mix 6% toward <code>--site-text</code> .
454a762	Logo mark invisible on some accents	Same difference-blending idea, applied to the mark inside the orb.	Name <code>--site-accent-fg</code> , the orb fill's paired foreground.
89496ed	Site-wide: text and icons not tracking their surface	Components setting a background without naming the paired ink.	Zero-specificity <code>:where()</code> defaults in <code>globals.css</code> supply each pairing automatically.

The fourth is the interesting one, because it is the systemic fix. Rather than auditing every call site, `globals.css` now declares the pairings as `:where()` rules — zero specificity, so an element that sets only a background still gets the correct ink, and anything that states its own colour still wins. Since lucide icons paint with `currentColor`, one mechanism covers text and icons together.

20.3 What replaced the clever idea

The final architecture is the least clever available and the only one that is correct by construction. Stated as a rule:

INK COMES FROM THE SURFACE'S PAIRED TOKEN, AND LIVES INSIDE THE SURFACE

Do not compute contrast. Do not derive it from the backdrop. Name the foreground token that the theme author paired with the background you used, and put the text inside the element that carries that background so it inherits. Both halves matter: naming the token makes it theme-correct, and living inside the element makes it *state-correct* — hover, focus and active change the pair together, with no cross-layer coordination.

The comment left on the logo's colour declaration is the epitaph, and it names the exact mechanism so nobody re-derives the idea:

An earlier revision used `mix-blend-mode: difference` here; blending only composites against the backdrop inside the element's OWN stacking context, which is exactly how the hub glyphs ended up white-on-white.

COMPONENTS/RADIAL/RADIAL.CSS — .RADIAL-HUB__LOGO

20.4 Why this class of bug is so easy to ship

Three properties make it uniquely dangerous, and they are worth recognising in the abstract.

It looks right in development. The default theme is the one the author has open, and the bug's whole character is that it depends on the resolved palette. An invisible control in a theme nobody opened is a control that ships.

It is invisible to type checking and to lint. Nothing is malformed. The CSS is valid, the tokens exist, the blend mode is supported. There is no rule that could have caught it short of rendering the page and looking at it.

The failure mode is total. An unreadable label is bad; an *invisible* one is a control that does not appear to exist. On the mobile dial, where sixteen glyphs are the entire navigation, a white-on-white glyph layer means the site has no navigation at all.

Which is why the platform's definition of done for a UI commit ends with the requirement that the change was *actually looked at* in the default, light and high-contrast themes, at a phone width and a desktop width, and once with reduced motion on. Three themes by two widths is stated as the floor. This family of four commits is why.

CHAPTER TWENTY - ONE

Tokens, themes and the accent

The hub contains no colours. Every fill, edge, shadow and ink is an expression over custom properties that each theme redefines — including themes authored by users and bought with the platform's own currency.

21.1 What the hub consumes

EVERY TOKEN THE HUB READS

TOKEN	WHERE THE HUB USES IT
<code>--site-accent</code>	The orb's fill; the aura's blobs; a sector's hover / focus / active fill; the foot's close button and sign-in pill.
<code>--site-accent-fg</code>	The mark inside the orb; a filled sector's glyph and label; the close button's icon.
<code>--site-text</code>	A resting sector's glyph; the centre caption; the foot's controls; the orb's focus ring; 6% of the sector fill and 14% of the bed; 55% of the hairline colour.
<code>--site-surface-opaque</code>	The base of the sector fill and of the divider bed — the material's solid companion, for the cases translucency cannot serve.
<code>--site-surface-hover</code>	The foot pill's controls on hover.
<code>--site-bg</code>	The blur's 20% tint; the foot pill's 62% tint; the docked orb's legibility scrim gradient.
<code>--site-border</code>	The dial's 1px outer ring; the foot pill's edge.
<code>--site-border-width</code>	The foot pill's border width — 1px normally, 2px in high contrast.
<code>--site-shadow</code>	The foot pill's elevation.
<code>--site-glass-blur-overlay</code>	The foot pill's blur radius (L4 tier).
<code>--site-glass-saturate</code>	The foot pill's saturation boost.
<code>--glass-user-tint / --glass-blur-factor</code>	The user's Glass clarity setting, five stops, scaling the foot pill's tint and blur.
<code>--site-hub-orb-size / -inset</code>	The orb's footprint. Declared in <code>globals.css</code> , not here, so the floating stack's reserve cannot drift from the orb's geometry.
<code>--ease-glass</code>	Every position transition, with a literal fallback.
<code>--z-floating / --z-hub-*</code>	The layer stack.

Sixteen tokens, no hex values. The single number that looks like a colour anywhere in the hub's CSS is the orb's `box-shadow`, which uses `rgba(0, 0, 0, 0.5)` — a shadow, which is a darkness rather than a hue, and which the platform treats as a legitimate exception.

21.2 The `color-mix` discipline

Three of the hub's most important values are mixes rather than tokens, and each is doing something a token could not.

`color-mix(--site-surface-opaque 94%, --site-text 6%)` — the sector

"The solid surface, nudged toward the ink." Distinguishable from any page background in any theme, because every theme's ink contrasts with its own background by definition.

`color-mix(--site-surface-opaque 86%, --site-text 14%)` — the bed

The same expression at more than double the strength. The 8-point difference between them is the divider: the sectors sit on the bed and the bed shows through the gaps, so the dividers are a relationship between two mixes rather than a drawn line with a colour of its own.

`color-mix(--site-text 55%, transparent)` — the hairline

Firm enough to read as structure, soft enough not to become the loudest thing on the disc. Expressed as a transparency of the ink rather than as a grey, so it darkens on a dark theme and lightens on a light one along with everything else.

The pattern across all three: **express a relationship, not a value**. A designer picking hex values for these would need one set per theme, and would be wrong the first time a user authored a theme nobody anticipated.

21.3 The accent, and where it lands

The accent is the loudest decision in the hub: it is the orb's entire fill, and it is what a sector becomes when you point at it. On the default theme the accent is *ink* — a near-black — so the orb is a black disc with a white filigree mark, and hovering a sector fills it black with white glyphs. On the high-contrast theme the accent is yellow. On a user's purchased theme it is whatever they authored.

Fourteen curated accent presets ship, applied as inline overrides of the `--site-accent*` tokens on `<html>`, keeping everything else from the theme. Custom accents are allowed too — and this is where `ensureReadableAccent()` earns its place: it contrast-checks the accent against its foreground at runtime and refuses to install an unreadable pair. Without it, a user could give themselves a hub whose glyphs vanish, and the geometry would be flawless and the menu unusable.

A SUBTLETY THE AUDIT FOUND IN A DIFFERENT COMPONENT

On the default theme the accent *is* the ink, which means "accent" and "theme default" are visually identical — finding AUD-315 records that the settings page's accent swatches made "Ink" indistinguishable from "Theme default". The hub inherits the same coincidence harmlessly, but it is worth knowing when reading its CSS: on the shipped default, `--site-accent` and `--site-text` resolve to nearly the same colour, so several declarations that look like they must differ do not.

21.4 User themes

The platform lets members author and publish themes, and other users buy them with coins. A `UserTheme` is a set of colours plus a few scalar knobs, and `themeCssVars()` derives the full `--site-*` and `--site-glass-*` contract from them.

This is the strongest argument for the token discipline, and it is not hypothetical. The hub has to look deliberate in themes that did not exist when it was written, authored by people who have never read this document, and it does — because a theme cannot restyle the hub, it can only supply the contract the hub reads. Token maps from an older schema version are upcast on read rather than rejected, so a theme bought a year ago still resolves a complete contract today.

The corollary is a hard constraint on future work on this component: **any new colour must be an expression over existing tokens, or a new token added to the contract and documented**. A hex value in this file would be a hub that is subtly wrong in a theme somebody paid for.

PART SIX

Input and access

Sixteen non-rectangular targets, a keyboard, a screen reader, a thumb, a home indicator, and four other floating controls that want the same corner.

22 Hit testing sixteen wedges
23 Keyboard and screen reader
24 The caption in the hole

25 Blocking scroll without a scroll lock
26 Safe areas and the floating stack

Hit testing sixteen wedges

There is no hit-testing code. Sixteen full-size overlapping squares, each clipped to a pie sector, and a browser that already knows a clip bounds pointer events. The only real work is switching the wrappers off so they do not blanket the disc.

22.1 The mechanism

Each wedge is a list item filling the whole dial square, containing an anchor that also fills the whole square, clipped to its sector. Sixteen of those, stacked. Without intervention the topmost one in DOM order catches every click on the entire disc.

WHY THE WRAPPERS ARE CLICK-TRANSPARENT — components/radial/radial.css:1077–1086

```
1077 .radial-hub__wedge-wrap {
1078   position: absolute;
1079   inset: 0;
1080   /* Each wrapper fills the whole dial square (only its child <a> is clipped to a
1081      slice), so a wrapper with hit-testing on would blanket the disc and steal
1082      every click for whichever wedge sits last in the DOM. Make the wrappers
1083      click-transparent; the clipped anchors below re-enable it, so each slice
1084      only catches clicks inside its own pie sector. */
1085   pointer-events: none;
1086 }
```

The wrappers are `pointer-events: none`; the clipped anchors set `pointer-events: auto`. So a pointer event tests only against the clipped regions, and because the clip is the exact sector polygon, each anchor catches only what is inside its own sector. The sectors tile the annulus with 1.4° gaps — which means a press exactly in a divider hits nothing, by design, rather than being resolved arbitrarily.

The glyph wrapper inside each anchor is also `pointer-events: none`, so clicks fall through it to the sector beneath. That matters because the glyph box is an axis-aligned rectangle centred on the sector's centroid, and near the dividers its corners extend into neighbouring territory. Left hit-testable, a label's corner would steal a neighbour's click in exactly the region where the user is aiming at a visible boundary.

22.2 Target sizes, honestly

The platform's touch-target floor is 44 pixels, published as `--site-touch-target`. A wedge is not square, so the question is what "44 pixels" means for an annulus sector.

WEDGE TARGET DIMENSIONS, BY VIEWPORT

VIEWPORT	DECK	RADIAL DEPTH	ARC AT INNER EDGE	ARC AT MID	VERDICT
320×568	inner (6)	41.3px	54.7px	76.3px	depth 3px under floor
320×568	outer (10)	38.5px	62.2px	74.3px	depth 5px under floor
390×844	inner (6)	50.3px	66.6px	92.9px	over floor
390×844	outer (10)	46.9px	75.8px	90.5px	over floor
768×1024	inner (6)	69.0px	91.4px	127.5px	generous
768×1024	outer (10)	64.4px	104.0px	124.2px	generous

The honest reading: at 320 pixels the wedges are a few pixels short of the floor in the radial direction, and

comfortably over it in the angular direction. A 41×55 target is larger in area than a 44×44 one, and the shortfall is in the axis where the neighbour is a *different ring* separated by a drawn 9-pixel gap rather than an adjacent target. A miss lands in the gap, not on the wrong destination.

That is a defensible position and it is not the same as meeting the guideline. It is recorded in Chapter 30 among the open items, because the cleanest fix — lifting the dial's 86vw to something larger below 360px — has a knock-on effect on the foot pill's clearance that nobody has worked through.

22.3 The pointer metaball, and AUD-503

The hub shares the module — and the filter bank — with the site's pointer metaball: a gooey drop that follows the mouse on desktop and the finger on touch, replacing the native cursor while it is driving one.

Its contract is worth reading as a piece of engineering discipline in its own right. Every spring converges at a fixed rate *per second* — $1 - e^{(-\lambda \cdot dt)}$ — rather than a fixed fraction per frame, so a 144Hz display gets the same curve as a 60Hz one instead of snapping to the pointer, and a dropped frame produces no jump. `dt` is clamped so returning to a backgrounded tab eases rather than teleports. The filter region is a fixed box with every blob offset clamped inside it, so the tail can stretch but can never enlarge or escape the region being blurred. `will-change` is toggled by the loop, so a parked pointer holds no compositor layer.

And it had the most quotable defect in the audit. AUD-503: the drop swelled ×1.85 over interactive elements — *largest precisely when covering a control's label*. "Play now" read as "▷ ●now". An effect designed to say *this is interactive* was obscuring the words that said what it was.

The repair caps the swollen diameter at about 22 pixels and thins the fill as it swells, so text reads through. Which is the same lesson as Chapter 18's two-layer split, arriving from a different direction: **a decorative liquid effect must never occupy the same pixels as something you have to read**. The dial solved it with a layer split; the cursor solved it with an opacity ramp; both are the same rule.

LEGIBILITY BUILT INTO THE SHAPE

Because the drop uses no blend mode, there is no inversion to rely on for contrast. So it is filled with `--site-text` and carries a halo in `--site-bg` — two `drop-shadow`s chained after the goo, so the rim traces the fused outline rather than each blob. Those tokens are contrast-paired by definition, so it reads in every theme. On the default theme, where the accent is the ink, hovering the compose button turns the whole mark into its own halo and it reads as a ring — a coincidence the code anticipates in a comment.

22.4 What the hub does not do with the pointer

No press-and-drag selection. No radial gesture recognition. No angular velocity. The dial is sixteen links and a pointer, and hovering one fills it and names it, and pressing it navigates.

This is a non-goal from Chapter 4, and the reason bears repeating here where the input model is under discussion: gesture-driven radial menus work in games because the player has hours of muscle memory and a fixed item set. A website's navigation is used by someone who arrived thirty seconds ago from a search engine. Any interaction that must be learned before it can be used is, for that person, a wall.

CHAPTER TWENTY-THREE

Keyboard and screen reader

A radial menu is a visual metaphor. To a screen reader it is a list of sixteen links with a name, and to a keyboard it is sixteen tab stops. Both of those are fine, and the parts that are not fine are worth naming precisely.

23.1 The semantics, as shipped

THE ACCESSIBILITY SURFACE

ELEMENT	SEMANTICS
The orb	<code><button></code> , <code>aria-haspopup="menu"</code> , <code>aria-expanded={phase !== 'closed'}</code> , translated <code>aria-label</code> .
The overlay	<code>role="presentation"</code> , <code>aria-hidden</code> while the menu is not visible.
The scrim	A real <code><button></code> with a translated "Close" label.
The dial	<code>role="menu"</code> with a translated <code>aria-label</code> — "Browse RMH Studios".
Each wedge	<code><Link></code> or <code><a></code> with <code>role="menuitem"</code> , a translated <code>aria-label</code> , and <code>aria-current="page"</code> on the active destination.
Wedge wrappers	<code>role="none"</code> , so the <code></code> does not interrupt the menu's required parent-child relationship.
Art layer	<code>aria-hidden</code> in its entirety.
Hairlines, caption, aura, scrim gradient	<code>aria-hidden</code> .
Icons	<code>aria-hidden</code> — every one, including the foot pill's.
Foot controls	Named: identity link, settings, sign-out, close. Icon-only ones carry translated <code>aria-label</code> s.
Every control while closed	<code>tabIndex=-1</code> , so the closed hub is absent from the tab order entirely.

The last row is the one most often got wrong in overlay components, and getting it right matters more than it sounds. A hidden menu whose links remain focusable produces a tab order with sixteen invisible stops in it — a keyboard user tabs from the page's last control into nothing, sixteen times. The hub computes `const tab = menuVisible ? 0 : -1` once and applies it to every interactive element in the overlay.

23.2 What is missing

Three gaps, stated plainly rather than buried.

No arrow-key navigation. The ARIA authoring practices for a `menu` expect arrow keys to move between `menuitems`, with Tab moving out of the menu entirely. The hub gives every wedge a real tab stop and no arrow handling, so it behaves like a list of links that happens to be labelled as a menu. For a radial layout the mapping is not obvious — should Right go clockwise, or to the next item, and what do Up and Down mean on a ring? — but "not obvious" is not the same as "not required".

No focus trap. Tab past the last wedge and focus continues to the foot pill, and past that into the document behind the overlay — which is `aria-hidden` and covered, so focus lands somewhere invisible.

No focus restore. Closing the dial does not return focus to the orb. A keyboard user who opens the menu and presses Escape is left with focus wherever it was inside a now-unmounted overlay.

All three are in Chapter 30. None of them prevents the menu from being used with a keyboard; each of them makes it worse than it should be.

23.3 Focus visibility, and the one place it is suppressed

A wedge's `:focus-visible` deliberately removes the outline:

```
.radial-hub__wedge:focus-visible { outline: none; }
```

This is normally a red flag. It is defensible here because the same selector list — `:hover`, `:focus-visible`, `.is-active` — fills the sector with the accent colour and flips its glyph to the accent's contrast ink. The focused wedge is not subtly outlined; it is the one wedge on the dial that is a solid block of accent. An outline on top of that would have to trace the sector's clip path, which it cannot — an `outline` follows the border box, and the border box here is the whole dial square, so the "focus ring" would be a rectangle around the entire menu.

The orb, whose border box is its own disc, keeps a real ring: `outline: 3px solid var(--site-text); outline-offset: 3px`.

THE TEST FOR SUPPRESSING A FOCUS RING

Removing `outline` is acceptable only when the same state produces an indication that is at least as strong and does not depend on colour perception alone. A full-sector fill with an inverted glyph passes: it changes the shape's value, not just its hue. A border brighten on a transparent fill does not — which the audit also caught, as AUD-118, on the outline button variant.

23.4 Translation

Every string in the hub goes through `t()` with an English `defaultValue`: the orb's label, the dial's label, the scrim's and close button's "Close", "Settings", "Sign out", "Sign in", and each of the sixteen destination names via its catalog `tKey`. Sixteen locales, two of them right-to-left.

The audit surfaced a subtle hazard in this arrangement. The orb's shipped English catalog entry is "Open menu"; the code's `defaultValue` is "Open navigation". Both are fine strings; they are different strings, and the live accessible name is whichever the catalog supplies. The audit's own harness selected the orb by accessible name and silently captured nothing.

The general form: a `defaultValue` is a fallback, not documentation. Anything that depends on the accessible name — a test, a screenshot harness, an analytics selector — must read the catalog, not the component. And a drift between the two is invisible in the UI, which is what makes it durable.

CHAPTER TWENTY-FOUR

The caption in the hole

Below 480 pixels the dial is sixteen unlabelled glyphs, which the audit correctly called recall rather than recognition. There is no room for sixteen labels on a 320-pixel dial. There is room for one, in the middle.

24.1 The finding

AUD-504(a), rated as part of a P1 group: *the mobile dial is entirely icon-only — sixteen unlabelled glyphs as the sole navigation*. Desktop wedges showed labels; phones did not. Chapter 12 established that this was not an oversight but arithmetic: the outer band on a 275-pixel dial gives each destination a box of roughly 37×38 pixels, and no typography puts "Adaptive Intelligence" in it.

So the fix could not be "add the labels back". It had to find room that was not there.

24.2 The hole is the room

The dial's centre hole exists to clear the orb — radius 19, which on a 275-pixel dial is a 104-pixel-wide circle. The orb itself is 56 pixels. That leaves an annulus of empty space inside the hole, and empty space in the middle of a dial is the one region every sector is equidistant from.

THE CENTRE CAPTION — components/radial/radial.css:1252–1273

```

1252 /* Centre caption – the label for the wedge under the pointer/focus, sitting in
1253    the dial's hole. It exists ONLY where the per-wedge labels don't: below
1254    480px the dial is sixteen unlabelled glyphs, which is recall, not
1255    recognition, for the site's entire navigation. The hole is the one place on
1256    a 320px dial with room for a word. */
1257 .radial-hub__caption {
1258   position: absolute;
1259   left: 50%;
1260   top: 50%;
1261   transform: translate(-50%, -50%);
1262   z-index: 2;
1263   max-width: 8rem;
1264   padding-top: 2.6rem;
1265   text-align: center;
1266   font-size: 0.75rem;
1267   font-weight: 700;
1268   line-height: 1.2;
1269   letter-spacing: 0.01em;
1270   color: var(--site-text);
1271   text-wrap: balance;
1272   pointer-events: none;
1273 }
```

The component tracks which destination the pointer or focus is on, in a single piece of state:

THE HOVERED-LABEL STATE, AND WHY — components/radial/RadialHub.tsx:113–121

```

113 const [phase, setPhase] = useState<Phase>('closed');
114 /**
115  * Name of the wedge the pointer/focus is on. Below 480px the dial is
116  * icon-only – sixteen unlabelled glyphs as the site's whole navigation – and
```

```

117  * there is genuinely no room for sixteen labels on a 320px dial. So the label
118  * moves to the one place that has room: the hole in the middle. CSS shows it
119  * only where the per-wedge labels are hidden.
120  */
121  const [hoveredLabel, setHoveredLabel] = useState<string | null>(null);

```

Four handlers per wedge set and clear it — `onPointerEnter`, `onPointerLeave`, `onFocus`, `onBlur` — so it responds to both the pointer and the keyboard. The clear handlers are guarded: `setHoveredLabel((cur) => (cur === label ? null : cur))`, so a leave event arriving after the next wedge's enter event cannot blank a label that has already been replaced. That ordering does happen, on touch especially, and without the guard the caption flickers.

An effect keyed on the phase clears the caption when the menu closes, so reopening never briefly shows the last thing you touched.

24.3 Exact complementarity

The caption and the per-wedge labels are exact complements, and both halves are declared in the same media query:

```

.radial-hub__wedge-label { display: none; }          /* base: phones */
.radial-hub__caption     { /* visible */ }

@media (min-width: 480px) {
  .radial-hub__wedge-label { display: -webkit-box; }
  .radial-hub__caption     { display: none; }
}

```

Exactly one of them is showing a name at any width. That is not merely tidy — it is the thing that makes the mechanism honest. A caption that also appeared on desktop would duplicate the label under the pointer, and the reader would have two sources for the same fact. A caption that appeared on neither would be the original bug.

The caption's own geometry has one deliberate oddity: `padding-top: 2.6rem`. It is centred on the dial's midpoint, and the orb is also there — so the padding pushes the text below the orb rather than behind it. It is a single-property solution to what would otherwise be an offset calculation against the orb's size token.

24.4 What it does not fix

The caption tells you what you are touching. It does not tell you the menu exists.

AUD-118 states the remaining problem in one clause: *sub-1120px navigation is a single unlabelled orb with no first-run hint*. Below the desktop rail's breakpoint, the site's entire navigation is one circular mark with a logo in it. A user who does not tap it never discovers that the site has sixteen destinations.

The audit's own suggestion — a transient "Menu" caption on first run — has not shipped. It is the single largest open item on this control, and Chapter 30 is direct about why it is hard to do well: a first-run hint that is too quiet is decoration, one that is too loud is an interstitial, and the platform already has enough first-run overlays that the audit had to suppress several to capture anything at all.

**The caption solved recognition for people already inside the menu.
Nothing yet solves discovery for people outside it.**

CHAPTER TWENTY-FIVE

Blocking scroll without a scroll lock

The most valuable thing the deleted push-drawer left behind. Both standard techniques for locking background scroll clip the document to the visual viewport on iOS, which leaves a band of bare page background under Safari's floating bottom bar. The hub uses neither.

25.1 The two techniques, and what they do

The conventional options for stopping the page scrolling behind an overlay are `body { overflow: hidden }` and the `position: fixed` body technique (fix the body, offset it by the current scroll position, restore on close).

Both work. Both have the same side effect on mobile Safari, and it is not obvious from a desktop browser: they clip the document to the *visual* viewport. Safari's layout viewport extends behind its floating bottom bar, and the page normally scrolls *under* that bar — that is why content is visible through it. Clip the document to the visual viewport and the content that would have been under the bar is no longer rendered, leaving a strip of bare page background there.

On a dark page under a translucent bar that is a grey band across the bottom of the screen, appearing the instant a menu opens. It is not a rendering glitch that can be worked around from inside the overlay; it is a consequence of what the technique does to the document.

25.2 What the hub does instead

TOUCH-ACTION, NOT A SCROLL LOCK — components/radial/radial.css:958–971

```

958 /* Background scroll is blocked by the OVERLAY, not by a CSS scroll-lock on the
959    document. Both overflow:hidden and the position:fixed body technique clip the
960    document to the visual viewport, which on iOS clips away the content that
961    normally scrolls under Safari's floating bottom bar and leaves the bare page
962    background showing there as a stray coloured band – established on-device for
963    the mobile push-drawer this hub replaced, which blocked background scroll from
964    its scrim for exactly this reason. touch-action:none makes the covered page
965    unpannable while the menu is up, so the document keeps flowing behind the
966    browser chrome exactly as it does when the menu is closed. Taps are
967    unaffected. */
968 .radial-hub[data-phase='open'] .radial-hub__overlay,
969 .radial-hub[data-phase='closing'] .radial-hub__overlay {
970   touch-action: none;
971 }
```

Two mechanisms, one per input class.

Touch: `touch-action: none` on the overlay. The overlay covers the viewport, so a touch that would have panned the page starts on an element that declines to be panned. The document is never modified — it keeps its scroll position, its full height and its normal relationship to the browser's chrome. Content still flows under Safari's bottom bar exactly as it does when the menu is closed.

Pointer: a `wheel` listener on the overlay with `{ passive: false }` calling `preventDefault()`, because `touch-action` governs touch and pen, not a mouse wheel.

Taps are unaffected — `touch-action` suppresses panning gestures, not clicks — so the scrim still dismisses on tap

and the wedges still navigate.

25.3 The provenance, and why the comment is so long

This was established on a real device, for the mobile push-drawer, which blocked background scroll from its scrim for exactly this reason. That drawer no longer exists — commit 0969119 deleted it as unreachable code — so the finding survives only as comments, and it is deliberately duplicated in two places: the effect in `RadialHub.tsx` that installs the wheel guard, and the CSS rule that sets `touch-action`.

Duplicating a comment is normally a smell. Here it is insurance. Somebody optimising the stylesheet would see a lone `touch-action: none` and reasonably wonder why the component does not just lock the body like everything else does; somebody reading the component would see a wheel guard and wonder the same. Both need the answer, and the answer is a device-specific finding that cannot be re-derived by reading either file.

THE TRANSFERABLE RULE

An on-device finding that led you to reject the obvious implementation must be written down *at the site of the unobvious one*. Otherwise the next person will "simplify" it back, ship the regression, and rediscover the finding on a device they may not have.

25.4 What is deliberately not blocked

The page keeps its scroll position, which means the feed is exactly where it was when the menu closes. No restoration logic, no scroll-position bookkeeping, no jump — because nothing was ever moved. This is a real advantage of the technique beyond the iOS bug: the `position: fixed` body approach has to save and restore a scroll offset, and every implementation of that has an edge case around resize, orientation change, or a page that grew while the overlay was open.

Scrolling *inside* the overlay is also not blocked, because there is nothing inside it to scroll. The dial is sized to fit the viewport by construction — `min(86vw, 70vh, 460px)` — and the foot pill is a single row. A radial menu that needed to scroll would be a radial menu with too many destinations, and the answer to that is another deck, not a scrollbar.

CHAPTER TWENTY-SIX

Safe areas and the floating stack

The single largest source of defects in the whole project, and the least glamorous. A fixed control at the bottom centre of every page has to coexist with a home indicator, a cookie banner, a mini player, a back-to-top button, a compose button, and the content scrolling underneath it.

26.1 The safe area

The site sets `viewport-fit=cover`, so the layout extends into the display's rounded corners and behind the home indicator. The body carries safe-area padding — and **that padding does not apply to `position: fixed` elements**, which position against the viewport rather than against the body's content box.

So every bottom-anchored fixed element in the module adds the inset itself:

```
--orb-bottom: calc(env(safe-area-inset-bottom, 0px) + var(--site-hub-orb-inset));
```

The orb, the aura, the foot pill and the feed's compose button all do this independently. Skipping it puts the control inside the home-indicator gesture strip, where taps are unreliable because the system claims them — and, visibly, makes it hang lower than the compose button beside it, which was always safe-area-aware. That mismatch is how the bug was noticed.

26.2 The floating-bottom stack

The orb is not the only thing that wants the bottom of the screen. The platform defines an explicit stack, and the orb is its bottom member:

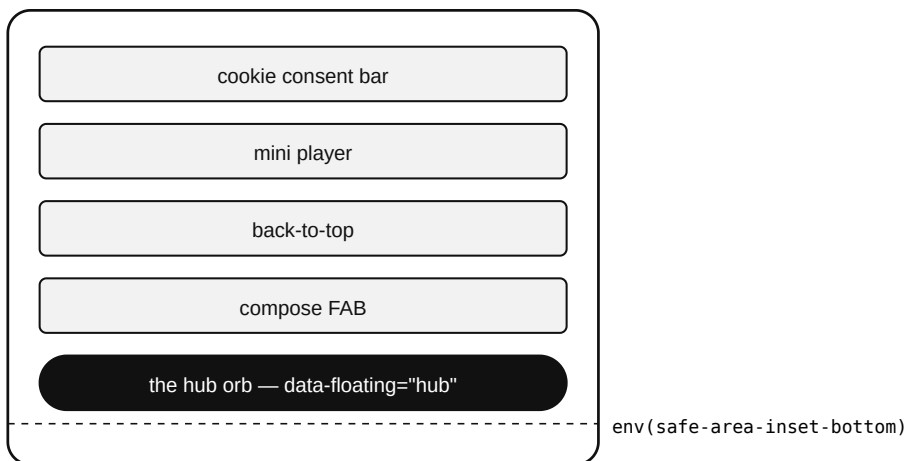


Figure 26.1 — The floating-bottom stack. The orb carries `data-floating="hub"` and everything else lifts above it. Its footprint comes from `--site-hub-orb-size` and `--site-hub-orb-inset`, declared in `globals.css` precisely so that the stack's reserve and the orb's geometry cannot drift apart.

The coordination is done with `:has()` in `globals.css`: the presence of `[data-floating="hub"]` lifts the cookie bar, the mini player and the back-to-top control clear of it. The compose button shares the orb's row but is shorter, so it sits beside rather than above.

And the bug — AUD-102, the most-cited instance of AUD-003 — was that this entire mechanism was wrapped in `@media (max-width: 767.98px)`. It ran on phones and never on anything wider. So at every viewport from 768 pixels up, on every page of the site, the orb sat on top of the cookie banner's text and its "Learn more" link. The fix was to extend the coordination to all widths.

WHY A MOBILE-ONLY MEDIA QUERY WAS THE WRONG CONTAINER

The reasoning that put it there is easy to reconstruct: floating controls pile up on phones, so coordinate them on phones. But the orb is fixed at *every* width — it is not a mobile control — and the cookie banner is full-width at every width. The collision was never about screen size. Scoping a coordination mechanism to the viewport where the problem was first noticed, rather than to the elements that actually participate, is a recurring shape of bug.

26.3 Clearance for content

Not colliding with other chrome is half the problem. The other half is that page content scrolls underneath the orb, and a 56-pixel disc floating over a feed covers text.

Two mechanisms, and they are different in kind.

Reserve at the end. `--site-floating-reserve` is derived from the orb's own size and inset tokens — not a guessed clamp — and every column that ends near the bottom of the page pads by it. Any full-height desktop panel stops `--rad-corner-reserve` short of the viewport bottom for the same reason. That solves the *end* of a scroll: the last feed card is not under the orb.

The legibility scrim. Nothing can stop text travelling through the orb's band mid-scroll. So the ground fades in behind it:

THE LEGIBILITY SCRIM UNDER THE DOCKED ORB — `components/radial/radial.css:756–783`

```

756 /* Legibility scrim under the docked orb. The orb is fixed bottom-centre over
757    scrolling content, so feed text, counters and card meta pass directly beneath
758    it. Columns reserve --site-floating-reserve at their END, but nothing can stop
759    text from travelling through the orb's band mid-scroll – so fade the page
760    ground in behind it, radially, and the type dissolves into the surface instead
761    of colliding with the disc. Decorative, never hit-testable, and it retracts
762    the moment the dial opens (the overlay owns the screen then). */
763 .radial-hub_scrim {
764     position: fixed;
765     left: 50%;
766     bottom: 0;
767     z-index: calc(var(--z-floating) - 2);
768     width: min(24rem, 92vw);
769     /* The orb's own band plus its inset, plus room for the fade to resolve. */
770     height: calc(
771         env(safe-area-inset-bottom, 0px) + var(--site-hub-orb-size) + var(--site-hub-orb-inset) *
2
772     );
773     transform: translateX(-50%);
774     pointer-events: none;
775     opacity: 1;
776     transition: opacity 0.28s ease;
777     background: radial-gradient(
778         60% 78% at 50% 100%,
779         var(--site-bg) 0%,
780         color-mix(in srgb, var(--site-bg) 72%, transparent) 42%,

```

```

781     transparent 78%
782   );
783 }

```

A radial gradient of the page background, 24rem wide, fading out by 78% of its radius, never hit-testable, and retracting the instant the dial opens because the overlay owns the screen then. Text passing beneath the orb dissolves into the page rather than colliding with a disc. It is the third of the audit's three prescribed fixes for AUD-003, and it is the one that turns an unavoidable overlap into a deliberate one.

26.4 The flexbox trap in the foot pill

AUD-504(b): the account pill's avatar disc painted over the settings gear beside it, at both viewports. The cause is one of the most common flexbox mistakes there is, and the fix is one line.

A flex item's default `min-width` is `auto`, which resolves to its *content* size. The identity pill contains an avatar and a display name; with a long name, its content size exceeded the space available, and because `min-width: auto` prevented it from shrinking below that, it kept its full width and overflowed onto its neighbour.

MIN-WIDTH: 0, AND WHAT ELLIPSES — components/radial/radial.css:1332–1342

```

1332   font-weight: 600;
1333   max-width: 40vw;
1334   /* The pill is the only flexible item in the foot row, and it has to be able
1335      to shrink: a flex item's default `min-width: auto` is its CONTENT, so a
1336      long display name kept the pill at full width and pushed its avatar disc
1337      on top of the settings gear beside it. `min-width: 0` lets it give way,
1338      and the name (not the avatar) is what ellipses. */
1339   flex: 0 1 auto;
1340   min-width: 0;
1341   overflow: hidden;
1342 }

```

The fix is `min-width: 0` on the pill, `flex: none` on the avatar so it is not what shrinks, and `overflow: hidden` plus `text-overflow: ellipsis` on the name so the *name* is what gives way. The icon buttons beside it also carry `flex: none`, so they can never be squeezed by the pill.

Worth stating as a rule, because it generalises well beyond this component: **in any flex row that mixes a fixed element with a text element, the text element needs `min-width: 0` and the fixed elements need `flex: none`.** Without the first, nothing shrinks; without the second, the wrong thing does.

26.5 The last-resort guarantee

One more line, at the level of the shell:

```
.radial-shell { overflow-x: clip; }
```

Not `hidden`. `clip`. The distinction matters: `overflow: hidden` makes an element a scroll container, and a scroll container breaks `position: sticky` for its descendants — which would silently unstick the top bar and both rails. `overflow-x: clip` clips without creating a scroll container.

It is there as a guarantee of last resort: whatever else goes wrong inside the shell, nothing can scroll the document sideways. The module's overlap-safety section is explicit that this is a backstop rather than a strategy — the real work is grid tracks with `minmax(0, ...)`, `min-width: 0` on every flex child, line clamps and `overflow-wrap`:

anywhere on every slot that takes user content. Where overlap can be made structurally impossible, it is. This line catches what is left.

PART SEVEN

The defect ledger

Twenty commits in five days, and the record of what each one broke. This is the part worth reading if you have to maintain the thing: every entry is a case where the design was right and the implementation had an undeclared dependency on something else.

27 A chronology

28 Six patterns

CHAPTER TWENTY - SEVEN

A chronology

Twenty commits between 24 and 28 July 2026. Eight built the thing, five fixed contrast and material failures, three fixed mobile chrome, two rebuilt the motion, and two are the audit remediation passes.

THE RADIAL HUB, COMMIT BY COMMIT

COMMIT	DATE	SUBJECT, AND WHAT IT DID TO THE HUB
e5003b1	07-24	radial front-end rewrite — monochrome RMH radial UI. The shell, the wheel feed, the first hub. <code>SiteShell</code> becomes a delegate to <code>RadialShell</code> ; the old shell nav and <code>FeedLayout</code> are removed.
ffa3282	07-24	Centre the feed prism, cut mobile load cost, add scroll haptics.
bedf361	07-24	wedge nav, metaball cursor, auto-contrast text, mobile polish. The dial becomes wedges. The first "auto-contrast" text — the idea that becomes Chapter 20's four-commit family.
f4a510f	07-24	RMH logo mark in the hub orb, with a stroke-draw bloom. The mark stops being decoration.
a7a7779	07-24	Native-scroll feed: variable heights, document scroll.
d5bda85	07-24	Radialise content pages — flatten page headings, unpin sticky headers site-wide, so every route flows like the feed.
426d1d1	07-24	avant-garde glass hub + document-scroll feed + inline compose. The hub becomes the expanding circular blur rather than a drawn disc.
20ad098	07-24	A consistent spacing system across the radial UI.
8a20189	07-24	unpile the bottom controls, respect the safe area, drop dead drawer UI. The safe-area inset arrives on every bottom-anchored fixed element. The floating-bottom stack is defined.
bfd84b6	07-24	hub opens/closes as one synchronous motion. The pre-centring step is removed; the orb, blur, dial and foot become one 500ms transition. The <code>centering</code> phase is deleted from the state machine.
0969119	07-25	delete the unreachable mobile push-drawer. Live code, in the bundle, with no path that opened it. Its scroll-blocking technique survives as comments.
4f18b76	07-26	Render the site in the Liquid Glass material — surfaces become translucent tints by token.
1b5c04a	07-26	metaball layer — fuse the radial chrome into liquid. <code>LiquidGoo</code> , the three-filter bank, the dial's art/hit split, the orb aura.
28e03b2	07-26	radial menu icons were white-on-white. The blend-mode glyph layer fails. Glyphs move inside their sectors.
454a762	07-26	Auto-contrast the logo mark, and a radial loading mark everywhere. The mark's ink becomes <code>--site-accent-fg</code> .
89496ed	07-26	make text and icons track the surface they sit on, site-wide. The systemic fix: zero-specificity <code>:where()</code> pairings in <code>globals.css</code> .
c631cb4	07-26	the radial menu was invisible on a light page. A tint of a tint. Sector fills rebuilt on <code>--site-surface-opaque</code> .
6e6c184	07-27	use the whole desktop window, and make the metaballs behave. The three-track frame; the pointer drop's blend-mode plate replaced with an SVG alpha ramp.
d308f26	07-27	spin the double-decked dial open, and draw the borders between levels. Counter-rotation, the light plate, the hairlines. Chapter 16.
4419355	07-28	Audit pass: P0 contrast failures, floating-chrome collisions, SSR drift. AUD-003's coordination extended to all widths.
feec2c	07-28	Audit pass: the remaining P2/P3 work. The centre caption, the foot pill's <code>min-width: 0</code> , the label budgets, the cursor swell cap.

27.1 The shape of the five days

Read as a curve, the history has three distinct phases, and the proportions are worth noting because they are typical and rarely admitted.

Day one: eight commits, the whole design. Shell, feed, hub, wedges, mark, glass, spacing, safe areas. Everything that this document's Parts I through III describe as the concept and the geometry existed by the end of 24 July.

Days three and four: seven commits, five of them fixes. The material landed, and with it the entire class of contrast failures. Four of the five fixes are the same bug wearing different clothes.

Day five: two commits, both audit remediation. Neither adds a feature. Both are lists of specific, measured, located defects with fix directions attached.

The design took a day. Making it correct in three themes, at thirteen viewports, with a home indicator and a cookie banner and a screen reader, took four more — and was not finished. That ratio is the honest summary of what building a control like this costs.

27.2 The commit that removed a state

One entry deserves attention because it is a deletion rather than an addition. `bfd84b6`, *"hub opens/closes as one synchronous motion"*, removed a phase from the state machine.

The original design opened in two beats: first the orb glided to the centre (`centering`), then the menu bloomed around it (`open`). That is the intuitive reading of the effect — cause, then consequence — and it is what the module's README still describes in one paragraph that was not updated.

It was worse. Two beats meant the orb arrived and then paused, and the pause read as latency rather than as choreography. It also doubled the state machine's complexity for the reversal case: interrupting during `centering` and interrupting during `open` are different situations, and both had to unwind correctly.

Collapsing them means the blur starts expanding while the orb is still travelling, which on paper is wrong — the menu begins before its origin has arrived — and in practice is much better, because both motions share the same front-loaded curve and therefore complete their visible portion together. The `Phase` type's comment still records the change: *"closed → open (orb glides to centre AS the menu blooms) → closing → closed."*

A NOTE ON DOCUMENTATION DRIFT

The README's description of the hub — *"tapping the fixed RMH orb glides it to the centre of the screen, then opens the menu"* — describes the two-beat version that was removed. It is a small, harmless inaccuracy, and it is a good illustration of why this record leans on the code and the commit messages rather than on prose descriptions: the component's own type comment was updated in the commit that changed the behaviour, and the README was not.

CHAPTER TWENTY-EIGHT

Six patterns

Every defect in this project is an instance of one of six patterns. None of them is specific to radial menus, and all six are worth carrying to the next component.

28.1 Derived contrast that depends on the layer tree

Instances: AUD-504 precursor (dial glyphs), the logo mark, the pointer drop's plate. **Commits:** 28e03b2 , 454a762 , 6e6c184 .

Blend modes composite against a stacking context, not against what you see. Any "automatic contrast" built on `mix-blend-mode` is a bet on a layer tree the compositor is entitled to change — and in the pointer drop's case, a bet that does not fail under software rasterisation, so testing cannot catch it.

The rule: name the paired foreground token, and put the ink inside the element that carries the background.

28.2 A tint of a tint

Instance: the whole dial invisible on a light page. **Commit:** c631cb4 .

In a material where surface tokens are *already* translucent, `color-mix(surface 60%, transparent)` is 60% of something that was already partly transparent. The expression looks like it produces a visible surface and produces almost nothing.

The rule: when a value must be visible against an unknown background, build it on the opaque companion token and *mix toward the ink* — a direction, not a colour.

28.3 A value whose correctness depends on being covered

Instance: the divider bed flashing black once the decks spun. **Commit:** d308f26 .

The bed was pure black, which was correct while the sectors covered it at every frame. Adding rotation and a sub-1 resting scale uncovered a 23-pixel ring of it for 550 milliseconds.

The rule: if a layer is only acceptable because something covers it, either write that assumption down at the declaration or make the layer safe to see. Adding motion to a layer changes the requirements of every layer beneath it.

28.4 Chrome that does not publish its footprint

Instances: AUD-003 and its eight dependents — the cookie banner, feed cards, the pricing scroll cue, the help chips, the settings rows, the shop tab strip, landscape viewport height. **Commits:** 8a20189 , 4419355 .

A fixed element is a claim on screen space. If that claim is not published in a form layouts can consume, every layout will eventually collide with it — and the collisions will be found one page at a time, by users.

The rule: publish the footprint as tokens (`--site-hub-orb-size` , `--site-hub-orb-inset`), derive clearance from those tokens rather than guessing (`--site-floating-reserve`), coordinate co-located chrome explicitly (the floating-bottom stack), and scope that coordination to the *elements* involved rather than to the viewport where the problem was first noticed.

28.5 Text boxes that spill in the constrained direction

Instance: AUD-504(c) — "Adaptive Intelligence" clipped, "Predictions" overflowing the disc, then "Communities" truncated by the over-correction. **Commit:** `feec2c`.

An axis-aligned label box inside a wedge grows angularly in width — across the clip — and radially in height, into the band, where there is room. The instinct is to widen the box; the correct move is to narrow it and let the text wrap.

The rule: identify which axis is bounded by the clip before choosing which dimension to spend. And when constraining a text box, prefer wrapping to truncation: a truncated label is a label that no longer names its destination.

28.6 Decoration in the same pixels as information

Instances: the goo filter chewing glyphs; the pointer drop swelling largest exactly over a control's label (AUD-503); the feed wheel's rake fading cards to 18% opacity (AUD-118); the full-viewport goo loader replacing the page and producing a 0.21 layout shift (AUD-302).

Four separate effects, one failure: a visual flourish placed where the reader's attention has to be. In each case the flourish was working exactly as designed.

The rule: a decorative effect gets its own layer, its own opacity floor, or its own cap. The dial split into art and hit layers; the cursor capped its diameter and thinned its fill; the wheel needed an opacity floor and a print reset; the loader needed skeletons that reserve its space. Same rule, four shapes.

Not one of these six is about radial menus. All six are about what happens when a correct value acquires a dependency nobody wrote down.

28.7 What the ledger says about process

Three observations, offered as the practical conclusion of this part.

The audit found what code review did not. Every one of these defects survived a commit that was written carefully, with comments, by someone who understood the system. What caught them was rendering forty routes at thirteen viewports in three themes and looking at the output. There is no substitute available.

The comments are the deliverable. The most valuable artefacts this project produced are not the 521 lines of component; they are the comment blocks explaining why the bed is light, why the glyphs live inside their sectors, why `touch-action` instead of a scroll lock, and why the pointer drop does not use the cheap CSS goo. Each one is a bug that cannot recur.

Systemic fixes beat instance fixes. `89496ed` — the `:where()` foreground pairings in `globals.css` — closed a whole class of contrast bugs site-wide rather than the two instances that had been reported. AUD-003's remediation order is explicit about the same principle: fix the coordination once and eight dependent findings heal.

PART EIGHT

Verification, and what is open

How the control is checked, what the checks cannot see, and the seven things that are still wrong with it.

29 How it is checked

30 What is still open

CHAPTER TWENTY-NINE

How it is checked

Ten CI workflows, a consistency test that fails the build on hand-rolled tab strips, a nine-point definition of done, and a manual floor of three themes by two widths. None of it would have caught most of Part VII.

29.1 The automated layer

WHAT RUNS, AND WHAT IT CAN SEE ABOUT THE HUB

CHECK	CATCHES
tsc --noEmit	The wedge type, the phase union, the props of every primitive the hub composes. Real value: the <code>Phase</code> type made removing the <code>centering</code> state a compile-time exercise.
eslint (jsx-ally at warn)	Missing <code>aria-label</code> on icon-only controls, missing <code>aria-hidden</code> on decorative icons. The rule is "add no new warnings relative to base".
vitest -- design-consistency	Hand-rolled tab strips and stray active-underline markers, build-failing. Not about the hub directly; it is the CI expression of the "one grammar" rule.
vitest -- i18n coverage	Namespaces registered, keys present. New English strings must be added to the <code>en</code> catalogs and registered in the suite's known-untranslated allowlist.
prettier	Formatting only.
synthetic-perf	Web Vitals over time. This is what would surface something like AUD-302's 0.21 layout shift.
senior-review	An LLM review gate on the diff.
build	That the module compiles into the six server bundles and the client build.

Read that table honestly and the conclusion is uncomfortable: **not one of these checks could have caught a single defect in Chapter 28**. White-on-white glyphs are valid CSS. A tint of a tint type-checks. A black plate uncovered by a rotation compiles. A mobile-only media query around the floating-stack coordination is well-formed. A label box clipped by a wedge's `clip-path` is not a lint error.

Everything in Part VII was found by rendering the page and looking at it.

29.2 The manual floor

Which is why the platform's definition of done ends with a rendering requirement rather than a test. Its ninth point: the change *was actually looked at* in `default`, `.style-light` and `.style-high-contrast`, at a phone width and a desktop width, and once with reduced motion on. Three themes by two widths is stated as the floor, with the audit's thirteen-viewport matrix as the extended version.

For this component specifically, the module README adds two of its own: verify the hub bloom and the wheel on a 320-pixel-wide screen when changing geometry, and check that nothing leaves the viewport.

29.3 The audit as an instrument

The July audit is the closest thing the project has to a systematic check, and its design is worth describing because it is reusable.

Forty-plus routes at thirteen viewports in three themes, plus reduced motion, plus first-run overlays, plus scripted interactions — the hub's bloom among them, captured deliberately rather than incidentally. Every finding names the route, the viewport, the evidence capture, the suspected source file with line numbers, the design principle violated,

and a fix direction. Findings carry stable identifiers, and a remediation order that groups them by root cause.

Two properties make it more than a screenshot dump. It is **addressed to agents as well as humans**, which forces every finding to be independently actionable — no "the spacing looks off here". And it records **its own harness incidents**: overlays that blocked captures, seven captures that silently rendered signed-out, and the hub captures that skipped because of the accessible-name drift in Chapter 23. An audit that does not report its own failures is an audit whose gaps are invisible.

THE GAP THIS LEAVES

An audit is a snapshot. It ran once, on 28 July, and two commits later the findings it produced were substantially fixed — at which point the instrument was spent. Nothing in CI re-renders the hub at thirteen viewports on every pull request. So the class of defect that dominates Part VII is, today, caught by a process that runs when somebody decides to run it. That is the single most consequential open item in this record, and it is not on the list in Chapter 30 because it is not about this control.

29.4 What a real regression suite would need to assert

Offered as a specification, since the analysis is already done. A useful automated check on this component would assert, per theme and per viewport:

1. Every wedge's computed `color` meets 4.5:1 against its own computed background, in resting, hover, focus and active states.
2. The dial's bed and sector fills differ by a measurable amount, so the dividers are visible.
3. No wedge label's rendered box exceeds its sector's clip bounds.
4. Every wedge's hit region is at least 44 pixels in its smaller dimension — which would currently fail at 320px, correctly.
5. The orb's rect does not intersect the rect of any other floating control, or of any element the page marks as content.
6. Nothing in the shell overflows the viewport horizontally at 320px.
7. With reduced motion, the dial's computed transform at rest equals its computed transform when open.
8. The orb's accessible name equals the shipped catalog string, not the `defaultValue`.

Eight assertions, all mechanically checkable in a headless browser, and between them they cover five of the six patterns in Chapter 28. The sixth — decoration in the same pixels as information — needs a human.

CHAPTER THIRTY

What is still open

Seven things, in rough order of how much they matter. The first is the one that could invalidate the whole design, and it is the one nobody has solved.

30.1 Discovery — no first-run hint

Below 1120 pixels, the entire navigation of a platform with sixteen destinations is one unlabelled circular mark. AUD-118 records it; nothing has shipped.

This is the item that could make the rest of this document moot. Every argument in Part I — thumb reach, collapsed controls, priority encoded as radius — presupposes that the user opens the dial. A user who does not recognise the orb as a menu has a worse experience than they would have had with a hamburger, because at least a hamburger is a learned convention.

Why it is hard: too quiet and the hint is decoration; too loud and it is an interstitial, on a platform that already has enough first-run overlays that the audit had to suppress several to capture anything. The audit's own suggestion — a transient "Menu" caption below the orb on first visit — is probably right, and the open questions are how long it persists, what dismisses it, and whether it re-appears for a signed-out visitor who has cleared storage.

30.2 Keyboard semantics

The dial is `role="menu"` with `menuitem` children and no arrow-key handling, no focus trap, and no focus restore on close. Chapter 23 states each gap. The honest position is that it currently behaves like a labelled list of links, which is usable and is not what its own ARIA role promises.

The design question underneath is real: on a two-deck ring, what should Up and Down do? Move between decks is the obvious answer, and Left/Right rotating within a deck is the other half, but that means Right at the last sector wraps to the first — and a wrapping tab order inside a document that does not wrap is its own confusion.

30.3 Touch targets at 320 pixels

Wedges are 41 pixels deep on the inner deck and 38 on the outer at the smallest supported width, against a 44-pixel floor. Chapter 22 gives the mitigating argument — the arc dimension is 55 to 76 pixels, and a miss lands in a drawn gap rather than on a neighbour — and it is a mitigating argument, not compliance.

The obvious fix is to raise the dial's `86vw` below 360 pixels. The unresolved part is the foot pill: at 320×568 the dial and the pill already have little room between them, and growing the dial pushes them together. Nobody has worked through whether the pill should shrink, move, or overlap the dial's rim at that width.

30.4 Landscape phones

AUD-105: at 844×390, the top bar, cookie bar and orb between them left roughly 150 pixels of usable content height. The audit's note is that this needs a *design decision* rather than a fix — collapse the orb to a corner under a `max-height` condition, or shrink the chrome, or accept it.

The dial itself is fine in landscape; `70vh` binds and it renders at 273 pixels. It is the docked state that is the problem, and specifically the interaction of three independently-reasonable pieces of fixed chrome on a viewport 390 pixels tall.

30.5 Right-to-left locales

Sector order does not mirror in the two RTL locales, and nobody has tested whether it should. Chapter 10's note stands: the correspondence between reading direction and rotational direction is a convention, not a rule, and clocks run clockwise everywhere. But "we did not think about it" and "we decided it was right" are different states, and this is the former.

30.6 The parked lens optic

The platform's SVG displacement lens — the effect that bends the backdrop through a height field at a pane's edge — is parked, because current Chromium composites the displacement map into the bevel rather than bending the backdrop through it. The filters and the supporting module still ship; re-enabling is restoring two `@supports` upgrades and one initialisation call.

The hub does not use it and would not obviously benefit. It is listed because anyone reading the design language will find `.glass-refract` documented as live and should know that one optic in it is not.

30.7 A continuous visual check

Covered in Chapter 29 and repeated here because it is the item most likely to determine whether the next twenty commits produce another Part VII. The audit was an instrument that ran once. The defects it found were the kind that only a rendered page reveals. There is no per-pull-request equivalent.

30.8 What is not on this list

Three things that might be expected and are deliberately absent.

Performance. Nothing in the hub is a measured performance problem. The expensive things are gated, the animation is compositor-only, and the one full-viewport `backdrop-filter` exists for a few seconds when the user asks for it.

The two-tap cost of going home. Chapter 5 defends it, and the defence stands: one unambiguous behaviour on the site's only universal control is worth more than one saved tap on a destination the user is usually already at.

The radial shape itself. This is the thing a reader might most expect to be hedged, and it is not. The geometry earns its place: it collapses three controls into one, it encodes priority as radius, it costs no layout space, it derives itself from a catalog, and it puts every target inside a thumb's sweep. If the design fails, it will fail on §30.1 — because people did not know it was there — and not because a dial was the wrong shape for what it does.

**The open questions are about discovery, keyboards and small screens.
None of them is about the circle.**

APPENDICES

Reference

The geometry table, the constants, the navigation catalog, the token contract, the commit and finding indexes, and a glossary.

A Geometry reference
B Constants and durations
C The navigation catalog

D Finding index
E Glossary

APPENDIX A

Geometry reference

Computed by the same port of `carve()` that draws every figure in this book, for the shipped sixteen-destination dial. Angles are degrees clockwise from twelve o'clock; radii and centroids are percentages of the dial's width.

THE SIXTEEN SECTORS, AS CARVED

#	DESTINATION	DECK	A ₀	A ₁	SPAN	CX	CY
0	Home	0	-89.30	-30.70	58.60	63.25	27.05
1	Explore	0	-29.30	29.30	58.60	76.50	50.00
2	Inbox	0	30.70	89.30	58.60	63.25	72.95
3	Creator Studio	0	90.70	149.30	58.60	36.75	72.95
4	Library	0	150.70	209.30	58.60	23.50	50.00
5	Communities	0	210.70	269.30	58.60	36.75	27.05
6	Store	1	-89.30	-54.70	34.60	63.29	9.10
7	Arcade	1	-53.30	-18.70	34.60	84.79	24.73
8	Predictions	1	-17.30	17.30	34.60	93.00	50.00
9	Developer	1	18.70	53.30	34.60	84.79	75.27
10	Services	1	54.70	89.30	34.60	63.29	90.90
11	RMH Capital	1	90.70	125.30	34.60	36.71	90.90
12	RMH PMC	1	126.70	161.30	34.60	15.21	75.27
13	Adaptive Intelligence	1	162.70	197.30	34.60	7.00	50.00
14	RMH Deeplink	1	198.70	233.30	34.60	15.21	24.73
15	Admin	1	234.70	269.30	34.60	36.71	9.10

Boundary radii handed to CSS: `[19, 34, 36, 50]`. Angles are degrees clockwise from twelve o'clock, where twelve o'clock is -90° in CSS coordinates; radii and centroids are percentages of the dial's width. Inner deck: six sectors, 60° nominal, 1.40° gap, band 19–34, mid-radius 26.5. Outer deck: ten sectors, 36° nominal, 1.40° gap, band 36–50, mid-radius 43.0. Centre-hole mask: `calc(38% - var(--dial-level-w))`.

SINGLE-RING MODE — THE SAME FUNCTION, SIX DESTINATIONS

#	DESTINATION	DECK	A ₀	A ₁	SPAN	CX	CY
0	Home	0	-89.30	-30.70	58.60	67.25	20.12
1	Explore	0	-29.30	29.30	58.60	84.50	50.00
2	Inbox	0	30.70	89.30	58.60	67.25	79.88
3	Creator Studio	0	90.70	149.30	58.60	32.75	79.88
4	Library	0	150.70	209.30	58.60	15.50	50.00
5	Communities	0	210.70	269.30	58.60	32.75	20.12

Boundary radii: `[19, 50]` — two, not four, so the dial draws two hairlines and the mask still reads `bounds[0]`. Every centroid sits at mid-radius 34.5.

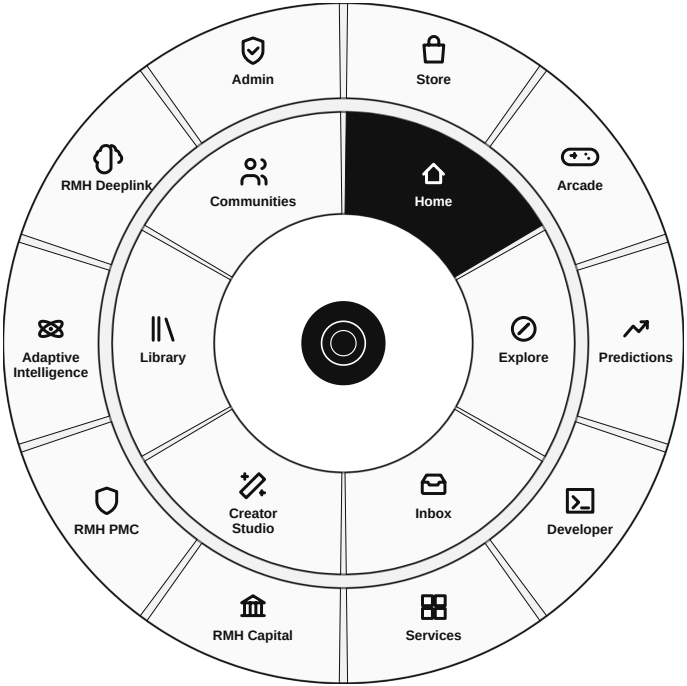


Figure — The shipped dial, signed in as an admin: sixteen destinations, six on the inner deck, ten on the outer. Drawn by the production `carve()`.

APPENDIX B

Constants and durations

EVERY CONSTANT IN RADIALHUB.TSX

NAME	VALUE	MEANING
RINGS[0]	19 → 34	Inner band, as % of dial width.
RINGS[1]	36 → 50	Outer band.
SINGLE_RING	19 → 50	The one band used at six destinations or fewer.
ART_BLEED_DEG	0.25	Fraction of the gap the art layer is widened by, angularly, each side.
SINGLE_RING_MAX	6	At or below this, one ring rather than two.
INNER_MAX	6	Cap on the inner deck — the shorter circumference.
MOTION_MS	500	The one duration, and the unmount gap on close.
DEG	$\pi/180$	Degrees to radians.
steps	10	Arc samples per side in <code>ringSector()</code> .
gap	<code>min(1.4, seg×0.06)</code>	Angular divider between sectors.

CSS CUSTOM PROPERTIES DECLARED BY THE DIAL

PROPERTY	VALUE	CONSUMER
--dial	<code>min(86vw, 70vh, 460px)</code>	The dial's width and height.
--dial-hole	<code>calc(38% - level-w)</code>	The mask's transparent radius. Supplied inline by the component.
--dial-level-w	2px	Hairline weight, and the mask's straddle correction.
--dial-level-c	<code>text @ 55%</code>	Hairline colour.
--ring-spin	<code>-55deg / +55deg</code>	Per-deck resting rotation.
--fade-delay	<code>calc(i × 20ms)</code>	Per-wedge opacity stagger, on open only.
--i	0...15	Bloom order across both decks, inner first.
--r	19/34/36/50%	One per hairline, from <code>bounds</code> .
--orb-bottom	<code>safe-area + inset</code>	The docked offset, and a term in the centring transform.

DECLARED TRANSITION DURATIONS

LAYER	PROPERTY	DURATION	CURVE
orb	transform	500ms	ease-glass
orb	box-shadow	280ms	ease
aura	transform	500ms	ease-glass
docked scrim	opacity	280ms	ease
blur	clip-path	500ms	ease-glass
dial	opacity	400ms	ease-glass
dial	transform	500ms	ease-glass
wedge wrapper	opacity	340ms	ease + stagger
wedge wrapper	transform	550ms	ease-glass
hairline	opacity	340ms	ease
hairline	transform	550ms	ease-glass
wedge link	background, color	180ms	ease
foot	opacity	400ms	ease
foot	transform	500ms	ease-glass
foot button	background, transform	180ms	ease
aura blob	orbit keyframe	7000ms	ease-in-out, -2.33s stagger

DECLARED TRANSITION DURATIONS (CONT.)

LAYER	PROPERTY	DURATION	CURVE
logo	bloom keyframe	1050ms	ease-glass, once on mount
logo	stroke draw	1300ms	ease, once on mount

--ease-glass is `cubic-bezier(0.16, 1, 0.3, 1)`, with that literal as the fallback at every use site.

APPENDIX C

The navigation catalog

`lib/sidebar-nav.ts`, flattened the way the hub flattens it: groups are expanded into their children, then auth and admin gating is applied. The `id` is the stable customisation key that users' saved order and hidden sets validate against — renaming one is a data migration.

SIDEBAR_NAV, AS THE DIAL CONSUMES IT

#	LABEL	ID / HREF	I18N KEY	ICON	GATE
0	Home	/	nav-home	Home	—
1	Explore	/search	nav-explore	Compass	—
2	Inbox	/messages	nav-inbox	Inbox	auth · badge
3	Creator Studio	/create	nav-creator-studio	Wand2	—
4	Library	/library	nav-library	Library	—
5	Communities	/communities	nav-communities	Users	—
6	Store	/store	nav-store	ShoppingBag	—
7	Arcade	/arcade	nav-arcade	Gamepad2	—
8	Predictions	/predictions	nav-predictions	TrendingUp	—
9	Developer	/developer	nav-developer	Terminal	—
10	Services	/services	nav-services	LayoutGrid	—
11	RMH Capital	/rmh-capital	nav-rmh-capital	Landmark	group child
12	RMH PMC	/rmh-pmc	nav-rmh-pmc	Shield	group child
13	Adaptive Intelligence	/adaptive-intelligence	nav-adaptive-intelligence	Atom	group child
14	RMH Deeplink	/deeplink	nav-rmh-deeplink	Brain	group child · external
15	Admin	/admin	nav-admin	ShieldCheck	admin · badge · unorderable

Two structural notes. Items 11–14 are the children of the `group:ventures` group, which the hub flattens because a dial has no notion of a collapsible section — a group's children become peers on the ring. And Admin's `id` is deliberately absent from the orderable set, so it always falls to the end of the outer deck and can never be hidden.

The badges — inbox count, pending admin review — render in the desktop rail and not on the dial. A sector has nowhere to put a count that would not collide with its label's angular budget, which is the one capability the rail has that the dial does not.

APPENDIX D

Finding index

AUDIT FINDINGS CITED IN THIS RECORD

ID	P	SUMMARY	CH.
AUD-003	1	The fixed orb occludes content and collides with other floating chrome. Eight dependents.	3, 26, 28
AUD-102	1	Cookie banner vs orb vs compose FAB at ≥768px, every route. The coordination was mobile-only.	3, 26
AUD-104	1	Orb occludes feed content on portrait phones; <40px clearance after the last card.	3, 26
AUD-105	1	844×390 landscape: floating chrome consumes ~60% of viewport height.	3, 30
AUD-111	1	Pricing hero's "Compare tiers" cue rendered inside the orb's footprint — never visible.	3
AUD-118	3	Sub-1120px navigation is a single unlabelled orb with no first-run hint. Also the wheel's 18% ghost cards.	3, 24, 28, 30
AUD-203	1	/help's bottom-pinned concierge chips sit under the orb permanently.	3
AUD-302	1	Phone loads swap a full-viewport goo loader for the page — CLS 0.13–0.21.	7, 28
AUD-313	2	Further orb occlusion: settings hex input, notification row, messages list.	3
AUD-315	2	Accent swatches: "Ink" indistinguishable from "Theme default" on the default theme.	21
AUD-322	1	/shop: category tab strip lands under the orb at 390px.	3
AUD-503	2	The metaball cursor occludes exactly what it points at — swells ×1.85 over labels.	3, 22, 28
AUD-504	2	Hub dial: (a) mobile is sixteen unlabelled glyphs; (b) avatar paints over the settings gear; (c) long labels clip at the wedge edge.	3, 12, 24, 26

COMMITTS CITED

COMMIT	SUBJECT	CH.
e5003b1	radial front-end rewrite — monochrome RMH radial UI	2, 27
bedf361	wedge nav, metaball cursor, auto-contrast text, mobile polish	27
f4a510f	RMH logo mark in the hub orb, with a stroke-draw bloom	7, 27
426d1d1	avant-garde glass hub + document-scroll feed + inline compose	27
8a20189	unpile the bottom controls, respect the safe area, drop dead drawer UI	26, 27, 28
bfd84b6	hub opens/closes as one synchronous motion	27
0969119	delete the unreachable mobile push-drawer	2, 25, 27
1b5c04a	metaball layer — fuse the radial chrome into liquid	19, 27
28e03b2	radial menu icons were white-on-white	20, 27, 28
454a762	auto-contrast the logo mark + a radial loading mark everywhere	20, 27, 28
89496ed	make text and icons track the surface they sit on, site-wide	20, 27, 28
c631cb4	the radial menu was invisible on a light page	6, 20, 27, 28
6e6c184	use the whole desktop window, and make the metaballs behave	27, 28
d308f26	spin the double-decked dial open, and draw the borders between levels	16, 27, 28
4419355	P0 contrast failures, floating-chrome collisions, SSR/schema drift	27, 28
feec2c	settings, rail, cursor and hub-dial defects	27, 28

APPENDIX E

Glossary

Annulus sector

A ring segment: the region between two radii over a span of angles. The dial's target shape, chosen over the pie slice for the four reasons in Chapter 8.

Art layer

The decorative, goo-filtered, `aria-hidden` copy of the sector geometry. Carries the fills and nothing readable.

Bed

The dial's own background, visible through the angular gaps between sectors, which is what the dividers are made of.

Bleed

The angular widening applied to the art layer to pay back the edge erosion caused by the goo filter's alpha threshold.

`ART_BLEED_DEG = 0.25` of the gap, each side.

Bloom

The open animation, collectively. Also, separately, the mark's one-shot mount animation.

Deck

One of the dial's two concentric rings. Inner deck = the first six catalog entries.

Foot pill

The floating capsule at the bottom of the open overlay: identity, settings, sign-out, close — or a sign-in call to action when signed out.

Goo

The metaball effect: blur, then a steep ramp on alpha, then the sharp original blended back. Three radii, in

`LiquidGoo`.

Hit layer

The real, unfiltered links — one per destination, each clipped to its exact sector, each containing its own icon and label.

Hole

The masked-out centre of the dial, radius 19, which the orb glides into. Also where the centre caption lives below 480px.

Hub

The whole control: orb, aura, scrim, overlay, blur, dial and foot pill.

Level hairline

One drawn circle per band boundary, so the decks are separated by an edge rather than by an absence of fill.

Liquid Glass / Radial Avant-Garde Glass

The platform's material and, with the radial architecture, its design language.

Orb

The fixed, bottom-centre disc carrying the RMH mark. The trigger, and the centre the dial radiates from.

Phase

`closed` | `open` | `closing`. One attribute on the wrapper; the entire animation state.

Ring / band

Used interchangeably for a deck's radial extent. `RINGS` in the source; "deck" in this document where the distinction from a single ring matters.

Token

A `--site-*` CSS custom property that every theme redefines. The contract the hub is built out of.

Wedge

One destination's sector, in either layer. Sixteen of them, signed in as an admin.

SOURCES

components/radial/RadialHub.tsx · components/radial/radial.css ·
components/radial/LiquidGoo.tsx · components/radial/RmhLogo.tsx ·
components/radial/README.md · lib/sidebar-nav.ts · docs/design-language.md · docs/page-
consistency.md · docs/ui-audit-2026-07-28.md · CLAUDE.md and the per-directory guides · commits
e5003b1 through feeec2c.